claude-windows / 45ea3a91-654d-4972-9cd3-65f35db54caf

Metadata

- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\45ea3a91-654d-4972-9cd3-65f35db54caf\subagents\workflows\wf_2a64bc88-2a8\agent-a67e419dad1b493da.jsonl`
- SHA-256: `e6295d4f70cd30b70e5747f391e5e9c54cf730f5707f62abe9c0782a3fa48bf5`
- Source modified: `2026-06-10T05:44:08+00:00`
- Imported at: `2026-07-05T16:48:27+00:00`
- project: `wf_2a64bc88-2a8`
- session_id: `45ea3a91-654d-4972-9cd3-65f35db54caf`

Transcript

1. user (2026-06-10T05:33:14.936Z)

CONTEXT ? two sibling repos on a Windows machine:
- INDEXER: C:/Users/avidu/Projects/badminton-highlight-indexer ? local AI badminton(->racket-sports) highlight indexer + rally detector. FastAPI + SQLite + ffmpeg + GPU CV via WSL; Gemini = paid cloud AI. docs/ tree is rich. Current branch docs/next-steps-multisport (master = main branch).
- COLLECTOR: C:/Users/avidu/Projects/sports-data-collector ? golden-label acquisition/curation/ingestion CLI (system-of-record candidate for training corpus).
Project state (June 2026): scaffolding complete; owned-model roadmap agreed (docs/OWNED_MODEL_IMPLEMENTATION_PLAN.md) but NO platform/owned-model implementation started; next committed platform slice = async job model. Licensing guardrail: MIT/Apache-2.0/BSD only for code+data+weights; paid LLMs are inference-only, never training-data sources.
ENVIRONMENT RULES: this dev environment has NO GPU/torch/cv2 ? do NOT attempt to run the video pipeline or import torch/cv2 modules. Test suites are offline/fully-mocked and SAFE to run. You are READ-ONLY: never edit/create/delete repo files; running tests and git read commands is allowed.
QUALITY BAR: cite file paths (and line numbers where possible) for every finding. Report what IS in the code/docs, not what you assume. Be skeptical and concrete. Your final structured output is consumed programmatically by an orchestrator ? return raw data, not prose for a human.
TASK: full audit (engineering + testing + reliability + docs) of the COLLECTOR repo at C:/Users/avidu/Projects/sports-data-collector.
Steps: read README.md and all of docs/ (DECISIONS.md, PIPELINE_ROLE.md, INGESTION_PIPELINE.md, MANUAL_INGESTION_POSITIONING.md, annotation/*). Map the src/ tree (Glob src/**/*.py) and deep-read the core: src/cli/curate.py, src/cli/normalize_annotations.py, src/parsers/ (esp. cvat/rally_cvat.py), the db layer, models/validation, reporting/. RUN the test suite (pytest from repo root ? offline, safe); capture real counts.
ASSESS: (1) engineering quality and architecture vs its stated role as the SYSTEM-OF-RECORD for the training corpus (docs/MANUAL_INGESTION_POSITIONING.md) ? is it actually ready for that role? what's missing (provenance, maturity ladder, labeled_window persistence ? check which of these exist TODAY vs are documented as P1/P2); (2) known landmines: import-local DELETE-then-INSERT destructiveness, int(float()) rally_number PK behavior, pydantic abort-whole-import semantics, the normalize_annotations guard coverage; (3) the indexer<->collector CONTRACT: does "curate export" output (manifest.json + per-video start,end CSV) actually match what the indexer's backend/eval/batch.py + run_phase3_eval.py consume (read those indexer files to cross-check field names/paths/Windows path normalization)? does convert-cvat's ATTR_MAP handle the real Roora CVAT attribute names (spaces, "(A-B)")? (4) test coverage gaps and quality; (5) reliability: what happens on partial/corrupt CVAT zips, duplicate imports, fps mismatch. Severity-rate as if the training corpus' integrity is on the line (it is ? this feeds the owned model).

2. user (2026-06-10T05:33:21.275Z)

README.md
__pycache__

cleanup.py
config
data
docs
pyproject.toml
src
tests

Sports Data Collector & Curator

A clean, extensible Python codebase and CLI toolchain designed to discover, ingest, validate, and curate sports video datasets. The repository stores annotations and video metadata in an SQLite database, supports sport-specific tables and schemas, manages video storage outside of Git, and enforces strict commercial license compliance.

> **Pipeline role:** this repo is the *data producer* feeding the **badminton-highlight-indexer**, which trains/evaluates an offline rally segmenter on this curated, commercially-licensed data. See docs/PIPELINE_ROLE.md for why `export`/`fetch` exist and how the two repos fit together.

Features

- **SQLite Backend**: Centralized `sports_metadata.db` with a generic `videos` table and sport-specific annotation tables (`badminton_rallies`, `tennis_points`, `cricket_deliveries`, `pickleball_rallies`, `tabletennis_rallies`).
- **Commercial License Flagging**: Each ingested source license is checked against a configured whitelist (`MIT`, `Apache-2.0`, `BSD`, `CC0`, `CC-BY`, `Public-Domain`, `Proprietary`) and the resulting `is_commercial` flag is stored alongside the video. Non-commercial sources (like `CC-BY-NC-SA`) are flagged as non-commercial but still ingested ? nothing is discarded automatically. The whitelist is read from `config/config.yaml` consistently across the parser, validator, and CLI.
- **Overlap & Duration Validation**: Ensures chronological event bounds (e.g. rally start/end times) do not overlap and that durations are positive.
- **Stroke-level CSV Parser**: Ingests ShuttleSet/CoachAI-style stroke CSV logs and aggregates them into rally durations using configurable start/end time buffers.
- **Local Ingestion Mode**: Manually registers local videos and associated CSV/JSON annotation files directly into the database as curated entries.
- **Interactive Curation CLI & Visual Play Harness**: Let's you query DB status, list staging entries, review sample timestamps, and approve/reject them. It includes a built-in play function that opens the video in your default browser at the exact rally segment (using HTML5 media fragments `#t=start,end` for local files or YouTube start time markers `&t=start` for remote videos) for manual visual verification.

Installation

Ensure you have Python 3.9+ installed.

1. Clone or download the repository.
2. Install the package and dependencies:
   ```bash
   pip install -e .            # core: curate / import-local / export / status
   pip install -e ".[fetch]"   # adds yt-dlp for video acquisition (fetch / crawler)
   pip install -e ".[dev]"     # test tooling (pytest, pytest-cov)
   ```

External requirements (not pip-installable)

The `fetch` command and the ShuttleSet crawler shell out to external binaries.
They degrade gracefully (a clear error / skipped download) when missing, but are
required for video acquisition:

- **FFmpeg / ffprobe** ? used to probe the real resolution/fps of downloaded files.
- **A JS runtime (e.g. [deno](https://deno.com))** ? needed by yt-dlp to solve
  YouTube's challenge for high-resolution formats. `fetch` auto-detects a deno
  install at `~/.deno/bin`.

Core curation/import/export/status commands work without any of these.

Usage

All commands should be executed from the root of the project. Make sure `PYTHONPATH` is set to
the current directory:

```bash
# On Windows PowerShell:
$env:PYTHONPATH="."
```

1. View Database Status
To check the current statistics of staging vs. curated items:
```bash
python -m src.cli.curate status
```

2. Ingest External Datasets to Staging
To parse external raw stroke-level datasets (e.g. CoachAI format) and register them in staging:
```bash
python -m src.cli.ingest --sport badminton --parser coachai --file-path
data/staging_coachai_match.csv --license CC-BY-NC-SA --video-id staging_match_non_comm
```

3. Launch the Interactive Curation Tool & Play Harness
To review staging items, inspect their timestamps, play individual rallies in the browser to
visually verify their start and end bounds, and approve or reject them:
```bash
python -m src.cli.curate curate
```

4. Manually Import Local Videos & Annotations
To register a local video file alongside its JSON/CSV annotation file (bypasses staging):
```bash
python -m src.cli.curate import-local --sport badminton --video-path
"data/videos/my_custom_match.mp4" --annotations-path "data/mock_local_badminton_anno.json"
--match-name "My Custom Local Match" --license Proprietary
```
`import-local` works for every configured sport ? `badminton`, `tennis`, `cricket`,
`pickleball`, and `tabletennis` ? from either a JSON or a CSV annotations file. For the
canonical rally CSV schema and the rater-facing instructions used to produce these files (e.g.
via an annotation vendor), see [`docs/annotation/`](docs/annotation/).

5. Fetch / Upgrade Videos to Best Resolution
Videos are **always downloaded at the best resolution available** (via yt-dlp `bv*+ba/b -S
res,...`, merged to MP4); the actual width/height/fps is ffprobed and stored ? never assumed.
The `fetch` command (re)downloads videos for entries already in the DB and updates
`local_path`/`resolution`/`fps` **in place**, preserving all rally annotations and curation
status (no clean slate needed):
```bash
# Upgrade every sub-720p video to best resolution, using a logged-in browser's cookies
python -m src.cli.curate fetch --cookies-from-browser firefox

# A single video, forcing a re-download even if a good local copy exists:
```

```
python -m src.cli.curate fetch --video-id shuttleset_1 --force --cookies-from-browser firefox
```
Flags: `--sport`, `--video-id`, `--status {staging,curated,all}` (default `all`), `--min-height` (skip videos already at/above it unless `--force`; default `720`), `--max-height` (cap resolution, e.g. `1080`; default best available), `--force`, `--include-noncommercial`, `--cookies-from-browser`/`--cookies`, `--player-client`. A global resolution cap can also be set via `video_storage.max_height` in `config/config.yaml`. Re-runs are cheap and resumable; a download that falls below `--min-height` is treated as a **failure** (discarded, DB left unchanged) so a flaky low-res result never overwrites a good entry.

> **Authenticated cookies are strongly recommended.** Anonymous YouTube downloads are throttled and non-deterministic (often falling back to 144p). Pass `--cookies-from-browser firefox` (or `chrome`/`edge`/`brave`, fully closed) ? or `--cookies cookies.txt` ? to use a logged-in (ideally Premium) session for fast, reliable best-resolution downloads. Chrome's app-bound cookie encryption may block direct reads on Windows; Firefox reads cleanly even while open.
>
> **JS runtime:** yt-dlp needs one (e.g. [deno](https://deno.com)) to solve YouTube's challenges for high-res formats; `fetch` auto-detects a deno install at `~/.deno/bin`. The default `--player-client` avoids `web`/`web_safari`, which YouTube forces into SABR streaming (drops to 144p).

6. Export Annotations as Indexer-Ready Eval/Training Sets

To turn curated annotations into ground-truth files the badminton-highlight-indexer can score and train against, export per-video `start,end` CSVs plus a `manifest.json`:

```bash
# Curated + commercially-safe videos across all supported sports (default):
python -m src.cli.curate export --out-dir data/eval_export

# One sport, also including staging entries:
python -m src.cli.curate export --sport badminton --include-staging
```

The exporter writes `<out-dir>/<sport>/<video_id>.csv` (decimal-second `start,end` rows) and a `manifest.json` pairing each CSV with its video `local_path`/`video_url`, fps, resolution, and license. The indexer keys videos by **file MD5**, not by our `video_id`, so the downstream eval/training run must process the exact video file named in the manifest. Flags: `--sport` (restrict to one sport), `--include-staging` (default: curated only), `--include-noncommercial` (default: commercial-safe only).

Configuration

Adjust settings in `config/config.yaml`:
- **`database_path`**: Path to the SQLite database.
- **`video_storage`**: Local folder path and automatic download size limit.
- **`commercial_licenses`**: Whitelist of licenses allowed for commercial use.
- **`supported_sports`**: Allowed sports categories.

Documentation

- docs/PIPELINE_ROLE.md ? how this repo feeds the **badminton-highlight-indexer** (`export` / `fetch`).
- docs/DECISIONS.md ? collector-local decision records (**ADR-001**: keep explicit per-sport code instead of a registry, with an "adding a new sport" checklist).
- docs/annotation/ ? the vendor annotation package used to produce golden rally labels:
  - [ROORA_BRIEF.md](docs/annotation/ROORA_BRIEF.md) ? vendor brief + the canonical rally CSV schema.
  - [ANNOTATION_GUIDE.md](docs/annotation/ANNOTATION_GUIDE.md) ? the illustrated rater guide (singles & doubles).

```
  - [INTAKE_MANIFEST_TEMPLATE.csv](docs/annotation/INTAKE_MANIFEST_TEMPLATE.csv) ? per-video
intake sheet.
  - [NDA_TEMPLATE.md](docs/annotation/NDA_TEMPLATE.md) ? mutual NDA template (have counsel
review).

---
```

Running Tests

```
We use `pytest` for unit and integration testing:
```bash
python -m pytest tests/
python -m pytest tests/ --cov=src --cov-report=term-missing   # with coverage (~87%)
```
```

3. user (2026-06-10T05:33:22.070Z)

```
src\core\schemas\tennis.py
src\core\schemas\cricket.py
src\parsers\base_parser.py
src\core\downloader.py
src\cli\ingest.py
src\core\config.py
src\core\parsing.py
src\core\schemas\base.py
src\core\validator.py
src\crawlers\badminton_crawler.py
src\parsers\badminton\coachai.py
src\parsers\cvat\__init__.py
src\core\db.py
src\core\schemas\badminton.py
src\core\schemas\pickleball.py
src\core\schemas\tabletennis.py
src\parsers\cvat\rally_cvat.py
src\cli\curate.py
src\reporting\__init__.py
src\reporting\adapter.py
src\cli\normalize_annotations.py
```

4. user (2026-06-10T05:33:22.660Z)

```
docs\annotation\INTAKE_MANIFEST_TEMPLATE.csv
docs\annotation\NDA_TEMPLATE.md
docs\DECISIONS.md
docs\PIPELINE_ROLE.md
docs\annotation\ANNOTATION_GUIDE.md
docs\annotation\ROORA_BRIEF.md
docs\INGESTION_PIPELINE.md
docs\MANUAL_INGESTION_POSITIONING.md
```

5. user (2026-06-10T05:33:28.165Z)

```
1       # Decision records
2
3       Lightweight ADRs for this repo. (The cross-repo training-pipeline ADRs live in
4       the indexer repo's `docs/DECISIONS.md`; this file is for collector-local choices.)
5
6       ---
7
8       ## ADR-001 ? Keep explicit per-sport code instead of a sport registry (for now)
9
10      **Status:** Accepted · **Date:** 2026-06-02
11
12      ### Context
```

````
    Supporting a sport currently requires parallel edits across several sites:
    - `SEGMENT_TABLES` + a `CREATE TABLE` + `insert_*` / `get_*` in `src/core/db.py`
    - a `validate_*` method in `src/core/validator.py`
    - `import-local` parse/validate/insert branches and interactive review/play in
`src/cli/curate.py`
    - a schema in `src/core/schemas/`

    A code review flagged this as copy-paste duplication and proposed a registry/plugin
    mechanism (e.g. `SPORT_REGISTRY: dict[SportType, SportSpec]` driving generic
    insert/get/validate/dispatch) to make adding a sport a single entry.

    ### Decision
    **We are keeping the explicit per-sport code and NOT introducing a registry
    abstraction at this time.**

    ### Rationale
    - **It's well-tested and mechanical.** Coverage is ~87%; adding a sport is a known,
      ~30-minute "mirror an existing sport" task (checklist below), not a research problem.
    - **Schemas are heterogeneous.** Rally sports (badminton, pickleball, table tennis)
      share `rally_number / shots_count / ending_reason`, but tennis (`set_score`,
      `game_score`, `server`, `ending_type`) and cricket (`over`, `ball`, `runs`,
      `wicket`, `extras`) differ enough that a single generic table/row mapper would leak
      special-cases ? trading visible duplication for hidden complexity.
    - **Explicit code is greppable.** Contributors can follow one sport end-to-end without
      learning a registry indirection. Premature abstraction is harder to undo than dup.
    - **Low, slow growth.** Five sports today; new ones arrive rarely.

    ### Consequences
    - Adding a sport touches ~5 sites (see checklist). We accept that boilerplate.
    - If the cost rises, ADR-001 should be revisited and superseded.

    ### Revisit triggers (when to build the registry)
    - Sport count grows beyond ~8, **or**
    - We add a 4th+ rally-identical sport and the dispatch churn starts causing bugs, **or**
    - A new cross-cutting feature would otherwise require editing every per-sport branch.

    ### Future design sketch (when revisited)
    ```
    @dataclass
    class SportSpec:
        schema_class: type
        table_name: str
        pk_columns: tuple[str, ...]
        columns: tuple[str, ...]
        validate_fn: Callable

    SPORT_REGISTRY: dict[SportType, SportSpec] = { ... }
    # Generic: insert_segments(sport, items) / get_segments(sport, video_id) /
    #          validate(sport, items) / import-local dispatch driven by the registry.
    ```

    ---

    ## Checklist: adding a new sport today (under ADR-001)

    1. `src/core/schemas/<sport>.py` ? define the segment model (mirror the closest existing
sport) with a `start_time < end_time` validator.
    2. `src/core/db.py` ? add to `SEGMENT_TABLES`, add a `CREATE TABLE`, `insert_<sport>()`,
`get_<sport>()`, and a cascade `DELETE` in `delete_video`.
    3. `src/core/validator.py` ? add `validate_<sport>()` (duration + no-overlap).
    4. `src/cli/curate.py` ? add branches in `import_local` (validate + insert),
`_parse_local_annotations` (JSON **and** CSV), and the interactive review/play.
    5. `config/config.yaml` ? add the sport to `supported_sports` (and confirm the
`SportType` enum value in `schemas/base.py`).
    6. `tests/` ? mirror the existing db/validator/cli tests for the new sport.
````

73

6. user (2026-06-10T05:33:28.628Z)

```
1       # Role in the rally-segmenter training pipeline
2
3       This repo is the **data producer** in a two-repo system. Its job is to curate
4       sports videos with per-rally `[start, end)` annotations and commercial-license
5       metadata, then hand that data to the **badminton-highlight-indexer**, which
6       processes, scores, and (Phase 4) *trains* an offline rally segmenter on it.
7
8       This doc explains *why* the `export` and `fetch` commands exist and how they fit
9       the bigger picture. The full decision records and end-to-end narrative live in the
10      indexer repo:
11
12      - `badminton-highlight-indexer/docs/ROADMAP.md` ? the four-phase plan and end-state.
13      - `badminton-highlight-indexer/docs/DECISIONS.md` ? ADR-002 (this bridge), ADR-003
14        (acquisition policy), ADR-004 (offline learned segmenter).
15
16      ## Why this repo matters
17
18      The indexer's best rally mode refines local detections with a **paid Gemini API**.
19      The strategic goal is to replace that with an **offline model trained on our own
20      commercially-licensed, annotated corpus** ? making rally detection free, offline,
21      and self-improving. None of that is possible without a clean, licensable,
22      accurately-labelled, full-resolution dataset. That dataset is what this repo
23      produces.
24
25      ## What the two key commands do (and why)
26
27      - **`curate export`** ? emits the indexer's ground-truth format: per-video
28        `start,end` CSVs + a `manifest.json` pairing each CSV with its video file.
29        Sport-agnostic (all supported sports ? badminton, tennis, cricket, pickleball,
30        table tennis ? share a `(video_id, ordinal, start, end)` shape) and gated on
31        `is_commercial`, so only licensable data flows
32        downstream. The annotations are **temporal-only** ? they train/evaluate rally
33        *segmentation*, not the frame-level shuttle detector (see ADR-002).
34
35      - **`fetch`** ? (re)downloads videos at **best available resolution**, updating
36        the DB **in place** so annotations and curation status are never lost. Uses
37        authenticated cookies for reliable, unthrottled downloads. The why and the
38        hard-won pitfalls (SABR?144p, Chrome DPAPI, rate-limiting) are in ADR-003.
39
40      ## The one contract that matters: video identity
41
42      The indexer keys videos by **file MD5**, this repo by a human `video_id`. A
43      re-encode/re-download changes the bytes, hence the MD5. So the manifest carries the
44      exact `local_path`, and any downstream processing must run on **that** file. This is
45      why acquisition (`fetch`) happens *before* the indexer processes anything.
46
```

7. user (2026-06-10T05:33:29.568Z)

```
1       # Rally Ingestion Pipeline ? End-to-End FAQ
2
3       How a vendor's rally annotations become training/eval ground truth, and how we
4       validate each delivery against our conventions. This is the operational companion
5       to the annotation spec (`docs/annotation/ROORA_BRIEF.md` + `ANNOTATION_GUIDE.md`).
6
7       ```
8         vendor delivery                 OUR side (collector)                modeling
(indexer)
9        ??????????????????   convert    ???????????????????  validate  ???????????????????
10       ? CVAT export    ? ??????????? ? canonical CSV ? ?????????? ? report .md   ?
11       ? (boxes + attrs)?             ? (per rally,   ?            ? report .json ?
```

```
12       ?   OR           ?              ?  decimal secs)?            ? review .csv  ???? human
verifies
13       ? canonical CSV  ?              ????????????????            ???????????????
flagged rallies
14       ????????????????                    ?                            ?
15                                            ?  import-local (validated)  ?
16                                            ?                      golden labels
17                                     ????????????????  export   ???????????????
18                                     ? SQLite (rally ? ????????? ? <id>.csv     ?
start,end (indexer GT)
19                                     ?  tables)      ?           ? manifest.json?
20                                     ????????????????           ???????????????
21                                                                       ?
22                                                                slice clips / calibrate /
train
23       ```
24
25       ## The four commands (in order)
26
27       ```bash
28       # 1. Normalise a CVAT export ? our canonical per-rally CSV (skip if the vendor sent a
CSV)
29       python -m src.cli.curate convert-cvat --input job.xml \
30           --video-path match.mp4 --out canonical.csv --issues issues.txt
31
32       # 2. Validate the delivery ? report + annotatable review sheet (the QA gate)
33       python -m src.cli.curate validate-delivery --input job.xml \
34           --video-path match.mp4 --video-id pilot_badminton_01 --out-dir data/validation
35       #   ...or validate a CSV directly:
36       python -m src.cli.curate validate-delivery --csv canonical.csv --video-id
pilot_badminton_01
37
38       # 3. After human review, import the (corrected) CSV into the DB
39       python -m src.cli.curate import-local --sport badminton \
40           --video-path match.mp4 --annotations-path canonical.csv --fps 59.94
41
42       # 4. Export indexer-ready ground truth (start,end CSVs + manifest)
43       python -m src.cli.curate export --out-dir data/eval_export
44       ```
45
46       ---
47
48       ## FAQ
49
50       ### What delivery formats do we accept?
51       Both. A **CVAT export** (image- or track-mode XML ? rally fields carried as box
52       attributes) is normalised by `convert-cvat`; an **already-canonical CSV** is taken
53       as-is. Vendors may keep working in CVAT ? we do **not** require them to hand-author
54       CSVs.
55
56       ### What is the "canonical" CSV?
57       One row per rally:
58       `rally_number, start_mmss, end_mmss, start_time, end_time, shots_count, ending_reason,
last_shot_outcome, score_before, score_after, notes`.
59       `start_time`/`end_time` are **decimal seconds**; `import-local` reads those.
60
61       ### How are timestamps handled? (the fps thing)
62       We recompute each rally's interval from its **frame range × the true fps**
63       (`seconds = frame / fps`). A vendor's CVAT time attributes are **ignored** ? a
64       step-sampled job that assumes 30 fps writes systematically wrong times. fps comes
65       from `--fps` or ffprobe of `--video-path`. `mm:ss` is also accepted everywhere.
66
67       ### What does validation check?
68       | code | severity | meaning |
69       |---|---|---|
```

```
70      | `overlap` | **blocker** | two rallies' intervals overlap (`end[N] > start[N+1]`) |
71      | `zero_duration` | **blocker** | `end <= start` (single-sample / stray box) |
72      | `frame_out_of_range` | high | a box frame is past the video's last frame |
73      | `shot_density` | medium | long rally with implausibly few shots ? likely an **inflated
span** (often hides a merged rally) |
74      | `possible_missed_rally` | medium | a long unlabeled gap between two rallies ? check
for an **omitted rally** |
75      | `ending_reason_vocab` | medium | `ending_reason` outside the agreed set |
76      | `outcome_vocab` | low | `last_shot_outcome` outside `{in,net,out,n_a}` |
77
78      ### Blocker vs. review ? what's the gate?
79      `validate-delivery` exits **non-zero if any blocker exists** (so it can gate a bulk
80      loop / CI). Non-blocker issues don't stop the gate but populate the **review sheet**
81      for the human pass. `import-local` independently rejects overlaps/zero-duration, so
82      bad ground truth can't slip into the DB.
83
84      ### How are *missed rallies* caught? (the worst error)
85      Two complementary checks, because a miss shows up two ways:
86      1. **Clean omission** (a real hole in the timeline) ? `possible_missed_rally` (gap
heuristic).
87      2. **Merged/inflated** (the missed rally got swallowed into a neighbour's box, leaving
88         no gap) ? `shot_density` flags the over-long, under-shot rally, and the human finds
89         the second rally inside it.
90      Neither is a guarantee ? they **surface regions to review**, they don't certify
91      completeness. (A future enhancement: a motion second-opinion over the raw video.)
92
93      ### What's the review sheet?
94      `<id>_review.csv` ? one row per rally with `auto_flags` (which checks fired) plus
95      blank `VERIFIED / true_start_mmss / true_end_mmss / true_ending_reason /
96      true_last_shot_outcome / reviewer_comment` columns. Open it next to the video
97      (timestamp overlay on) and fill the flagged rows. This is the human gate that turns
98      a delivery into trusted golden labels.
99
100     ### Why two ending fields (`ending_reason` + `last_shot_outcome`)?
101     They're orthogonal: `ending_reason` is *why* the point ended (winner / forced_error /
102     unforced_error / service_fault / let / other); `last_shot_outcome` is *where* the
103     last shot went (in / net / out / n_a). A shuttle that flew long is
104     `forced_error/unforced_error` + `out` ? both kept, no information lost.
105
106     ### How do I onboard a brand-new video?
107     1. Get fps into the intake manifest (we provide it to the vendor).
108     2. `convert-cvat` (if CVAT) ? `validate-delivery` ? read the report, fill the review
109        sheet for flagged rallies ? correct the CSV ? `import-local` ? `export`.
110     3. The same suite runs for every video, so quality stays consistent at scale.
111
112     ### How do I pull a single rally out to eyeball it?
113     The slicer (indexer):
114     ```bash
115     python -m backend.tools.rally_slicer --video match.mp4 \
116         --labels golden_labels.csv --rallies 7,16 --out-dir output/rally_clips
117     ```
118     Clips are padded to match the highlight compiler (0.5s lead / 1.0s tail).
119
120     ### Does coarse frame sampling hurt boundaries?
121     Not in practice: ~0.5s sampling slack is absorbed by the highlight compiler's clip
122     padding, so we don't flag sampling density.
123
124     ### Will a non-commercial license block export?
125     The video is **kept** in the DB regardless of license, but the default `export` is
126     **commercial-only** ? a non-commercial source (e.g. `CC-BY-NC-SA`) is *excluded*
127     from the eval/training set unless you pass `--include-noncommercial`. So if a video
128     "goes missing" downstream, check its `is_commercial` flag first. (The observability
129     report raises a `license_noncommercial` warning for exactly this case.)
130
131     ### What is the observability report? (`*.report.json` / `*.report.md` / `*.summary.md`)
```

```
132     Every `validate-delivery`, `convert-cvat`, and `import-local` run now also writes a
133     per-video **observability report** (next to the video for `import-local`; into the
134     `--out-dir` / next to the canonical CSV otherwise):
135     - **`<id>.report.json`** ? machine-readable: every stage, metric, and finding. Diff
136       two runs to see what changed between deliveries.
137     - **`<id>.report.md`** ? the **technical** report: parse / validation / registration
138       stages plus **findings** (one per delivery issue, each citing this FAQ). Read this
139       to debug a delivery.
140     - **`<id>.summary.md`** ? a **delivery summary** for the vendor / data team: the
141       rally count and coverage plus the plain-language list of rallies to review ? no
142       internal codes or severities.
143     This is in addition to the existing `<id>_report.md` / `_report.json` / `_review.csv`
144     (the human review sheet), which are unchanged.
145
```

8. user (2026-06-10T05:33:29.784Z)

```
1       # Positioning: manual (rally-annotator) golden-label ingestion
2
3       How `sports-data-collector` should be positioned to ingest **manual** rally labels
4       produced by the VLC [rally-annotator](https://github.com/avidullu/rally-annotator),
5       alongside vendor (Roora/CVAT) deliveries, and serve them to the
6       badminton-highlight-indexer eval/training pipeline.
7
8       > Source: a 10-agent analysis workflow across this repo, the indexer, and the
9       > annotator, verified against the real `Adarsh and Avi.rallies.csv`. Confidence: high.
10
11      ## Bottom line
12
13      Position the collector as the **single canonical system-of-record for golden rally
14      labels**: the one place where *any* annotation source ? manual VLC lean CSV, the rich
15      review sheet, or vendor CVAT/CSV ? becomes a validated, license-gated, registered rally
16      record. The indexer consumes **only** the collector's narrow export contract:
17      `manifest.json` + a per-video `start,end` CSV (`curate.py::export_eval_set`;
18      `calibrate_wasb.py::load_golden_gts` reads `start_time`/`end_time` only).
19
20      The load-bearing property to protect: **the seam to the indexer stays intervals-only.**
21      Everything else ? `ending_reason`, `shots_count`, scores, provenance, maturity, the
22      labeled window ? is validated at the hub and rides in the **manifest**, never the eval
23      CSV. A new sport or source becomes an adapter change at the hub, never an indexer
change.
24
25      ## Role in the pipeline
26
27      ```
28       producers ???????????????  collector (hub) ?????????????  consumer
29       ? VLC rally-annotator       parse ? validate ? license-gate    badminton-highlight-
30         (lean 5-col CSV)          ? register (SQLite) ? export        indexer (eval/training)
31       ? manual review sheet                                          reads ONLY:
32         (rich 14-col CSV)         everything but (start,end) is        manifest.json +
33       ? vendor Roora / CVAT       validated then dropped before        per-video start,end
CSV
34         (CVAT XML / canonical)    the indexer sees it
35      ```
36
37      ## It already works today (zero code changes)
38
39      The lean annotator CSV header `rally_number,start_time,end_time,ending_reason,sport`
40      matches the `import-local` `DictReader` exactly (`curate.py::_parse_local_annotations`),
41      and the validator accepts gapped/partial coverage as-is
42      (`validator.py::validate_badminton_rallies` checks only positive duration + no overlap).
43      Verified end-to-end on the real file ? 40 rallies ? export ? 40-interval indexer CSV.
44
45      ```bash
46      python -m src.cli.curate import-local --sport badminton \
```

```
  --video-path "?/Adarsh and Avi.MP4" \
  --annotations-path "?/Adarsh and Avi.rallies.csv" --license Proprietary --fps 59.94
python -m src.cli.curate export   # ? manifest.json + <video_id>.csv (start,end)
```

P0 ? the two-landmine guard (`normalize_annotations.py`, shipped)

Partial human data has two failure modes the raw path handles badly. The
`src/cli/normalize_annotations.py` pre-import normalizer fixes both, as a **standalone
step** that feeds `import-local` (it does **not** touch the DB schema or the
`rally_number`-ordered export):

1. **An unusable time row aborts the *entire* import.** A blank/missing time makes
   `safe_float_required` raise (`parsing.py`); a reversed `end_time <= start_time` or a
   negative `start_time` makes the `BadmintonRally` pydantic validator raise
   (`schemas/badminton.py::validate_timestamps`). Either way the `try/except` in
   `import-local` turns it into `sys.exit(1)`, killing the whole video. ? the normalizer
   drops such rows (mirroring the indexer loader, which already skips `not end >
start`).
2. **A fractional/duplicate `rally_number` (e.g. `7.5`, `16.5`) silently corrupts
data.**
   `int(float("7.5")) ? 7` (`parsing.py::safe_int`) collides on the
   `(video_id, rally_number)` primary key, and the DELETE-then-INSERT upsert
   (`db.py::insert_badminton_rallies`) destroys the colliding rally. ? the normalizer
   renumbers `1..N` by `start_time`, recording the original id for traceability.

It also converts `mm:ss`/`HH:MM:SS` ? decimal seconds and folds a pipe-delimited
`ending_reason` ("forced_error|other") to its primary token.

```bash
python -m src.cli.normalize_annotations --in "Match.rallies.csv" --out clean.csv
python -m src.cli.curate import-local --sport badminton --video-path "?/Match.MP4" \
  --annotations-path clean.csv --license Proprietary
```

Do not widen the PK to float ? the DB column is INTEGER, export orders by it, and
the indexer ignores `rally_number` entirely. Resolve ids at the adapter boundary.

Annotator ? canonical field mapping

Annotator field	Canonical	Handling
`rally_number` (lean) / `rally` (rich)	`rally_number:int`	Lean maps directly;
fractional/dup ids renumbered at the normalizer (PK is INT)
`start_time` / `start_mmss`	`start_time:float`	Decimal direct; `mm:ss` converted
to seconds
`end_time` / `end_mmss`	`end_time:float`	**Blank / ? start dropped, not raised**
`ending_reason`	`ending_reason:str?`	Vocab matches the documented canonical set
(free-text, not an enforced enum); pipe-delimited folded to primary; **stored but dropped on
export**
`sport`	? (from `--sport` flag)	CSV column ignored; operator sets the flag
rich-only `shots`, `true_*`, `VERIFIED`, `notes`	`shots_count` + QA/provenance
Carried where present; none reach the indexer |

Roadmap to the canonical-hub end-state

All additive, video-level, backward-compatible (consumers ignore unknown manifest keys).

- **P1 ? persist the labeled window in the manifest.** Partial labels cover only a
  prefix; a full-video eval penalizes correct detections in the unlabeled tail. The
  indexer already restricts to a window (`eval_partial_labels.py::restrict_to_window`)
  but currently *infers* `window_end = max(end_time)` ? which is **wrong when a video is
  tagged past its last rally** (valid predictions in the labeled-but-rally-free tail get
  scored as false positives). The collector should own `labeled_window_start/end`
  (the VLC tool is resumable + monotonic, so the session extent is known), carry it in

```
105       the manifest, and let the indexer apply it automatically. Short-term: pass
106       `--window-end` at eval time.
107     - **P1 ? maturity ladder + provenance.** `CurationStatus` (STAGING/CURATED/REJECTED) is
108       a *workflow* state, not a *trust* state. Add a separate `needs_review ? confirmed ?
109       gold` enum + `annotation_source` (manual_vlc / manual_review / vendor_cvat) +
110       `annotator_id` + timestamp. Route self-rated imports to `needs_review` instead of the
111       hardcoded `CURATED` (`curate.py`), so the golden-set regression flow restricts the
gold
112       tier to `maturity=gold` and an unreviewed re-tag can't silently become the answer key.
113       (Today a single-rater self-rating and a vendor-QA'd delivery are indistinguishable.)
114     - **P2 ? unify sources behind one adapter** emitting `(List[RallyRecord], List[Issue])`,
115       and run the **same Issue detectors** (gap>15s possible-missed-rally, shot-density,
116       vocab, overlap) on manual imports that vendor CVAT already gets. Today self-produced
117       labels get *less* scrutiny than a vendor delivery ? backwards.
118     - **P2 ? freeze the intervals-only export contract** in one doc both repos reference +
119       a round-trip test (header matching is exact case-insensitive, so a vendor
120       `start_seconds` would fail silently).
121
122     ## Open question
123
124     Is the **rich** `rally_review.csv` (mm:ss times, a `rally` column with fractional ids,
125     pipe-delimited reasons, `true_*` correction columns) ever the *source of truth*, or is
126     the lean VLC `.rallies.csv` always canonical? Neither `import-local` nor
127     `load_canonical_csv` (which keys on `rally_number`) ingest the rich file directly. The
128     `normalize_annotations.py` step already handles its `mm:ss` + `rally` + pipe-reason
129     quirks for the common case; a fuller converter that applies `true_*` overrides is only
130     worth building if the rich sheet is canonical.
131
```

9. user (2026-06-10T05:33:34.815Z)

```
1       import os
2       import sys
3       import time
4       import json
5       import csv
6       import argparse
7       from typing import List, Dict, Any, Tuple
8       from tabulate import tabulate
9
10      from src.core.config import load_config
11      from src.core.parsing import safe_int as _safe_int, safe_float_required as _safe_float
12      from src.core.db import SportsDatabase
13      from src.core.schemas.base import VideoMetadata, SportType, LicenseType, CurationStatus
14      from src.core.schemas.badminton import BadmintonRally
15      from src.core.schemas.tennis import TennisPoint
16      from src.core.schemas.cricket import CricketDelivery
17      from src.core.schemas.pickleball import PickleballRally
18      from src.core.schemas.tabletennis import TableTennisRally
19      from src.core.validator import DatasetValidator
20      from src.core.downloader import download_best_video, probe_resolution_fps,
height_to_label
21
22
23      class SportsDataCLI:
24          def __init__(self, config_path: str = "config/config.yaml"):
25              self.config_path = config_path
26              self.config = self._load_config()
27
28              # Resolve DB path
29              db_path = self.config.get("database_path", "data/sports_metadata.db")
30              self.db = SportsDatabase(db_path)
31              self.validator = DatasetValidator(config_path)
32
33          def _load_config(self) -> Dict[str, Any]:
```

```
            if not os.path.exists(self.config_path):
                print(f"Warning: Configuration file not found at {self.config_path}. Using defaults.")
            return load_config(self.config_path)

        def status(self):
            """Displays status summary of all videos in the database."""
            # Query total count by status via the public DB read API.
            rows = self.db.get_status_counts()  # list of (sport, status, cnt)

            if not rows:
                print("\nDatabase is currently empty. Run crawlers or import local data.\n")
                return

            data = [[sport, status, cnt] for sport, status, cnt in rows]
            print("\n=== Dataset Ingestion Status ===")
            print(tabulate(data, headers=["Sport", "Status", "Count"], tablefmt="grid"))
            print()

        def import_local(self, args):
            """Manually registers a local video and annotation file into the database."""
            sport_str = args.sport.lower()
            if sport_str not in self.config.get("supported_sports", ["badminton"]):
                print(f"Error: Sport '{sport_str}' is not supported. Supported: {self.config.get('supported_sports')}")
                sys.exit(1)

            # Validate video path
            if not os.path.exists(args.video_path):
                print(f"Warning: Local video path '{args.video_path}' does not exist on disk, but continuing registration.")

            # Validate annotation file
            if not os.path.exists(args.annotations_path):
                print(f"Error: Annotations file '{args.annotations_path}' not found.")
                sys.exit(1)

            match_name = args.match_name or os.path.basename(args.video_path).split('.')[0]
            video_id = args.video_id or match_name.lower().replace(" ", "_")

            # Parse license
            try:
                license_type = LicenseType(args.license)
            except ValueError:
                print(f"Warning: Unknown license '{args.license}'. Defaulting to Unknown (non-commercial).")
                license_type = LicenseType.UNKNOWN

            is_commercial = self.validator.check_license_commercial(license_type)

            # 1. Parse Annotations
            annotations = []
            try:
                annotations = self._parse_local_annotations(sport_str, video_id, args.annotations_path)
            except Exception as e:
                print(f"Error parsing annotations: {e}")
                sys.exit(1)

            # 2. Run Validation
            is_valid = True
            errors = []
            if sport_str == "badminton":
                is_valid, errors = self.validator.validate_badminton_rallies(annotations)
            elif sport_str == "tennis":
```

```
94                  is_valid, errors = self.validator.validate_tennis_points(annotations)
95              elif sport_str == "cricket":
96                  is_valid, errors = self.validator.validate_cricket_deliveries(annotations)
97              elif sport_str == "pickleball":
98                  is_valid, errors = self.validator.validate_pickleball_rallies(annotations)
99              elif sport_str == "tabletennis":
100                 is_valid, errors = self.validator.validate_tabletennis_rallies(annotations)
101
102             if not is_valid:
103                 print(f"Error: Annotations failed validation!")
104                 for err in errors:
105                     print(f"  - {err}")
106                 sys.exit(1)
107
108             # 3. Create Video Metadata
109             video_meta = VideoMetadata(
110                 video_id=video_id,
111                 sport=SportType(sport_str),
112                 video_url=None,
113                 local_path=args.video_path,
114                 license_type=license_type,
115                 license_url=None,
116                 is_commercial=is_commercial,
117                 status=CurationStatus.CURATED,  # Custom local imports bypass staging by
default
118                 match_name=match_name,
119                 fps=args.fps,
120                 resolution=args.resolution
121             )
122
123             # 4. Insert into Database
124             self.db.insert_video(video_meta)
125
126             count = 0
127             if sport_str == "badminton":
128                 count = self.db.insert_badminton_rallies(annotations)
129             elif sport_str == "tennis":
130                 count = self.db.insert_tennis_points(annotations)
131             elif sport_str == "cricket":
132                 count = self.db.insert_cricket_deliveries(annotations)
133             elif sport_str == "pickleball":
134                 count = self.db.insert_pickleball_rallies(annotations)
135             elif sport_str == "tabletennis":
136                 count = self.db.insert_tabletennis_rallies(annotations)
137
138             print(f"\nSuccessfully imported local video '{match_name}':")
139             print(f"  - Video ID: {video_id}")
140             print(f"  - Sport: {sport_str}")
141             print(f"  - Commercial Use Allowed: {is_commercial} (License:
{license_type.value})")
142             print(f"  - Registered {count} annotated segments.\n")
143
144             # Per-video observability report (next to the video). Never raises.
145             from src.reporting import emit_import_report
146             src_fmt = os.path.splitext(args.annotations_path)[1].lstrip(".").lower() or
"unknown"
147             report_paths = emit_import_report(
148                 video_meta_obj=video_meta, annotations=annotations, is_valid=True,
149                 errors=[], count=count, sport=sport_str, source_format=src_fmt,
150                 fallback_dir=os.path.dirname(os.path.abspath(args.annotations_path)),
151             )
152             if report_paths:
153                 print(f"  - Observability report: {report_paths.get('technical_md')}")
154                 print(f"  - Customer summary    : {report_paths.get('customer_md')}\n")
155
```

```
        def _parse_local_annotations(self, sport: str, video_id: str, filepath: str) ->
List[Any]:
            """Parses simple JSON or CSV annotations files."""
            parsed_items = []
            ext = os.path.splitext(filepath)[1].lower()

            if ext == '.json':
                with open(filepath, 'r', encoding='utf-8') as f:
                    data = json.load(f)

                if not isinstance(data, list):
                    raise ValueError("JSON file must contain a list of objects.")

                # Uses the same safe casts as the CSV path: a missing/blank required
                # time raises a clear ValueError (caught by the caller) instead of a
                # bare KeyError, and optional ints degrade to sensible defaults.
                for idx, obj in enumerate(data):
                    if sport == "badminton":
                        parsed_items.append(BadmintonRally(
                            video_id=video_id,
                            rally_number=_safe_int(obj.get("rally_number"), idx + 1),
                            start_time=_safe_float(obj.get("start_time")),
                            end_time=_safe_float(obj.get("end_time")),
                            score_before=obj.get("score_before"),
                            score_after=obj.get("score_after"),
                            shots_count=_safe_int(obj.get("shots_count"), None),
                            ending_reason=obj.get("ending_reason"),
                            last_shot_outcome=obj.get("last_shot_outcome")
                        ))
                    elif sport == "tennis":
                        parsed_items.append(TennisPoint(
                            video_id=video_id,
                            point_number=_safe_int(obj.get("point_number"), idx + 1),
                            start_time=_safe_float(obj.get("start_time")),
                            end_time=_safe_float(obj.get("end_time")),
                            set_score=obj.get("set_score"),
                            game_score=obj.get("game_score"),
                            server=obj.get("server"),
                            ending_type=obj.get("ending_type")
                        ))
                    elif sport == "cricket":
                        parsed_items.append(CricketDelivery(
                            video_id=video_id,
                            over=_safe_int(obj.get("over"), 0),
                            ball=_safe_int(obj.get("ball"), 0),
                            start_time=_safe_float(obj.get("start_time")),
                            end_time=_safe_float(obj.get("end_time")),
                            batsman=obj.get("batsman"),
                            bowler=obj.get("bowler"),
                            runs=_safe_int(obj.get("runs"), None),
                            wicket=bool(obj.get("wicket", False)),
                            extras=_safe_int(obj.get("extras"), 0)
                        ))
                    elif sport == "pickleball":
                        parsed_items.append(PickleballRally(
                            video_id=video_id,
                            rally_number=_safe_int(obj.get("rally_number"), idx + 1),
                            start_time=_safe_float(obj.get("start_time")),
                            end_time=_safe_float(obj.get("end_time")),
                            score_before=obj.get("score_before"),
                            score_after=obj.get("score_after"),
                            shots_count=_safe_int(obj.get("shots_count"), None),
                            ending_reason=obj.get("ending_reason"),
                            last_shot_outcome=obj.get("last_shot_outcome")
                        ))
```

```
                    elif sport == "tabletennis":
                        parsed_items.append(TableTennisRally(
                            video_id=video_id,
                            rally_number=_safe_int(obj.get("rally_number"), idx + 1),
                            start_time=_safe_float(obj.get("start_time")),
                            end_time=_safe_float(obj.get("end_time")),
                            score_before=obj.get("score_before"),
                            score_after=obj.get("score_after"),
                            shots_count=_safe_int(obj.get("shots_count"), None),
                            ending_reason=obj.get("ending_reason"),
                            last_shot_outcome=obj.get("last_shot_outcome")
                        ))
            elif ext == '.csv':
                with open(filepath, 'r', encoding='utf-8') as f:
                    reader = csv.DictReader(f)
                    # Normalize headers
                    reader.fieldnames = [n.lower().strip() for n in reader.fieldnames] if
reader.fieldnames else []

                    for idx, row in enumerate(reader):
                        if sport == "badminton":
                            parsed_items.append(BadmintonRally(
                                video_id=video_id,
                                rally_number=_safe_int(row.get("rally_number"), idx + 1),
                                start_time=_safe_float(row.get("start_time")),
                                end_time=_safe_float(row.get("end_time")),
                                score_before=row.get("score_before"),
                                score_after=row.get("score_after"),
                                shots_count=_safe_int(row.get("shots_count"), None),
                                ending_reason=row.get("ending_reason"),
                                last_shot_outcome=row.get("last_shot_outcome")
                            ))
                        elif sport == "tennis":
                            parsed_items.append(TennisPoint(
                                video_id=video_id,
                                point_number=_safe_int(row.get("point_number"), idx + 1),
                                start_time=_safe_float(row.get("start_time")),
                                end_time=_safe_float(row.get("end_time")),
                                set_score=row.get("set_score"),
                                game_score=row.get("game_score"),
                                server=row.get("server"),
                                ending_type=row.get("ending_type")
                            ))
                        elif sport == "cricket":
                            parsed_items.append(CricketDelivery(
                                video_id=video_id,
                                over=_safe_int(row.get("over"), 0),
                                ball=_safe_int(row.get("ball"), 0),
                                start_time=_safe_float(row.get("start_time")),
                                end_time=_safe_float(row.get("end_time")),
                                batsman=row.get("batsman"),
                                bowler=row.get("bowler"),
                                runs=_safe_int(row.get("runs"), None),
                                wicket=(row.get("wicket") or "").lower() in ["true", "1",
"yes"],
                                extras=_safe_int(row.get("extras"), 0)
                            ))
                        elif sport == "pickleball":
                            parsed_items.append(PickleballRally(
                                video_id=video_id,
                                rally_number=_safe_int(row.get("rally_number"), idx + 1),
                                start_time=_safe_float(row.get("start_time")),
                                end_time=_safe_float(row.get("end_time")),
                                score_before=row.get("score_before"),
                                score_after=row.get("score_after"),
```

```
                                shots_count=_safe_int(row.get("shots_count"), None),
                                ending_reason=row.get("ending_reason"),
                                last_shot_outcome=row.get("last_shot_outcome")
                            ))
                        elif sport == "tabletennis":
                            parsed_items.append(TableTennisRally(
                                video_id=video_id,
                                rally_number=_safe_int(row.get("rally_number"), idx + 1),
                                start_time=_safe_float(row.get("start_time")),
                                end_time=_safe_float(row.get("end_time")),
                                score_before=row.get("score_before"),
                                score_after=row.get("score_after"),
                                shots_count=_safe_int(row.get("shots_count"), None),
                                ending_reason=row.get("ending_reason"),
                                last_shot_outcome=row.get("last_shot_outcome")
                            ))
            else:
                raise ValueError("Unsupported annotations file extension. Use .json or
.csv.")

            return parsed_items

        def _fetch_one_with_retry(self, url, dest_path, args, max_height, max_size_mb):
            """Download one video at best resolution, retrying transient failures.

            A below-min-height result almost always means YouTube rate-limited the
            request and served a low fallback (not that the video lacks HD), so we
            retry with a growing backoff. Returns (DownloadResult, None) on success,
            or (None, failure_reason) once retries are exhausted. A below-min file is
            discarded each attempt so it never lands in the DB.
            """
            attempts = max(1, args.retries + 1)
            reason = "unknown error"
            for i in range(attempts):
                result = download_best_video(
                    url, dest_path,
                    max_height=max_height, max_size_mb=max_size_mb,
                    cookies_from_browser=args.cookies_from_browser,
                    cookies_file=args.cookies,
                    player_client=args.player_client,
                )
                if result.success and (result.height is None or result.height >=
args.min_height):
                    return result, None

                if not result.success:
                    reason = result.error or "unknown error"
                else:
                    reason = (f"got {result.resolution_label} (< min-height
{args.min_height}p) ? "
                              f"YouTube served a low format (rate-limited?)")
                    try:
                        os.remove(result.path)
                    except OSError:
                        pass

                if i < attempts - 1:
                    backoff = args.retry_backoff * (i + 1)
                    print(f"  attempt {i+1}/{attempts} failed: {reason}\n  retrying in
{backoff:.0f}s...")
                    time.sleep(backoff)
            return None, reason

        def fetch(self, args):
            """(Re)download videos at best available resolution, updating the DB in place.
```

```
344
345             This is the in-place upgrade path: rows, rally annotations, and curation
346             status are all preserved ? only the video file plus its `local_path`,
347             `resolution`, and `fps` are refreshed. By default it skips videos that
348             already have a local file at or above `--min-height`, so re-runs are cheap
349             and resumable; `--force` re-fetches everything.
350             """
351             storage = self.config.get("video_storage", {})
352             local_dir = storage.get("local_dir", "./data/videos")
353             max_download_size_mb = storage.get("max_download_size_mb")
354             max_height = args.max_height if args.max_height is not None else
storage.get("max_height")
355             os.makedirs(local_dir, exist_ok=True)
356
357             commercial_only = not args.include_noncommercial
358             if args.status == "all":
359                 statuses = [CurationStatus.CURATED, CurationStatus.STAGING]
360             else:
361                 statuses = [CurationStatus(args.status)]
362
363             supported = self.config.get("supported_sports", ["badminton"])
364             sport_strs = [args.sport] if args.sport else supported
365
366             # Collect target videos.
367             videos = []
368             for sport_str in sport_strs:
369                 try:
370                     sport = SportType(sport_str.lower())
371                 except ValueError:
372                     print(f"Warning: skipping unknown sport '{sport_str}'.")
373                     continue
374                 for status in statuses:
375                     videos.extend(self.db.get_videos_by_status(status, sport))
376             if args.video_id:
377                 videos = [v for v in videos if v.video_id == args.video_id]
378
379             if not videos:
380                 print("\nNo matching videos to fetch.\n")
381                 return
382
383             results = []  # (video_id, action, detail)
384             for video in videos:
385                 if commercial_only and not video.is_commercial:
386                     results.append((video.video_id, "skip", "non-commercial"))
387                     continue
388                 if not video.video_url or "watch?v=example" in video.video_url:
389                     # Either no URL, or the parser's placeholder (the ShuttleSet catalog
390                     # had no URL for this match) ? nothing to download.
391                     results.append((video.video_id, "skip", "no real source URL"))
392                     continue
393
394                 dest_path = os.path.join(local_dir, f"{video.video_id}.mp4")
395
396                 # Skip if we already have a good-enough local file (unless --force).
397                 if not args.force and os.path.exists(dest_path):
398                     _, height, _ = probe_resolution_fps(dest_path)
399                     if height is not None and height >= args.min_height:
400                         results.append((video.video_id, "have", f"{height_to_label(height)}
already"))
401                         continue
402
403                 print(f"Fetching {video.video_id} ({video.match_name})...")
404                 result, fail_reason = self._fetch_one_with_retry(
405                     video.video_url, dest_path, args, max_height, max_download_size_mb)
406                 if fail_reason is not None:
```

```
                    results.append((video.video_id, "FAIL", fail_reason))
                    if args.sleep:
                        time.sleep(args.sleep)
                    continue

                # Update only the file-derived fields; preserve everything else (UPSERT
                # mutates the row in place, leaving rallies intact).
                video.local_path = result.path
                if result.resolution_label:
                    video.resolution = result.resolution_label
                if result.fps:
                    video.fps = result.fps
                self.db.insert_video(video)

                detail = (f"{result.resolution_label or '?'}"
                          f"{f' {result.fps:g}fps' if result.fps else ''},
{result.size_mb:.0f} MB")
                results.append((video.video_id, "fetched", detail))
                if args.sleep:
                    time.sleep(args.sleep)

            print("\n=== Fetch summary ===")
            print(tabulate([[vid, action, detail] for vid, action, detail in results],
                           headers=["Video ID", "Action", "Detail"], tablefmt="grid"))
            fetched = sum(1 for _, a, _ in results if a == "fetched")
            failed = [r for r in results if r[1] == "FAIL"]
            print(f"\nFetched/updated : {fetched}   Already-good : {sum(1 for _,a,_ in
results if a=='have')}   "
                  f"Skipped : {sum(1 for _,a,_ in results if a=='skip')}   Failed :
{len(failed)}")
            if failed:
                print("Failures (left unchanged in DB):")
                for vid, _, detail in failed:
                    print(f"  - {vid}: {detail}")
            print()

        def export_eval_set(self, args):
            """Export curated annotations as indexer-ready eval/training sets.

            Emits, per qualifying video, a `<out_dir>/<sport>/<video_id>.csv` in the
            indexer's `start,end` ground-truth format (decimal seconds), plus a single
            `<out_dir>/manifest.json` that pairs each CSV with its video file. The
            manifest is the contract the downstream eval/training driver reads: the
            indexer keys videos by *file MD5*, not by our `video_id`, so it must run
            against the exact `local_path`/`video_url` recorded here.

            By default only CURATED + commercially-safe videos are exported, mirroring
            the license gate already enforced elsewhere in the toolchain.
            """
            out_dir = args.out_dir
            commercial_only = not args.include_noncommercial
            statuses = [CurationStatus.CURATED]
            if args.include_staging:
                statuses.append(CurationStatus.STAGING)

            supported = self.config.get("supported_sports", ["badminton"])
            sport_strs = [args.sport] if args.sport else supported

            manifest = []
            skipped_noncomm = 0
            skipped_empty = 0

            for sport_str in sport_strs:
                try:
                    sport = SportType(sport_str.lower())
```

```
                except ValueError:
                    print(f"Warning: skipping unknown sport '{sport_str}'.")
                    continue

                videos = []
                for status in statuses:
                    videos.extend(self.db.get_videos_by_status(status, sport))

                for video in videos:
                    if commercial_only and not video.is_commercial:
                        skipped_noncomm += 1
                        continue

                    intervals = self.db.get_segment_intervals(video.video_id, sport)
                    if not intervals:
                        skipped_empty += 1
                        continue

                    sport_dir = os.path.join(out_dir, sport.value)
                    os.makedirs(sport_dir, exist_ok=True)
                    csv_path = os.path.join(sport_dir, f"{video.video_id}.csv")
                    with open(csv_path, "w", newline="", encoding="utf-8") as f:
                        writer = csv.writer(f)
                        writer.writerow(["start", "end"])
                        for start, end in intervals:
                            writer.writerow([start, end])

                    manifest.append({
                        "video_id": video.video_id,
                        "sport": sport.value,
                        "csv": os.path.relpath(csv_path, out_dir).replace(os.sep, "/"),
                        "local_path": video.local_path,
                        "video_url": video.video_url,
                        "match_name": video.match_name,
                        "fps": video.fps,
                        "resolution": video.resolution,
                        "license_type": video.license_type.value,
                        "is_commercial": video.is_commercial,
                        "status": video.status.value,
                        "num_segments": len(intervals),
                    })

            os.makedirs(out_dir, exist_ok=True)
            manifest_doc = {
                "commercial_only": commercial_only,
                "statuses": [s.value for s in statuses],
                "total_videos": len(manifest),
                "total_segments": sum(m["num_segments"] for m in manifest),
                "eval_sets": manifest,
            }
            manifest_path = os.path.join(out_dir, "manifest.json")
            with open(manifest_path, "w", encoding="utf-8") as f:
                json.dump(manifest_doc, f, indent=2)

            print(f"\n=== Exported eval/training set to '{out_dir}' ===")
            if manifest:
                summary = [[m["sport"], m["video_id"], m["num_segments"],
                            "YES" if m["is_commercial"] else "NO",
                            os.path.basename(m["local_path"]) if m["local_path"] else "(no
local file)"]
                           for m in manifest]
                print(tabulate(summary, headers=["Sport", "Video ID", "Segments", "Comm?",
"Video file"], tablefmt="grid"))
            print(f"\nVideos exported : {len(manifest)}")
            print(f"Total segments  : {manifest_doc['total_segments']}")
```

```
            if skipped_noncomm:
                print(f"Skipped (non-commercial) : {skipped_noncomm}  (pass --include-
noncommercial to include)")
            if skipped_empty:
                print(f"Skipped (no annotations) : {skipped_empty}")
            print(f"Manifest        : {manifest_path}\n")

        def convert_cvat(self, args):
            """Convert a vendor CVAT rally export into our canonical per-rally CSV.

            Absorbs the CVAT bounding-box delivery format (rally fields carried as box
            attributes) on the ingestion side: each rally's time is recomputed from
            its frame range at the video's true fps, and content defects (overlaps,
            zero-duration, off-vocabulary endings, coarse sampling) are written to an
            issues report rather than silently "fixed". The emitted CSV imports via
            `import-local`; any blocker issue must be resolved first, since the
            importer's validator rejects overlapping / zero-duration rallies.
            """
            from src.parsers.cvat.rally_cvat import (
                convert, write_canonical_csv, render_issue_report,
            )
            try:
                result = convert(args.input, fps=args.fps,
                                 video_path=args.video_path, step=args.step)
            except (FileNotFoundError, ValueError) as e:
                print(f"Error: {e}")
                sys.exit(1)

            write_canonical_csv(result.records, args.out)

            # Observability report next to the canonical CSV. Never raises.
            from src.reporting import emit_delivery_report
            src_name = os.path.splitext(os.path.basename(args.input))[0]
            dur = (result.meta.get("stop_frame") / result.fps) if
(result.meta.get("stop_frame") and result.fps) else None
            emit_delivery_report(
                out_dir=os.path.dirname(os.path.abspath(args.out)) or ".", source=src_name,
                records=result.records, issues=result.issues, fps=result.fps,
video_path=args.video_path,
                video_duration_s=dur, source_format="cvat", step=result.step,
            )

            report = render_issue_report(result, os.path.basename(args.input))
            if args.issues:
                with open(args.issues, "w", encoding="utf-8") as f:
                    f.write(report)
            print(report)
            print(f"Canonical CSV written: {args.out}  ({len(result.records)} rallies)")
            if result.n_blockers:
                print(f"\n  {result.n_blockers} BLOCKER issue(s) ? import-local will reject
"
                      f"this file until they are resolved (by the vendor or manual
review).")
            else:
                print("\n  No blockers. Next:")
                print(f"    python -m src.cli.curate import-local --sport <sport> "
                      f"--video-path <video> --annotations-path {args.out} --fps
{result.fps:g}")

        def validate_delivery(self, args):
            """Run the full validation suite on a rally delivery and emit a report.

            Accepts a CVAT export (--input + --video-path/--fps) or an already-canonical
            CSV (--csv). Writes, into --out-dir:
              <id>_report.md    human report (scoreboard + issues + rallies to review)
```

```
              <id>_report.json  machine summary (for CI / a bulk-ingest loop)
              <id>_review.csv   the annotatable manual-verification sheet
            Exits non-zero if any blocker issue exists, so it can gate a bulk pipeline.
            """
            import json
            from src.parsers.cvat.rally_cvat import (
                convert, load_canonical_csv, detect_issues,
                build_summary, render_validation_md, write_review_csv,
            )
            vid = args.video_id or "delivery"
            fps = None
            step = None
            stop_frame = None
            if args.input:
                try:
                    result = convert(args.input, fps=args.fps, video_path=args.video_path)
                except (FileNotFoundError, ValueError) as e:
                    print(f"Error: {e}")
                    sys.exit(1)
                records, issues, fps = result.records, result.issues, result.fps
                step = result.step
                stop_frame = result.meta.get("stop_frame")
            elif args.csv:
                try:
                    records = load_canonical_csv(args.csv)
                except FileNotFoundError as e:
                    print(f"Error: {e}")
                    sys.exit(1)
                if not records:
                    print(f"Error: no usable rally rows in {args.csv}")
                    sys.exit(1)
                issues = detect_issues(records)
            else:
                print("Error: provide --input <cvat.xml> or --csv <canonical.csv>")
                sys.exit(1)

            os.makedirs(args.out_dir, exist_ok=True)
            summary = build_summary(records, issues, source=vid, fps=fps)
            md = render_validation_md(records, issues, source=vid)
            json_path = os.path.join(args.out_dir, f"{vid}_report.json")
            md_path = os.path.join(args.out_dir, f"{vid}_report.md")
            review_path = os.path.join(args.out_dir, f"{vid}_review.csv")
            with open(json_path, "w", encoding="utf-8") as f:
                json.dump(summary, f, indent=2)
            with open(md_path, "w", encoding="utf-8") as f:
                f.write(md)
            write_review_csv(records, issues, review_path)

            # Observability report (envelope JSON + technical md + customer summary),
            # colocated with the existing artifacts. Never raises.
            from src.reporting import emit_delivery_report
            duration_s = (stop_frame / fps) if (stop_frame and fps) else None
            report_paths = emit_delivery_report(
                out_dir=args.out_dir, source=vid, records=records, issues=issues, fps=fps,
                video_path=args.video_path, video_duration_s=duration_s,
                source_format=("cvat" if args.input else "csv"), step=step,
            )

            print(md)
            print(f"\nWrote:\n  {md_path}\n  {json_path}\n  {review_path}")
            if report_paths:
                print(f"  {report_paths.get('technical_md')}\n
{report_paths.get('customer_md')}\n  {report_paths.get('json')}")
            if summary["n_blockers"]:
                print(f"\n  GATE: BLOCKED ? {summary['n_blockers']} blocker(s) must be
```

```
resolved.")
                sys.exit(2)
            print("\n  GATE: PASS ? review the flagged rallies, then `import-local`.")

        def curate(self):
            """Interactive loop to review and curate staging video annotations."""
            staging_videos = self.db.get_videos_by_status(CurationStatus.STAGING)
            if not staging_videos:
                print("\nNo staging video annotations to review.\n")
                return

            print(f"\nFound {len(staging_videos)} matches in STAGING state for curation:")
            table_data = []
            for idx, video in enumerate(staging_videos):
                table_data.append([
                    idx + 1,
                    video.video_id,
                    video.match_name,
                    video.sport.value,
                    video.license_type.value,
                    "YES" if video.is_commercial else "NO"
                ])

            print(tabulate(table_data, headers=["#", "Video ID", "Match Name", "Sport",
"License", "Commercial Safe?"], tablefmt="grid"))
            print("\nEnter a number (1-{}) to review details, or 'q' to
quit:".format(len(staging_videos)))

            choice = input("> ").strip()
            if choice.lower() == 'q':
                return

            try:
                idx = int(choice) - 1
                if idx < 0 or idx >= len(staging_videos):
                    print("Invalid choice.")
                    return
                self._curate_video_interactive(staging_videos[idx])
            except ValueError:
                print("Invalid input.")

        def _curate_video_interactive(self, video: VideoMetadata):
            """Displays details of a specific staging video and prompts for curation
action."""

            # Load and count events
            rallies = []
            pts = []
            delivs = []
            if video.sport == SportType.BADMINTON:
                rallies = self.db.get_badminton_rallies(video.video_id)
            elif video.sport == SportType.PICKLEBALL:
                rallies = self.db.get_pickleball_rallies(video.video_id)
            elif video.sport == SportType.TABLE_TENNIS:
                rallies = self.db.get_tabletennis_rallies(video.video_id)
            elif video.sport == SportType.TENNIS:
                pts = self.db.get_tennis_points(video.video_id)
            elif video.sport == SportType.CRICKET:
                delivs = self.db.get_cricket_deliveries(video.video_id)

            while True:
                print("\n================ REVIEWING VIDEO ================")
                print(f"Match Name   : {video.match_name}")
                print(f"Video ID     : {video.video_id}")
                print(f"Sport        : {video.sport.value}")
```

```
716                 print(f"Video URL    : {video.video_url or 'N/A'}")
717                 print(f"Local Path   : {video.local_path or 'N/A'}")
718                 print(f"License      : {video.license_type.value} ({video.license_url or 'no
URL'})")
719                 print(f"Commercial OK: {video.is_commercial}")
720
721                 if video.sport in (SportType.BADMINTON, SportType.PICKLEBALL,
SportType.TABLE_TENNIS):
722                     print(f"Total Rallies: {len(rallies)}")
723                     if rallies:
724                         print("Sample rallies:")
725                         sample_data = [[r.rally_number, f"{r.start_time}s - {r.end_time}s",
r.score_after, r.shots_count] for r in rallies[:5]]
726                         print(tabulate(sample_data, headers=["Rally #", "Duration", "Score",
"Shots"], tablefmt="simple"))
727                 elif video.sport == SportType.TENNIS:
728                     print(f"Total Points : {len(pts)}")
729                     if pts:
730                         print("Sample points:")
731                         sample_data = [[p.point_number, f"{p.start_time}s - {p.end_time}s",
p.game_score] for p in pts[:5]]
732                         print(tabulate(sample_data, headers=["Point #", "Duration",
"Score"], tablefmt="simple"))
733                 elif video.sport == SportType.CRICKET:
734                     print(f"Total Balls  : {len(delivs)}")
735                     if delivs:
736                         print("Sample deliveries:")
737                         sample_data = [[f"{d.over}.{d.ball}", f"{d.start_time}s -
{d.end_time}s", d.runs, "Wicket" if d.wicket else ""] for d in delivs[:5]]
738                         print(tabulate(sample_data, headers=["Over.Ball", "Duration",
"Runs", "Wicket"], tablefmt="simple"))
739
740                 print("\nChoose Curation Action:")
741                 print("  [a] Approve (promote to CURATED)")
742                 print("  [r] Reject (delete from database)")
743                 print("  [p] Play a rally/point segment in browser")
744                 print("  [k] Keep in Staging (do nothing and go back)")
745
746                 action = input("> ").strip().lower()
747                 if action == 'a':
748                     self.db.update_status(video.video_id, CurationStatus.CURATED)
749                     print(f"Successfully promoted '{video.match_name}' to CURATED.\n")
750                     break
751                 elif action == 'r':
752                     self.db.delete_video(video.video_id)
753                     print(f"Successfully rejected and deleted '{video.match_name}'.\n")
754                     break
755                 elif action == 'k':
756                     print("Curation state unchanged.\n")
757                     break
758                 elif action == 'p':
759                     self._play_segment_interactive(video, rallies, pts, delivs)
760                 else:
761                     print("Invalid action selected.")
762
763         def _play_segment_interactive(self, video: VideoMetadata, rallies:
List[BadmintonRally], pts: List[TennisPoint], delivs: List[CricketDelivery]):
764             import webbrowser
765
766             # Get start/end based on user input
767             start_time = None
768             end_time = None
769             label = "segment"
770
771             if video.sport in (SportType.BADMINTON, SportType.PICKLEBALL,
```

```
SportType.TABLE_TENNIS):
                if not rallies:
                    print("No rallies to play.")
                    return
                idx_str = input(f"Enter Rally number to play (1-{len(rallies)}): ").strip()
                try:
                    rally_num = int(idx_str)
                    rally = next((r for r in rallies if r.rally_number == rally_num), None)
                    if rally:
                        start_time = rally.start_time
                        end_time = rally.end_time
                        label = f"Rally {rally_num}"
                except ValueError:
                    pass
            elif video.sport == SportType.TENNIS:
                if not pts:
                    print("No points to play.")
                    return
                idx_str = input(f"Enter Point number to play (1-{len(pts)}): ").strip()
                try:
                    pt_num = int(idx_str)
                    pt = next((p for p in pts if p.point_number == pt_num), None)
                    if pt:
                        start_time = pt.start_time
                        end_time = pt.end_time
                        label = f"Point {pt_num}"
                except ValueError:
                    pass
            elif video.sport == SportType.CRICKET:
                if not delivs:
                    print("No deliveries to play.")
                    return
                over_str = input("Enter Over number: ").strip()
                ball_str = input("Enter Ball number: ").strip()
                try:
                    o_num = int(over_str)
                    b_num = int(ball_str)
                    d = next((x for x in delivs if x.over == o_num and x.ball == b_num),
None)
                    if d:
                        start_time = d.start_time
                        end_time = d.end_time
                        label = f"Over {o_num} Ball {b_num}"
                except ValueError:
                    pass

            if start_time is None or end_time is None:
                print("Segment not found or invalid input.")
                return

            print(f"Playing {label} (start: {start_time}s, end: {end_time}s)...")

            # Check video location
            play_url = None
            if video.local_path and os.path.exists(video.local_path):
                abs_path = os.path.abspath(video.local_path)
                # Create file:// URL with media fragment (browsers support #t=start,end
fragment)
                play_url = f"file:///{abs_path.replace(os.sep,
'/')}#t={start_time},{end_time}"
            elif video.video_url:
                # YouTube URL: seek to start time
                if "youtube.com" in video.video_url or "youtu.be" in video.video_url:
                    t_sec = int(start_time)
                    sep = "&" if "?" in video.video_url else "?"
```

```
                    play_url = f"{video.video_url}{sep}t={t_sec}s"
                else:
                    play_url = video.video_url

            if play_url:
                print(f"Opening browser: {play_url}")
                webbrowser.open(play_url)
            else:
                print("Error: No local video file or remote URL available to play.")


    def main():
        parser = argparse.ArgumentParser(description="Sports Data Curator CLI")
        subparsers = parser.add_subparsers(dest="command")

        # Status command
        subparsers.add_parser("status", help="Display summary status of sports metadata
database")

        # Curate command
        subparsers.add_parser("curate", help="Launch interactive curation tool for staging
entries")

        # Import-local command
        import_parser = subparsers.add_parser("import-local", help="Manually import a local
video and annotations")
        import_parser.add_argument("--sport", required=True, help="Sport type (e.g.
badminton, tennis, cricket)")
        import_parser.add_argument("--video-path", required=True, help="Path to local video
file")
        import_parser.add_argument("--annotations-path", required=True, help="Path to local
JSON/CSV annotations file")
        import_parser.add_argument("--match-name", help="Name of the match (defaults to
video filename)")
        import_parser.add_argument("--video-id", help="Custom alphanumeric video ID")
        import_parser.add_argument("--license", default="Proprietary", help="License type
(e.g. MIT, CC-BY-NC, Proprietary)")
        import_parser.add_argument("--fps", type=float, default=30.0, help="Frames per
second of the video")
        import_parser.add_argument("--resolution", default="1080p", help="Resolution of the
video (e.g., 1080p, 720p)")

        # Fetch command ? (re)download videos at best resolution, update DB in place
        fetch_parser = subparsers.add_parser(
            "fetch", help="(Re)download videos at best available resolution, updating the DB
in place")
        fetch_parser.add_argument("--sport", help="Restrict to a single sport (default: all
supported_sports)")
        fetch_parser.add_argument("--video-id", help="Fetch a single video by ID")
        fetch_parser.add_argument("--status", choices=["staging", "curated", "all"],
default="all",
                                  help="Which curation statuses to fetch (default: all)")
        fetch_parser.add_argument("--min-height", type=int, default=720,
                                  help="Skip videos already at/above this height unless
--force (default: 720)")
        fetch_parser.add_argument("--max-height", type=int, default=None,
                                  help="Cap download resolution (e.g. 1080); default: best
available")
        fetch_parser.add_argument("--force", action="store_true",
                                  help="Re-download even if a good-enough local file already
exists")
        fetch_parser.add_argument("--include-noncommercial", action="store_true",
                                  help="Include videos whose license is not commercially
safe (default: commercial only)")
        fetch_parser.add_argument("--cookies-from-browser", dest="cookies_from_browser",
```

```
metavar="BROWSER",
                                help="Use a logged-in browser's YouTube cookies (e.g.
chrome, edge, firefox, brave). "
                                     "A Premium login gives reliable best-resolution
downloads. "
                                     "Chromium browsers must be CLOSED on Windows (cookie
DB lock).")
        fetch_parser.add_argument("--cookies", metavar="FILE",
                                help="Path to a Netscape-format cookies.txt (alternative
to --cookies-from-browser)")
        fetch_parser.add_argument("--player-client", dest="player_client",
default="default", metavar="CLIENT",
                                help="YouTube player client yt-dlp uses (default:
'default'). Avoid web/web_safari "
                                     "(SABR-forced -> drops to 144p). 'tv'/'mweb' also
serve 1080p if needed.")
        fetch_parser.add_argument("--sleep", type=float, default=4.0, metavar="SECONDS",
                                help="Pause between videos to avoid YouTube rate-limiting
(default: 4)")
        fetch_parser.add_argument("--retries", type=int, default=2, metavar="N",
                                help="Retries per video when a download fails or returns a
low format (default: 2)")
        fetch_parser.add_argument("--retry-backoff", type=float, default=20.0,
metavar="SECONDS",
                                help="Base backoff between retries; grows linearly per
attempt (default: 20)")

        # Convert-cvat command ? normalize a vendor CVAT export into our canonical CSV
        cvat_parser = subparsers.add_parser(
            "convert-cvat",
            help="Convert a vendor CVAT rally export (bounding-box XML) into the canonical
per-rally CSV")
        cvat_parser.add_argument("--input", required=True,
                                help="Path to the CVAT XML export (image- or track-mode)")
        cvat_parser.add_argument("--out", required=True,
                                help="Path to write the canonical per-rally CSV (import-
local input)")
        cvat_parser.add_argument("--video-path",
                                help="Source video; its fps is read via ffprobe (seconds =
frame / fps)")
        cvat_parser.add_argument("--fps", type=float,
                                help="Override fps instead of probing the video")
        cvat_parser.add_argument("--step", type=int,
                                help="Frame sampling step (default: read from CVAT meta,
else inferred)")
        cvat_parser.add_argument("--issues",
                                help="Path to also write the human-readable issues report")

        # Validate-delivery command ? run the full check suite + emit a structured report
        vd_parser = subparsers.add_parser(
            "validate-delivery",
            help="Validate a rally delivery (CVAT export or canonical CSV) and emit report +
review sheet")
        vd_parser.add_argument("--input", help="CVAT XML export (use with --video-
path/--fps)")
        vd_parser.add_argument("--csv", help="An already-canonical per-rally CSV to
validate")
        vd_parser.add_argument("--video-path",
                               help="Source video; fps via ffprobe (when using --input)")
        vd_parser.add_argument("--fps", type=float, help="Override fps (when using
--input)")
        vd_parser.add_argument("--video-id", help="Names the report files (<id>_report.md,
etc.)")
        vd_parser.add_argument("--out-dir", default="data/validation",
                               help="Where to write the report + review sheet")
```

```
925
926         # Export command
927         export_parser = subparsers.add_parser(
928             "export", help="Export curated annotations as indexer-ready eval/training CSVs +
manifest")
929         export_parser.add_argument("--out-dir", default="data/eval_export",
930                                    help="Output directory for per-video CSVs and
manifest.json")
931         export_parser.add_argument("--sport",
932                                    help="Restrict to a single sport (default: all
supported_sports)")
933         export_parser.add_argument("--include-staging", action="store_true",
934                                    help="Also export videos still in staging (default:
curated only)")
935         export_parser.add_argument("--include-noncommercial", action="store_true",
936                                    help="Include videos whose license is not commercially
safe (default: commercial only)")
937
938         args = parser.parse_args()
939         cli = SportsDataCLI()
940
941         if args.command == "status":
942             cli.status()
943         elif args.command == "curate":
944             cli.curate()
945         elif args.command == "import-local":
946             cli.import_local(args)
947         elif args.command == "convert-cvat":
948             cli.convert_cvat(args)
949         elif args.command == "validate-delivery":
950             cli.validate_delivery(args)
951         elif args.command == "export":
952             cli.export_eval_set(args)
953         elif args.command == "fetch":
954             cli.fetch(args)
955         else:
956             # Default: print help
957             parser.print_help()
958
959     if __name__ == "__main__":
960         main()
961
```

10. user (2026-06-10T05:33:40.596Z)

```
1       import sqlite3
2       import os
3       from contextlib import contextmanager
4       from typing import List, Dict, Any, Optional
5       from src.core.schemas.base import VideoMetadata, SportType, CurationStatus, LicenseType
6       from src.core.schemas.badminton import BadmintonRally
7       from src.core.schemas.tennis import TennisPoint
8       from src.core.schemas.cricket import CricketDelivery
9       from src.core.schemas.pickleball import PickleballRally
10      from src.core.schemas.tabletennis import TableTennisRally
11
12      # Maps each sport to its annotation table and the column(s) that order segments
13      # chronologically. All child tables expose (start_time, end_time), so a
14      # single generic reader can serve every sport ? adding a new sport is one entry
15      # here plus its CREATE TABLE, with no new query code.
16      SEGMENT_TABLES = {
17          SportType.BADMINTON: ("badminton_rallies", "rally_number"),
18          SportType.TENNIS: ("tennis_points", "point_number"),
19          SportType.CRICKET: ("cricket_deliveries", "over_num, ball_num"),
20          SportType.PICKLEBALL: ("pickleball_rallies", "rally_number"),
```

```
        SportType.TABLE_TENNIS: ("tabletennis_rallies", "rally_number"),
    }


    class SportsDatabase:
        def __init__(self, db_path: str):
            self.db_path = db_path
            # Ensure target directory exists
            os.makedirs(os.path.dirname(os.path.abspath(db_path)), exist_ok=True)
            self._init_db()

        @contextmanager
        def _get_connection(self):
            """Yield a SQLite connection and always close it.

            Used as ``with self._get_connection() as conn:``. A plain
            ``sqlite3.connect(...)`` used as a context manager commits/rolls back the
            transaction but does NOT close the connection, which leaks handles and
            raises ResourceWarnings. This wrapper guarantees the connection is closed
            on exit; writers still commit explicitly inside the block.
            """
            conn = sqlite3.connect(self.db_path)
            conn.row_factory = sqlite3.Row
            # SQLite disables foreign keys per-connection by default; enable so the
            # ON DELETE CASCADE declarations on the child tables actually fire.
            conn.execute("PRAGMA foreign_keys = ON")
            try:
                yield conn
            finally:
                conn.close()

        def _init_db(self):
            with self._get_connection() as conn:
                cursor = conn.cursor()

                # 1. Create common videos table
                cursor.execute("""
                CREATE TABLE IF NOT EXISTS videos (
                    video_id TEXT PRIMARY KEY,
                    sport TEXT NOT NULL,
                    video_url TEXT,
                    local_path TEXT,
                    license_type TEXT NOT NULL,
                    license_url TEXT,
                    is_commercial INTEGER NOT NULL DEFAULT 0,
                    status TEXT NOT NULL DEFAULT 'staging',
                    match_name TEXT NOT NULL,
                    fps REAL,
                    resolution TEXT
                )
                """)

                # 2. Create badminton_rallies table
                cursor.execute("""
                CREATE TABLE IF NOT EXISTS badminton_rallies (
                    video_id TEXT NOT NULL,
                    rally_number INTEGER NOT NULL,
                    start_time REAL NOT NULL,
                    end_time REAL NOT NULL,
                    score_before TEXT,
                    score_after TEXT,
                    shots_count INTEGER,
                    ending_reason TEXT,
                    last_shot_outcome TEXT,
                    PRIMARY KEY (video_id, rally_number),
```

```
                    FOREIGN KEY (video_id) REFERENCES videos(video_id) ON DELETE CASCADE
                )
                """)

                # 3. Create tennis_points table
                cursor.execute("""
                CREATE TABLE IF NOT EXISTS tennis_points (
                    video_id TEXT NOT NULL,
                    point_number INTEGER NOT NULL,
                    start_time REAL NOT NULL,
                    end_time REAL NOT NULL,
                    set_score TEXT,
                    game_score TEXT,
                    server TEXT,
                    ending_type TEXT,
                    PRIMARY KEY (video_id, point_number),
                    FOREIGN KEY (video_id) REFERENCES videos(video_id) ON DELETE CASCADE
                )
                """)

                # 4. Create cricket_deliveries table
                cursor.execute("""
                CREATE TABLE IF NOT EXISTS cricket_deliveries (
                    video_id TEXT NOT NULL,
                    over_num INTEGER NOT NULL,
                    ball_num INTEGER NOT NULL,
                    start_time REAL NOT NULL,
                    end_time REAL NOT NULL,
                    batsman TEXT,
                    bowler TEXT,
                    runs INTEGER,
                    wicket INTEGER DEFAULT 0,
                    extras INTEGER DEFAULT 0,
                    PRIMARY KEY (video_id, over_num, ball_num),
                    FOREIGN KEY (video_id) REFERENCES videos(video_id) ON DELETE CASCADE
                )
                """)

                # 5. Create pickleball_rallies table
                cursor.execute("""
                CREATE TABLE IF NOT EXISTS pickleball_rallies (
                    video_id TEXT NOT NULL,
                    rally_number INTEGER NOT NULL,
                    start_time REAL NOT NULL,
                    end_time REAL NOT NULL,
                    score_before TEXT,
                    score_after TEXT,
                    shots_count INTEGER,
                    ending_reason TEXT,
                    last_shot_outcome TEXT,
                    PRIMARY KEY (video_id, rally_number),
                    FOREIGN KEY (video_id) REFERENCES videos(video_id) ON DELETE CASCADE
                )
                """)

                # 6. Create tabletennis_rallies table
                cursor.execute("""
                CREATE TABLE IF NOT EXISTS tabletennis_rallies (
                    video_id TEXT NOT NULL,
                    rally_number INTEGER NOT NULL,
                    start_time REAL NOT NULL,
                    end_time REAL NOT NULL,
                    score_before TEXT,
                    score_after TEXT,
                    shots_count INTEGER,
```

```
                    ending_reason TEXT,
                    last_shot_outcome TEXT,
                    PRIMARY KEY (video_id, rally_number),
                    FOREIGN KEY (video_id) REFERENCES videos(video_id) ON DELETE CASCADE
                )
                """)

                # Migration: add last_shot_outcome to rally tables that predate the
                # column. CREATE TABLE IF NOT EXISTS won't alter an existing table, so
                # do it explicitly and idempotently (skip if the column is already there).
                for tbl in ("badminton_rallies", "pickleball_rallies",
"tabletennis_rallies"):
                    existing = [r[1] for r in cursor.execute(f"PRAGMA
table_info({tbl})").fetchall()]
                    if "last_shot_outcome" not in existing:
                        cursor.execute(f"ALTER TABLE {tbl} ADD COLUMN last_shot_outcome
TEXT")

                conn.commit()

        def insert_video(self, video: VideoMetadata) -> bool:
            """Insert a video entry, or update it in place if it already exists.

            Uses an UPSERT rather than INSERT OR REPLACE: REPLACE would DELETE the
            existing row, which (with foreign keys enabled) would cascade-delete the
            video's child annotations. UPSERT mutates the row in place and leaves
            children untouched.
            """
            with self._get_connection() as conn:
                cursor = conn.cursor()
                cursor.execute("""
                INSERT INTO videos (
                    video_id, sport, video_url, local_path, license_type,
                    license_url, is_commercial, status, match_name, fps, resolution
                ) VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?)
                ON CONFLICT(video_id) DO UPDATE SET
                    sport=excluded.sport,
                    video_url=excluded.video_url,
                    local_path=excluded.local_path,
                    license_type=excluded.license_type,
                    license_url=excluded.license_url,
                    is_commercial=excluded.is_commercial,
                    status=excluded.status,
                    match_name=excluded.match_name,
                    fps=excluded.fps,
                    resolution=excluded.resolution
                """, (
                    video.video_id,
                    video.sport.value,
                    video.video_url,
                    video.local_path,
                    video.license_type.value,
                    video.license_url,
                    1 if video.is_commercial else 0,
                    video.status.value,
                    video.match_name,
                    video.fps,
                    video.resolution
                ))
                conn.commit()
                return True

        def insert_badminton_rallies(self, rallies: List[BadmintonRally]) -> int:
            if not rallies:
                return 0
```

```
            with self._get_connection() as conn:
                cursor = conn.cursor()
                # First clear existing rallies for this video_id to avoid key conflicts on updates
                video_id = rallies[0].video_id
                cursor.execute("DELETE FROM badminton_rallies WHERE video_id = ?", (video_id,))

                for rally in rallies:
                    cursor.execute("""
                    INSERT INTO badminton_rallies (
                        video_id, rally_number, start_time, end_time,
                        score_before, score_after, shots_count, ending_reason, last_shot_outcome
                    ) VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?)
                    """, (
                        rally.video_id,
                        rally.rally_number,
                        rally.start_time,
                        rally.end_time,
                        rally.score_before,
                        rally.score_after,
                        rally.shots_count,
                        rally.ending_reason,
                        rally.last_shot_outcome
                    ))
                conn.commit()
                return len(rallies)

        def insert_tennis_points(self, points: List[TennisPoint]) -> int:
            if not points:
                return 0
            with self._get_connection() as conn:
                cursor = conn.cursor()
                video_id = points[0].video_id
                cursor.execute("DELETE FROM tennis_points WHERE video_id = ?", (video_id,))

                for pt in points:
                    cursor.execute("""
                    INSERT INTO tennis_points (
                        video_id, point_number, start_time, end_time,
                        set_score, game_score, server, ending_type
                    ) VALUES (?, ?, ?, ?, ?, ?, ?, ?)
                    """, (
                        pt.video_id,
                        pt.point_number,
                        pt.start_time,
                        pt.end_time,
                        pt.set_score,
                        pt.game_score,
                        pt.server,
                        pt.ending_type
                    ))
                conn.commit()
                return len(points)

        def insert_cricket_deliveries(self, deliveries: List[CricketDelivery]) -> int:
            if not deliveries:
                return 0
            with self._get_connection() as conn:
                cursor = conn.cursor()
                video_id = deliveries[0].video_id
                cursor.execute("DELETE FROM cricket_deliveries WHERE video_id = ?", (video_id,))

```

```
                for deliv in deliveries:
                    cursor.execute("""
                    INSERT INTO cricket_deliveries (
                        video_id, over_num, ball_num, start_time, end_time,
                        batsman, bowler, runs, wicket, extras
                    ) VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?)
                    """, (
                        deliv.video_id,
                        deliv.over,
                        deliv.ball,
                        deliv.start_time,
                        deliv.end_time,
                        deliv.batsman,
                        deliv.bowler,
                        deliv.runs,
                        1 if deliv.wicket else 0,
                        deliv.extras
                    ))
                conn.commit()
                return len(deliveries)

        def insert_pickleball_rallies(self, rallies: List[PickleballRally]) -> int:
            if not rallies:
                return 0
            with self._get_connection() as conn:
                cursor = conn.cursor()
                video_id = rallies[0].video_id
                cursor.execute("DELETE FROM pickleball_rallies WHERE video_id = ?",
(video_id,))

                for rally in rallies:
                    cursor.execute("""
                    INSERT INTO pickleball_rallies (
                        video_id, rally_number, start_time, end_time,
                        score_before, score_after, shots_count, ending_reason,
last_shot_outcome
                    ) VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?)
                    """, (
                        rally.video_id,
                        rally.rally_number,
                        rally.start_time,
                        rally.end_time,
                        rally.score_before,
                        rally.score_after,
                        rally.shots_count,
                        rally.ending_reason,
                        rally.last_shot_outcome
                    ))
                conn.commit()
                return len(rallies)

        def insert_tabletennis_rallies(self, rallies: List[TableTennisRally]) -> int:
            if not rallies:
                return 0
            with self._get_connection() as conn:
                cursor = conn.cursor()
                video_id = rallies[0].video_id
                cursor.execute("DELETE FROM tabletennis_rallies WHERE video_id = ?",
(video_id,))

                for rally in rallies:
                    cursor.execute("""
                    INSERT INTO tabletennis_rallies (
                        video_id, rally_number, start_time, end_time,
                        score_before, score_after, shots_count, ending_reason,
```

```
last_shot_outcome
                    ) VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?)
                    """, (
                        rally.video_id,
                        rally.rally_number,
                        rally.start_time,
                        rally.end_time,
                        rally.score_before,
                        rally.score_after,
                        rally.shots_count,
                        rally.ending_reason,
                        rally.last_shot_outcome
                    ))
                conn.commit()
                return len(rallies)

        def get_video(self, video_id: str) -> Optional[VideoMetadata]:
            with self._get_connection() as conn:
                cursor = conn.cursor()
                cursor.execute("SELECT * FROM videos WHERE video_id = ?", (video_id,))
                row = cursor.fetchone()
                if row:
                    return VideoMetadata(
                        video_id=row['video_id'],
                        sport=SportType(row['sport']),
                        video_url=row['video_url'],
                        local_path=row['local_path'],
                        license_type=LicenseType(row['license_type']),
                        license_url=row['license_url'],
                        is_commercial=bool(row['is_commercial']),
                        status=CurationStatus(row['status']),
                        match_name=row['match_name'],
                        fps=row['fps'],
                        resolution=row['resolution']
                    )
            return None

        def get_videos_by_status(self, status: CurationStatus, sport: Optional[SportType] =
None) -> List[VideoMetadata]:
            query = "SELECT * FROM videos WHERE status = ?"
            params = [status.value]
            if sport:
                query += " AND sport = ?"
                params.append(sport.value)

            videos = []
            with self._get_connection() as conn:
                cursor = conn.cursor()
                cursor.execute(query, params)
                for row in cursor.fetchall():
                    videos.append(VideoMetadata(
                        video_id=row['video_id'],
                        sport=SportType(row['sport']),
                        video_url=row['video_url'],
                        local_path=row['local_path'],
                        license_type=LicenseType(row['license_type']),
                        license_url=row['license_url'],
                        is_commercial=bool(row['is_commercial']),
                        status=CurationStatus(row['status']),
                        match_name=row['match_name'],
                        fps=row['fps'],
                        resolution=row['resolution']
                    ))
            return videos

```

```
        def get_status_counts(self) -> List[tuple]:
            """Return (sport, status, count) rows for the status summary.

            Public read API so callers (e.g. the CLI `status` command) don't have to
            reach into the private `_get_connection` context manager.
            """
            with self._get_connection() as conn:
                cursor = conn.cursor()
                cursor.execute(
                    "SELECT sport, status, COUNT(*) as cnt FROM videos GROUP BY status,
sport"
                )
                return [(r["sport"], r["status"], r["cnt"]) for r in cursor.fetchall()]

        def get_badminton_rallies(self, video_id: str) -> List[BadmintonRally]:
            rallies = []
            with self._get_connection() as conn:
                cursor = conn.cursor()
                cursor.execute("""
                SELECT * FROM badminton_rallies
                WHERE video_id = ?
                ORDER BY rally_number
                """, (video_id,))
                for row in cursor.fetchall():
                    rallies.append(BadmintonRally(
                        video_id=row['video_id'],
                        rally_number=row['rally_number'],
                        start_time=row['start_time'],
                        end_time=row['end_time'],
                        score_before=row['score_before'],
                        score_after=row['score_after'],
                        shots_count=row['shots_count'],
                        ending_reason=row['ending_reason'],
                        last_shot_outcome=row['last_shot_outcome']
                    ))
            return rallies

        def get_tennis_points(self, video_id: str) -> List[TennisPoint]:
            points = []
            with self._get_connection() as conn:
                cursor = conn.cursor()
                cursor.execute("""
                SELECT * FROM tennis_points
                WHERE video_id = ?
                ORDER BY point_number
                """, (video_id,))
                for row in cursor.fetchall():
                    points.append(TennisPoint(
                        video_id=row['video_id'],
                        point_number=row['point_number'],
                        start_time=row['start_time'],
                        end_time=row['end_time'],
                        set_score=row['set_score'],
                        game_score=row['game_score'],
                        server=row['server'],
                        ending_type=row['ending_type']
                    ))
            return points

        def get_cricket_deliveries(self, video_id: str) -> List[CricketDelivery]:
            deliveries = []
            with self._get_connection() as conn:
                cursor = conn.cursor()
                cursor.execute("""
                SELECT * FROM cricket_deliveries
```

```
                WHERE video_id = ?
                ORDER BY over_num, ball_num
                """, (video_id,))
                for row in cursor.fetchall():
                    deliveries.append(CricketDelivery(
                        video_id=row['video_id'],
                        over=row['over_num'],
                        ball=row['ball_num'],
                        start_time=row['start_time'],
                        end_time=row['end_time'],
                        batsman=row['batsman'],
                        bowler=row['bowler'],
                        runs=row['runs'],
                        wicket=bool(row['wicket']),
                        extras=row['extras']
                    ))
            return deliveries

        def get_pickleball_rallies(self, video_id: str) -> List[PickleballRally]:
            rallies = []
            with self._get_connection() as conn:
                cursor = conn.cursor()
                cursor.execute("""
                SELECT * FROM pickleball_rallies
                WHERE video_id = ?
                ORDER BY rally_number
                """, (video_id,))
                for row in cursor.fetchall():
                    rallies.append(PickleballRally(
                        video_id=row['video_id'],
                        rally_number=row['rally_number'],
                        start_time=row['start_time'],
                        end_time=row['end_time'],
                        score_before=row['score_before'],
                        score_after=row['score_after'],
                        shots_count=row['shots_count'],
                        ending_reason=row['ending_reason'],
                        last_shot_outcome=row['last_shot_outcome']
                    ))
            return rallies

        def get_tabletennis_rallies(self, video_id: str) -> List[TableTennisRally]:
            rallies = []
            with self._get_connection() as conn:
                cursor = conn.cursor()
                cursor.execute("""
                SELECT * FROM tabletennis_rallies
                WHERE video_id = ?
                ORDER BY rally_number
                """, (video_id,))
                for row in cursor.fetchall():
                    rallies.append(TableTennisRally(
                        video_id=row['video_id'],
                        rally_number=row['rally_number'],
                        start_time=row['start_time'],
                        end_time=row['end_time'],
                        score_before=row['score_before'],
                        score_after=row['score_after'],
                        shots_count=row['shots_count'],
                        ending_reason=row['ending_reason'],
                        last_shot_outcome=row['last_shot_outcome']
                    ))
            return rallies

        def get_segment_intervals(self, video_id: str, sport: SportType) -> List[tuple]:
```

```
528             """Return chronologically-ordered (start_time, end_time) tuples for a video,
529             regardless of sport. This is the sport-agnostic view used to export
530             indexer-ready evaluation/training labels.
531
532             Sports that have no annotation table yet (e.g. tabletennis, pickleball)
533             yield an empty list rather than raising.
534             """
535             mapping = SEGMENT_TABLES.get(sport)
536             if mapping is None:
537                 return []
538             table, order_by = mapping  # both come from the trusted map above, never user
input
539             with self._get_connection() as conn:
540                 cursor = conn.cursor()
541                 cursor.execute(
542                     f"SELECT start_time, end_time FROM {table} "
543                     f"WHERE video_id = ? ORDER BY {order_by}",
544                     (video_id,),
545                 )
546                 return [(row["start_time"], row["end_time"]) for row in cursor.fetchall()]
547
548         def update_status(self, video_id: str, status: CurationStatus) -> bool:
549             with self._get_connection() as conn:
550                 cursor = conn.cursor()
551                 cursor.execute("UPDATE videos SET status = ? WHERE video_id = ?",
(status.value, video_id))
552                 conn.commit()
553                 return cursor.rowcount > 0
554
555         def delete_video(self, video_id: str) -> bool:
556             """Deletes a video and cascading child entries."""
557             with self._get_connection() as conn:
558                 cursor = conn.cursor()
559                 # Execute cascade deletes manually just in case SQLite foreign key support
is not active
560                 cursor.execute("DELETE FROM badminton_rallies WHERE video_id = ?",
(video_id,))
561                 cursor.execute("DELETE FROM tennis_points WHERE video_id = ?", (video_id,))
562                 cursor.execute("DELETE FROM cricket_deliveries WHERE video_id = ?",
(video_id,))
563                 cursor.execute("DELETE FROM pickleball_rallies WHERE video_id = ?",
(video_id,))
564                 cursor.execute("DELETE FROM tabletennis_rallies WHERE video_id = ?",
(video_id,))
565                 cursor.execute("DELETE FROM videos WHERE video_id = ?", (video_id,))
566                 conn.commit()
567                 return cursor.rowcount > 0
568
```

11. user (2026-06-10T05:33:47.272Z)

```
1       """Adapter: CVAT rally exports -> our canonical per-rally records.
2
3       Why this exists
4       ---------------
5       Our annotation brief (``docs/annotation/ROORA_BRIEF.md``) asks vendors for a
6       per-rally **CSV** that imports directly via ``curate import-local``. A vendor may
7       instead do the work in CVAT and hand back the *CVAT* export: per-frame bounding
8       boxes, with the rally fields (``rally_number``, ``shots_count``,
9       ``ending_reason`` ...) carried as CVAT **object attributes** on each box. The box
10      geometry is irrelevant to us ? it is just a carrier the rater attached the rally
11      metadata to.
12
13      This module is the ingestion-side adapter that absorbs that vendor format so the
14      modeling repo never has to. It does two things, both deliberately on *our* side
```

```
     so we fix mechanical problems rather than renegotiating them:

     1. **Recompute time from the frame range.** A rally's interval is taken from the
        first/last *frame* a box for that rally appears on:
        ``seconds = frame / fps`` at the video's true fps. We **ignore** any
        ``start_time``/``end_time`` attributes the vendor wrote ? a CVAT job that
        samples every ``step`` frames and assumes 30 fps yields systematically wrong
        times (we have seen ``start_time == frame / (step * 30)``, i.e. off by the
        sampling step at the wrong fps). The frame index is reliable; the vendor's
        seconds are not.

     2. **Surface ? never silently fix ? content defects.** Overlapping rallies,
        zero-duration rallies, ending_reason values outside the agreed set, and
        too-coarse sampling are reported as :class:`Issue`s. Guessing a corrected
        rally boundary would poison ground truth, so that stays a human/vendor call.

     Supported input: the single-file CVAT XML export, in either **image mode**
     (``<image><box>...``) or **track/interpolation mode** (``<track><box frame=...>``).
     The two PASCAL-VOC / COCO sibling exports encode the same data and can be added
     later if a vendor sends only those.
     """
     from __future__ import annotations

     import os
     import xml.etree.ElementTree as ET
     from collections import Counter
     from dataclasses import dataclass, field
     from typing import Any, Dict, List, Optional, Tuple

     from src.core.parsing import safe_int
     from src.core.downloader import probe_resolution_fps

     # The agreed cross-sport ending_reason vocabulary for net-separated sports.
     # Kept here as a soft check only (a warning, never a hard failure) so this
     # adapter does not depend on the annotation-spec wording.
     DEFAULT_ALLOWED_REASONS = {
         "winner",
         "forced_error",
         "unforced_error",
         "service_fault",
         "let",
         "other",
     }

     # The observational outcome of the last shot ? orthogonal to ending_reason
     # (winner can be 'in'; an error can be 'net' or 'out'; service_fault/let -> 'n_a').
     DEFAULT_ALLOWED_OUTCOMES = {"in", "net", "out", "n_a"}

     # CVAT attribute names we map onto canonical columns. The vendor's CVAT label
     # schema has used suffixed names like "score_before (A-B)"; we match leniently.
     _ATTR_ALIASES = {
         "rally_number": ("rally_number",),
         "shots_count": ("shots_count",),
         "ending_reason": ("ending_reason",),
         "last_shot_outcome": ("last_shot_outcome", "outcome", "last_shot_outcome
(in/net/out)"),
         "score_before": ("score_before", "score_before (a-b)"),
         "score_after": ("score_after", "score_after (a-b)"),
         "notes": ("notes", "note"),
     }

     CANONICAL_COLUMNS = [
         "rally_number", "start_mmss", "end_mmss", "start_time", "end_time",
         "shots_count", "ending_reason", "last_shot_outcome", "score_before", "score_after",
"notes",
```

```
    ]


    @dataclass
    class RallyRecord:
        """One rally recovered from the CVAT export, time re-derived from frames."""
        rally_number: int
        start_time: float            # seconds, = frame_min / fps
        end_time: float              # seconds, = frame_max / fps (see padding note)
        shots_count: Optional[int]
        ending_reason: Optional[str]
        last_shot_outcome: Optional[str]
        score_before: Optional[str]
        score_after: Optional[str]
        notes: Optional[str]
        frame_min: int
        frame_max: int
        n_boxes: int
        vendor_start_time: Optional[str] = None   # the (discarded) attribute value
        vendor_end_time: Optional[str] = None

        @property
        def duration(self) -> float:
            return self.end_time - self.start_time


    @dataclass
    class Issue:
        """A defect found in the delivery. ``blocker`` issues stop a clean import."""
        severity: str   # 'blocker' | 'high' | 'medium' | 'low'
        code: str
        message: str
        rally_number: Optional[int] = None


    @dataclass
    class ConversionResult:
        records: List[RallyRecord]
        issues: List[Issue]
        fps: float
        step: int
        meta: Dict[str, Any] = field(default_factory=dict)

        @property
        def n_blockers(self) -> int:
            return sum(1 for i in self.issues if i.severity == "blocker")


    def _mmss(seconds: float) -> str:
        """Format seconds as ``m:ss.mmm`` (matches the brief's mm:ss columns)."""
        m = int(seconds // 60)
        s = seconds - m * 60
        return f"{m}:{s:06.3f}"


    def _attr_lookup(attrs: Dict[str, str], canonical: str) -> Optional[str]:
        for alias in _ATTR_ALIASES[canonical]:
            if alias in attrs and attrs[alias] != "":
                return attrs[alias]
        return None


    def parse_cvat_boxes(xml_path: str) -> Tuple[Dict[str, Any], List[Dict[str, Any]]]:
        """Parse a CVAT XML export into (meta, boxes).

```

```
        ``boxes`` is a flat list of ``{"frame": int, "attrs": {name: value}}`` ? one
        entry per visible box, from either image-mode or track-mode exports.
        ``meta`` carries ``step`` (frame_filter) and ``stop_frame`` when present.
        """
        if not os.path.exists(xml_path):
            raise FileNotFoundError(f"CVAT export not found: {xml_path}")

        root = ET.parse(xml_path).getroot()
        meta: Dict[str, Any] = {}

        job = root.find(".//job")
        if job is not None:
            stop = job.findtext("stop_frame")
            if stop is not None and stop.strip().isdigit():
                meta["stop_frame"] = int(stop.strip())
            ff = job.findtext("frame_filter") or ""
            if "step=" in ff:
                try:
                    meta["step"] = int(ff.split("step=")[1].split()[0])
                except (ValueError, IndexError):
                    pass

        def _attrs(el: ET.Element) -> Dict[str, str]:
            out: Dict[str, str] = {}
            for a in el.findall("attribute"):
                out[(a.get("name") or "").strip().lower()] = (a.text or "").strip()
            return out

        boxes: List[Dict[str, Any]] = []

        # Image mode: <image id=frame> ... <box>...</box>
        for img in root.findall("image"):
            try:
                frame = int(img.get("id"))
            except (TypeError, ValueError):
                continue
            for box in img.findall("box"):
                boxes.append({"frame": frame, "attrs": _attrs(box)})

        # Track mode: <track> ... <box frame=N outside=0>...</box>
        for track in root.findall("track"):
            for box in track.findall("box"):
                if box.get("outside") == "1":
                    continue  # a track's terminating keyframe carries no real label
                try:
                    frame = int(box.get("frame"))
                except (TypeError, ValueError):
                    continue
                boxes.append({"frame": frame, "attrs": _attrs(box)})

        return meta, boxes


    def _infer_step(frames: List[int]) -> int:
        """Most common positive gap between consecutive distinct sampled frames."""
        uniq = sorted(set(frames))
        gaps = [b - a for a, b in zip(uniq, uniq[1:]) if b - a > 0]
        if not gaps:
            return 1
        return Counter(gaps).most_common(1)[0][0]


    def build_records(boxes: List[Dict[str, Any]], fps: float) -> List[RallyRecord]:
        """Group boxes by rally_number and derive each rally's frame range -> seconds."""
        if fps is None or fps <= 0:
```

```
            raise ValueError("fps must be a positive number to convert frames to seconds")

        by_rally: Dict[int, List[Dict[str, Any]]] = {}
        for b in boxes:
            rn = safe_int(_attr_lookup(b["attrs"], "rally_number"), None)
            if rn is None:
                continue
            by_rally.setdefault(rn, []).append(b)

        records: List[RallyRecord] = []
        for rn in sorted(by_rally):
            items = by_rally[rn]
            frames = sorted(b["frame"] for b in items)
            attrs = items[0]["attrs"]
            fmin, fmax = frames[0], frames[-1]
            records.append(RallyRecord(
                rally_number=rn,
                start_time=fmin / fps,
                end_time=fmax / fps,
                shots_count=safe_int(_attr_lookup(attrs, "shots_count"), None),
                ending_reason=_attr_lookup(attrs, "ending_reason"),
                last_shot_outcome=_attr_lookup(attrs, "last_shot_outcome"),
                score_before=_attr_lookup(attrs, "score_before"),
                score_after=_attr_lookup(attrs, "score_after"),
                notes=_attr_lookup(attrs, "notes"),
                frame_min=fmin,
                frame_max=fmax,
                n_boxes=len(frames),
                vendor_start_time=attrs.get("start_time"),
                vendor_end_time=attrs.get("end_time"),
            ))
        return records


    def detect_issues(records: List[RallyRecord],
                      stop_frame: Optional[int] = None,
                      allowed_reasons: Optional[set] = None,
                      allowed_outcomes: Optional[set] = None,
                      max_gap_s: float = 15.0) -> List[Issue]:
        """Find the defects that block or degrade a clean ground-truth import.

        Note: we deliberately do NOT flag the sampling density (step/fps). A boundary
        slack of ~0.5s is acceptable for this pipeline because the downstream
        highlight compiler pads every clip on output (badminton-highlight-indexer,
        backend/stitcher/compiler.py: begin_padding=0.5s, end_padding=1.0s), which
        absorbs the sampling uncertainty when highlights are trimmed.
        """
        allowed = allowed_reasons if allowed_reasons is not None else
DEFAULT_ALLOWED_REASONS
        allowed_out = allowed_outcomes if allowed_outcomes is not None else
DEFAULT_ALLOWED_OUTCOMES
        issues: List[Issue] = []

        ordered = sorted(records, key=lambda r: r.rally_number)
        for i, r in enumerate(ordered):
            if r.end_time <= r.start_time:
                issues.append(Issue(
                    "blocker", "zero_duration",
                    f"Rally {r.rally_number}: end <= start "
                    f"(frames {r.frame_min}-{r.frame_max}); single-sample or stray box.",
                    r.rally_number))

            if r.ending_reason is not None and r.ending_reason not in allowed:
                issues.append(Issue(
                    "medium", "ending_reason_vocab",
```

```
                    f"Rally {r.rally_number}: ending_reason '{r.ending_reason}' "
                    f"not in the agreed set {sorted(allowed)}.",
                    r.rally_number))

            if r.last_shot_outcome is not None and r.last_shot_outcome not in allowed_out:
                issues.append(Issue(
                    "low", "outcome_vocab",
                    f"Rally {r.rally_number}: last_shot_outcome '{r.last_shot_outcome}' "
                    f"not in {sorted(allowed_out)}.",
                    r.rally_number))

            # Inflated span: a long rally with implausibly few shots usually means a
            # stray/duplicated box stretched the frame range past the real ending.
            if r.shots_count and r.duration >= 5.0:
                sps = r.shots_count / r.duration
                if sps < 0.3:
                    issues.append(Issue(
                        "medium", "shot_density",
                        f"Rally {r.rally_number}: {r.shots_count} shots over "
                        f"{r.duration:.1f}s ({sps:.2f}/s) is implausibly low ? the span "
                        f"may be inflated by a stray box.",
                        r.rally_number))

            if stop_frame is not None and r.frame_max > stop_frame:
                issues.append(Issue(
                    "high", "frame_out_of_range",
                    f"Rally {r.rally_number}: frame {r.frame_max} exceeds the video's "
                    f"last frame {stop_frame}.",
                    r.rally_number))

            if i > 0:
                prev = ordered[i - 1]
                if r.start_time < prev.end_time:
                    issues.append(Issue(
                        "blocker", "overlap",
                        f"Rally {prev.rally_number} (ends {prev.end_time:.2f}s) overlaps "
                        f"Rally {r.rally_number} (starts {r.start_time:.2f}s) by "
                        f"{prev.end_time - r.start_time:.2f}s ? rallies must be disjoint.",
                        r.rally_number))
                elif r.start_time - prev.end_time > max_gap_s:
                    # A long unlabeled stretch between two rallies is a cheap, label-only
                    # second opinion for the worst error (a missed rally). It won't catch a
                    # rally MERGED into its neighbour's span (shot_density/inflated catches
                    # that), but it does catch an omitted rally that left a visible hole.
                    gap = r.start_time - prev.end_time
                    issues.append(Issue(
                        "medium", "possible_missed_rally",
                        f"{gap:.1f}s of unlabeled time between Rally {prev.rally_number} "
                        f"(ends {prev.end_time:.1f}s) and Rally {r.rally_number} "
                        f"(starts {r.start_time:.1f}s) ? check for an unlabeled rally in the
gap.",
                        r.rally_number))
        return issues


    def convert(xml_path: str,
                fps: Optional[float] = None,
                video_path: Optional[str] = None,
                step: Optional[int] = None,
                allowed_reasons: Optional[set] = None) -> ConversionResult:
        """Full pipeline: parse CVAT XML -> records (+ issues) at the true fps.

        ``fps`` resolution order: explicit ``fps`` arg > ffprobe of ``video_path`` >
        the CVAT meta (none) -> error. We never fall back to a guessed 30 fps, since
        that silent guess is exactly the bug this adapter exists to undo.
```

```
        """
        meta, boxes = parse_cvat_boxes(xml_path)

        if fps is None and video_path:
            _, _, probed = probe_resolution_fps(video_path)
            fps = probed
        if fps is None or fps <= 0:
            raise ValueError(
                "Could not determine fps. Pass fps=... or a readable video_path so "
                "frames can be converted to seconds (seconds = frame / fps).")

        if step is None:
            step = meta.get("step") or _infer_step([b["frame"] for b in boxes])

        records = build_records(boxes, fps)
        issues = detect_issues(records,
                               stop_frame=meta.get("stop_frame"),
                               allowed_reasons=allowed_reasons)
        return ConversionResult(records=records, issues=issues, fps=fps, step=step,
meta=meta)


    def write_canonical_csv(records: List[RallyRecord], out_path: str) -> None:
        """Write records as the canonical per-rally CSV that ``import-local`` reads."""
        import csv
        os.makedirs(os.path.dirname(os.path.abspath(out_path)), exist_ok=True)
        with open(out_path, "w", newline="", encoding="utf-8") as f:
            w = csv.writer(f)
            w.writerow(CANONICAL_COLUMNS)
            for r in sorted(records, key=lambda x: x.rally_number):
                w.writerow([
                    r.rally_number,
                    _mmss(r.start_time), _mmss(r.end_time),
                    f"{r.start_time:.3f}", f"{r.end_time:.3f}",
                    r.shots_count if r.shots_count is not None else "",
                    r.ending_reason or "",
                    r.last_shot_outcome or "",
                    r.score_before or "", r.score_after or "", r.notes or "",
                ])


    def render_issue_report(result: ConversionResult, source: str) -> str:
        """A human-readable issue report (for review / sending back to the vendor)."""
        lines = [
            f"CVAT rally conversion report ? {source}",
            f"  fps used        : {result.fps:.3f}",
            f"  sampling step   : {result.step} frames (~{result.step / result.fps:.2f}s)",
            f"  rallies parsed  : {len(result.records)}",
            f"  issues          : {len(result.issues)} ({result.n_blockers} blocker)",
            "",
        ]
        if not result.issues:
            lines.append("No issues detected.")
        else:
            for sev in ("blocker", "high", "medium", "low"):
                group = [i for i in result.issues if i.severity == sev]
                if not group:
                    continue
                lines.append(f"[{sev.upper()}]")
                for i in group:
                    lines.append(f"  - ({i.code}) {i.message}")
                lines.append("")
        return "\n".join(lines)


```

```
    # ---------------------------------------------------------------------------- #
    #  Validation reporting ? used by `curate validate-delivery`.                  #
    #  These turn a (records, issues) pair into the artifacts a human reviews:     #
    #  a machine summary (JSON), a human report (md), and an annotatable sheet.    #
    # ---------------------------------------------------------------------------- #

    def _parse_secs(value: Optional[str]) -> Optional[float]:
        """Decimal seconds or mm:ss / hh:mm:ss -> float; None if blank/unparseable."""
        s = (value or "").strip()
        if not s:
            return None
        if ":" in s:
            try:
                total = 0.0
                for part in s.split(":"):
                    total = total * 60 + float(part)
                return total
            except ValueError:
                return None
        try:
            return float(s)
        except ValueError:
            return None


    def load_canonical_csv(csv_path: str) -> List[RallyRecord]:
        """Load a canonical per-rally CSV into RallyRecords, so a *CSV* delivery (not
        just a CVAT export) can be validated. Decimal start/end preferred, mm:ss
        fallback; rows with no usable start/end are skipped. (frame_* are -1 here, as a
        CSV carries no frame info ? only the frame-derived checks become inapplicable.)
        """
        import csv as _csv
        if not os.path.exists(csv_path):
            raise FileNotFoundError(f"CSV not found: {csv_path}")
        records: List[RallyRecord] = []
        with open(csv_path, newline="", encoding="utf-8") as f:
            reader = _csv.DictReader(f)
            reader.fieldnames = [(n or "").strip().lower() for n in (reader.fieldnames or
[])]
            for raw in reader:
                row = {(k or "").strip().lower(): v for k, v in raw.items()}
                rn = safe_int(row.get("rally_number"), None)
                if rn is None:
                    continue
                start = _parse_secs(row.get("start_time"))
                if start is None:
                    start = _parse_secs(row.get("start_mmss"))
                end = _parse_secs(row.get("end_time"))
                if end is None:
                    end = _parse_secs(row.get("end_mmss"))
                if start is None or end is None:
                    continue
                records.append(RallyRecord(
                    rally_number=rn, start_time=start, end_time=end,
                    shots_count=safe_int(row.get("shots_count"), None),
                    ending_reason=(row.get("ending_reason") or "").strip() or None,
                    last_shot_outcome=(row.get("last_shot_outcome") or "").strip() or None,
                    score_before=(row.get("score_before") or "").strip() or None,
                    score_after=(row.get("score_after") or "").strip() or None,
                    notes=(row.get("notes") or "").strip() or None,
                    frame_min=-1, frame_max=-1, n_boxes=0,
                ))
        return records


```

```
    def build_summary(records: List[RallyRecord], issues: List[Issue],
                      source: str = "", fps: Optional[float] = None) -> Dict[str, Any]:
        """Machine-readable validation summary (JSON / CI)."""
        by_sev = {s: [i for i in issues if i.severity == s]
                  for s in ("blocker", "high", "medium", "low")}
        flagged = sorted({i.rally_number for i in issues if i.rally_number is not None})
        return {
            "source": source,
            "fps": fps,
            "n_rallies": len(records),
            "n_issues": len(issues),
            "counts_by_severity": {s: len(v) for s, v in by_sev.items()},
            "n_blockers": len(by_sev["blocker"]),
            "passed": len(by_sev["blocker"]) == 0,
            "flagged_rallies": flagged,
            "issues": [{"severity": i.severity, "code": i.code,
                        "rally_number": i.rally_number, "message": i.message} for i in
issues],
        }


    def render_validation_md(records: List[RallyRecord], issues: List[Issue], source: str)
-> str:
        """Human validation report: scoreboard + issues by severity + rallies to review."""
        n = len(records)
        by_sev = {s: [i for i in issues if i.severity == s]
                  for s in ("blocker", "high", "medium", "low")}
        flagged = sorted({i.rally_number for i in issues if i.rally_number is not None})
        clean = n - len(flagged)
        lines = [
            f"# Delivery validation ? {source}",
            "",
            f"- rallies: **{n}**",
            f"- issues: **{len(issues)}**  (blocker {len(by_sev['blocker'])}, "
            f"high {len(by_sev['high'])}, medium {len(by_sev['medium'])}, low
{len(by_sev['low'])})",
            f"- structurally-clean rallies: **{clean}/{n}**"
            + (f" ({100 * clean // n}%)" if n else ""),
            f"- gate: **{'PASS' if not by_sev['blocker'] else 'BLOCKED'}**",
            "",
        ]
        if issues:
            for sev in ("blocker", "high", "medium", "low"):
                grp = by_sev[sev]
                if not grp:
                    continue
                lines.append(f"## {sev.upper()}")
                lines += [f"- ({i.code}) {i.message}" for i in grp]
                lines.append("")
        else:
            lines.append("No issues detected. ?\n")
        if flagged:
            lines.append(f"## Rallies to review: {flagged}")
        return "\n".join(lines)


    def write_review_csv(records: List[RallyRecord], issues: List[Issue], out_path: str) ->
None:
        """Auto-generate the annotatable manual-verification sheet (blank `true_*` cols)."""
        import csv
        flags: Dict[int, List[str]] = {}
        for i in issues:
            if i.rally_number is not None:
                flags.setdefault(i.rally_number, []).append(i.code)
        os.makedirs(os.path.dirname(os.path.abspath(out_path)), exist_ok=True)
```

```
        cols = ["rally_number", "start_mmss", "end_mmss", "dur_s", "shots_count",
                "ending_reason", "last_shot_outcome", "auto_flags",
                "VERIFIED(Y/N)", "true_start_mmss", "true_end_mmss",
                "true_ending_reason", "true_last_shot_outcome", "reviewer_comment"]
        with open(out_path, "w", newline="", encoding="utf-8") as f:
            w = csv.writer(f)
            w.writerow(cols)
            for r in sorted(records, key=lambda x: x.rally_number):
                w.writerow([
                    r.rally_number, _mmss(r.start_time), _mmss(r.end_time),
                    f"{r.duration:.1f}", r.shots_count if r.shots_count is not None else "",
                    r.ending_reason or "", r.last_shot_outcome or "",
                    ";".join(flags.get(r.rally_number, [])),
                    "", "", "", "", "", "",
                ])

```

12. user (2026-06-10T05:33:52.750Z)

```
    """Normalize a VLC rally-annotator CSV into a clean lean CSV that ``import-local``
    ingests safely.

    Two verified landmines in PARTIAL human annotation data (a video tagged only up to
    some point, with mid-session corrections):

      1. An unusable time aborts the WHOLE import (one bad row kills the entire video,
         via the try/except around _parse_local_annotations in curate.py import-local):
         a blank/missing time makes ``safe_float_required`` raise; a reversed
         ``end <= start`` or a negative ``start`` makes the ``BadmintonRally`` pydantic
         validator raise. -> we DROP such rows, mirroring the downstream indexer loader
         which already skips ``not end > start``.
      2. A fractional or duplicate ``rally_number`` (e.g. ``7.5`` inserted for a rally
         the annotator missed) truncates via ``int(float())`` and COLLIDES on the
         ``(video_id, rally_number)`` primary key; the DELETE-then-INSERT upsert then
         silently destroys one rally. -> we RENUMBER 1..N by start_time, recording the
         original id for traceability.

    It also accepts mm:ss / HH:MM:SS times (converted to decimal seconds) and folds a
    pipe-delimited ``ending_reason`` ('forced_error|other') to its primary token, so the
    output is always the canonical lean schema::

        rally_number,start_time,end_time,ending_reason,sport[,shots_count]

    plus traceability columns (``original_rally_number``, ``notes``) that import-local
    ignores. Standalone on purpose: it does NOT touch the DB schema or the
    rally_number-ordered export. Run it, then feed the output to ``curate import-local``.

        python -m src.cli.normalize_annotations --in "Match.rallies.csv" --out clean.csv
    """
    from __future__ import annotations

    import argparse
    import csv
    import os
    from dataclasses import dataclass, field
    from typing import Any, Dict, List, Optional, Tuple

    # Canonical lean output columns + traceability (import-local reads the first four/
    # five; original_rally_number + notes are ignored by the importer but kept for humans).
    OUT_FIELDS = ["rally_number", "start_time", "end_time", "ending_reason", "sport",
                  "shots_count", "original_rally_number", "notes"]


    @dataclass
    class NormalizeReport:
```

```
47          rows_in: int = 0
48          rows_out: int = 0
49          dropped: List[Dict[str, Any]] = field(default_factory=list)  # {orig_id, reason,
start, end}
50          renumbered: bool = False
51          fractional_ids: List[str] = field(default_factory=list)
52          duplicate_ids: List[str] = field(default_factory=list)
53          mmss_converted: int = 0
54
55          def summary(self) -> str:
56              lines = [f"rows in: {self.rows_in}  ->  rows out: {self.rows_out}"]
57              if self.dropped:
58                  lines.append(f"dropped {len(self.dropped)} bad row(s):")
59                  for d in self.dropped:
60                      lines.append(f"  - orig id {d['orig_id']!r}: {d['reason']} "
61                                   f"(start={d['start']!r}, end={d['end']!r})")
62              if self.renumbered:
63                  extra = []
64                  if self.fractional_ids:
65                      extra.append(f"fractional ids {self.fractional_ids}")
66                  if self.duplicate_ids:
67                      extra.append(f"duplicate ids {self.duplicate_ids}")
68                  lines.append("renumbered rallies 1..N by start_time"
69                               + (f" ({'; '.join(extra)})" if extra else ""))
70              if self.mmss_converted:
71                  lines.append(f"converted {self.mmss_converted} mm:ss timestamp(s) to
seconds")
72              if not self.dropped and not self.renumbered and not self.mmss_converted:
73                  lines.append("clean input ? no changes beyond column normalization")
74              return "\n".join(lines)
75
76
77      def _to_seconds(raw: Any) -> Optional[Tuple[float, bool]]:
78          """(seconds, was_mmss) or None if blank/unparseable. Decimal seconds or mm:ss /
HH:MM:SS."""
79          if raw is None:
80              return None
81          s = str(raw).strip()
82          if not s:
83              return None
84          try:
85              if ":" in s:
86                  parts = [float(p) for p in s.split(":")]
87                  if len(parts) == 3:
88                      return parts[0] * 3600 + parts[1] * 60 + parts[2], True
89                  if len(parts) == 2:
90                      return parts[0] * 60 + parts[1], True
91                  return None
92              return float(s), False
93          except ValueError:
94              return None
95
96
97      def _primary_reason(raw: Any) -> Tuple[str, str]:
98          """Pipe-delimited 'forced_error|other' -> ('forced_error', 'other'). Blank -> ('',
'')."""
99          if raw is None:
100             return "", ""
101         s = str(raw).strip()
102         if not s:
103             return "", ""
104         parts = [p.strip() for p in s.split("|") if p.strip()]
105         if not parts:
106             return "", ""
107         return parts[0], "; ".join(parts[1:])
```

```
    def _get(row: Dict[str, Any], *keys: str) -> Any:
        """First non-empty value among the given (already-lowercased) keys."""
        for k in keys:
            v = row.get(k)
            if v is not None and str(v).strip() != "":
                return v
        return None


    def normalize_rows(rows: List[Dict[str, Any]]) -> Tuple[List[Dict[str, Any]],
NormalizeReport]:
        """Pure core: lowercase-keyed annotator rows -> clean lean rows + report."""
        rep = NormalizeReport(rows_in=len(rows))
        kept: List[Dict[str, Any]] = []

        for row in rows:
            orig_id = _get(row, "rally_number", "rally")
            s_parsed = _to_seconds(_get(row, "start_time", "start_mmss", "start"))
            e_parsed = _to_seconds(_get(row, "end_time", "end_mmss", "end"))
            start = s_parsed[0] if s_parsed else None
            end = e_parsed[0] if e_parsed else None

            if start is None:
                rep.dropped.append({"orig_id": orig_id, "reason": "missing/invalid
start_time",
                                    "start": _get(row, "start_time", "start_mmss"), "end":
_get(row, "end_time", "end_mmss")})
                continue
            if start < 0:
                # BadmintonRally's pydantic validator rejects start<0 -> would abort the
                # whole import; drop it here like any other unusable row.
                rep.dropped.append({"orig_id": orig_id, "reason": "negative start_time",
                                    "start": start, "end": end})
                continue
            if end is None:
                rep.dropped.append({"orig_id": orig_id, "reason": "missing/invalid
end_time",
                                    "start": start, "end": _get(row, "end_time",
"end_mmss")})
                continue
            if end <= start:
                rep.dropped.append({"orig_id": orig_id, "reason": "end_time <= start_time",
                                    "start": start, "end": end})
                continue
            if (s_parsed and s_parsed[1]):
                rep.mmss_converted += 1
            if (e_parsed and e_parsed[1]):
                rep.mmss_converted += 1

            reason, reason_note = _primary_reason(_get(row, "ending_reason"))
            note_in = _get(row, "notes", "note", "reviewer_comment")
            notes = "; ".join([x for x in [reason_note, (str(note_in).strip() if note_in
else "")] if x])
            kept.append({
                "orig_id": orig_id, "start": float(start), "end": float(end),
                "ending_reason": reason, "sport": _get(row, "sport"),
                "shots_count": _get(row, "shots_count", "shots"), "notes": notes,
            })

        kept.sort(key=lambda r: r["start"])

        # Renumber 1..N by start order; detect whether ids were fractional / duplicate /
        # not already a clean 1..N sequence (any of which means import-local would collide).
```

```
        orig_seq = [str(r["orig_id"]) if r["orig_id"] is not None else "" for r in kept]
        new_seq = [str(i) for i in range(1, len(kept) + 1)]
        rep.renumbered = orig_seq != new_seq
        seen: Dict[str, int] = {}
        for oid in orig_seq:
            seen[oid] = seen.get(oid, 0) + 1
        rep.duplicate_ids = sorted({oid for oid, c in seen.items() if c > 1 and oid})
        rep.fractional_ids = sorted({oid for oid in orig_seq if "." in oid})

        out: List[Dict[str, Any]] = []
        for i, r in enumerate(kept, start=1):
            out.append({
                "rally_number": i,
                "start_time": f"{r['start']:.3f}",
                "end_time": f"{r['end']:.3f}",
                "ending_reason": r["ending_reason"] or "",
                "sport": r["sport"] or "",
                "shots_count": "" if r["shots_count"] is None else
str(r["shots_count"]).strip(),
                "original_rally_number": "" if r["orig_id"] is None else str(r["orig_id"]),
                "notes": r["notes"],
            })
        rep.rows_out = len(out)
        return out, rep


    def normalize_file(in_path: str, out_path: str) -> NormalizeReport:
        with open(in_path, "r", newline="", encoding="utf-8-sig") as f:
            reader = csv.DictReader(f)
            reader.fieldnames = [(n or "").lower().strip() for n in (reader.fieldnames or
[])]
            rows = [{(k or "").lower().strip(): v for k, v in r.items()} for r in reader]
        out, rep = normalize_rows(rows)
        os.makedirs(os.path.dirname(os.path.abspath(out_path)) or ".", exist_ok=True)
        with open(out_path, "w", newline="", encoding="utf-8") as f:
            w = csv.DictWriter(f, fieldnames=OUT_FIELDS)
            w.writeheader()
            w.writerows(out)
        return rep


    def main() -> None:
        ap = argparse.ArgumentParser(description="Normalize a rally-annotator CSV for safe
import-local.")
        ap.add_argument("--in", dest="in_path", required=True, help="rally-annotator
.rallies.csv")
        ap.add_argument("--out", dest="out_path", required=True, help="clean lean CSV to
write")
        args = ap.parse_args()
        rep = normalize_file(args.in_path, args.out_path)
        print("=== normalize-annotations ===")
        print(rep.summary())
        print(f"wrote -> {args.out_path}")


    if __name__ == "__main__":
        main()

```

13. user (2026-06-10T05:33:57.807Z)

```
      """Shared parsing helpers used by the CLI importer and dataset parsers.

      Previously these were duplicated as private ``_safe_int`` / ``_safe_float`` /
      ``_parse_time_string`` in both ``src/cli/curate.py`` and the CoachAI parser.
```

```
5       """
6       from typing import Any, Optional
7
8
9       def safe_int(value: Any, default: Optional[int] = 0) -> Optional[int]:
10          """Parse an int from a possibly-empty/float/None value; return default on
failure."""
11          if value is None:
12              return default
13          s = str(value).strip()
14          if not s:
15              return default
16          try:
17              return int(float(s))
18          except (TypeError, ValueError):
19              return default
20
21
22      def safe_float_required(value: Any) -> float:
23          """Parse a float; raise ValueError if missing/empty (caller treats it as
required)."""
24          if value is None:
25              raise ValueError("missing required numeric value")
26          s = str(value).strip()
27          if not s:
28              raise ValueError("missing required numeric value")
29          return float(s)
30
31
32      def parse_time_string(time_str: Any) -> float:
33          """Convert 'HH:MM:SS' / 'MM:SS' / float-strings to float seconds; 0.0 if
blank/invalid."""
34          if not time_str:
35              return 0.0
36          s = str(time_str).strip()
37          if ":" in s:
38              parts = s.split(":")
39              try:
40                  if len(parts) == 3:
41                      return float(parts[0]) * 3600 + float(parts[1]) * 60 + float(parts[2])
42                  if len(parts) == 2:
43                      return float(parts[0]) * 60 + float(parts[1])
44              except ValueError:
45                  return 0.0
46          try:
47              return float(s)
48          except ValueError:
49              return 0.0
50
```

14. user (2026-06-10T05:33:58.213Z)

```
1       from typing import List, Tuple, Union
2       from src.core.config import load_config
3       from src.core.schemas.base import VideoMetadata, LicenseType
4       from src.core.schemas.badminton import BadmintonRally
5       from src.core.schemas.tennis import TennisPoint
6       from src.core.schemas.cricket import CricketDelivery
7       from src.core.schemas.pickleball import PickleballRally
8       from src.core.schemas.tabletennis import TableTennisRally
9
10      class DatasetValidator:
11          def __init__(self, config_path: str = "config/config.yaml"):
12              self.config_path = config_path
13              self.commercial_licenses = self._load_licenses_whitelist()
```

```

        def _load_licenses_whitelist(self) -> List[str]:
            # load_config returns {} on a missing/unreadable file, so the default list
            # below is the single fallback (keep in sync with config.yaml).
            config = load_config(self.config_path)
            return config.get(
                'commercial_licenses',
                ["MIT", "Apache-2.0", "BSD", "CC0", "CC-BY", "Public-Domain",
"Proprietary"],
            )

        def check_license_commercial(self, license_type: Union[LicenseType, str]) -> bool:
            """Determines if a license type is safe for commercial use based on the
configuration whitelist."""
            val = license_type.value if isinstance(license_type, LicenseType) else
license_type
            return val in self.commercial_licenses

        def validate_badminton_rallies(self, rallies: List[BadmintonRally]) -> Tuple[bool,
List[str]]:
            """
            Validates badminton rallies:
            - Sorted chronologically.
            - Start time < End time (handled by Pydantic, but double checked).
            - No temporal overlap between sequential rallies.
            """
            errors = []
            if not rallies:
                return True, errors

            # Sort by rally number to check progression
            sorted_rallies = sorted(rallies, key=lambda r: r.rally_number)

            for i in range(len(sorted_rallies)):
                r = sorted_rallies[i]
                if r.start_time >= r.end_time:
                    errors.append(f"Rally {r.rally_number}: start_time ({r.start_time}) >=
end_time ({r.end_time})")

                if i > 0:
                    prev = sorted_rallies[i - 1]
                    if r.start_time < prev.end_time:
                        errors.append(
                            f"Rally overlap detected between Rally {prev.rally_number} "
                            f"(ends at {prev.end_time}s) and Rally {r.rally_number} "
                            f"(starts at {r.start_time}s)"
                        )

            return len(errors) == 0, errors

        def validate_tennis_points(self, points: List[TennisPoint]) -> Tuple[bool,
List[str]]:
            errors = []
            if not points:
                return True, errors

            sorted_pts = sorted(points, key=lambda p: p.point_number)
            for i in range(len(sorted_pts)):
                pt = sorted_pts[i]
                if pt.start_time >= pt.end_time:
                    errors.append(f"Point {pt.point_number}: start ({pt.start_time}) >= end
({pt.end_time})")

                if i > 0:
                    prev = sorted_pts[i - 1]
```

```
                    if pt.start_time < prev.end_time:
                        errors.append(
                            f"Point overlap detected: Point {prev.point_number} (ends
{prev.end_time}s) "
                            f"and Point {pt.point_number} (starts {pt.start_time}s)"
                        )
            return len(errors) == 0, errors

        def validate_cricket_deliveries(self, deliveries: List[CricketDelivery]) ->
Tuple[bool, List[str]]:
            errors = []
            if not deliveries:
                return True, errors

            # Sort by over, then ball within over
            sorted_delivs = sorted(deliveries, key=lambda d: (d.over, d.ball))
            for i in range(len(sorted_delivs)):
                d = sorted_delivs[i]
                if d.start_time >= d.end_time:
                    errors.append(f"Over {d.over} Ball {d.ball}: start ({d.start_time}) >=
end ({d.end_time})")

                if i > 0:
                    prev = sorted_delivs[i - 1]
                    if d.start_time < prev.end_time:
                        errors.append(
                            f"Delivery overlap: Over {prev.over} Ball {prev.ball} (ends
{prev.end_time}s) "
                            f"and Over {d.over} Ball {d.ball} (starts {d.start_time}s)"
                        )
            return len(errors) == 0, errors

        def validate_pickleball_rallies(self, rallies: List[PickleballRally]) -> Tuple[bool,
List[str]]:
            """Validates pickleball rallies: positive durations and no temporal overlap
            between chronologically-ordered rallies (same contract as badminton)."""
            errors = []
            if not rallies:
                return True, errors

            sorted_rallies = sorted(rallies, key=lambda r: r.rally_number)
            for i in range(len(sorted_rallies)):
                r = sorted_rallies[i]
                if r.start_time >= r.end_time:
                    errors.append(f"Rally {r.rally_number}: start_time ({r.start_time}) >=
end_time ({r.end_time})")

                if i > 0:
                    prev = sorted_rallies[i - 1]
                    if r.start_time < prev.end_time:
                        errors.append(
                            f"Rally overlap detected between Rally {prev.rally_number} "
                            f"(ends at {prev.end_time}s) and Rally {r.rally_number} "
                            f"(starts at {r.start_time}s)"
                        )
            return len(errors) == 0, errors

        def validate_tabletennis_rallies(self, rallies: List[TableTennisRally]) ->
Tuple[bool, List[str]]:
            """Validates table-tennis rallies: positive durations and no temporal overlap
            between chronologically-ordered rallies (same contract as badminton)."""
            errors = []
            if not rallies:
                return True, errors

```

```
            sorted_rallies = sorted(rallies, key=lambda r: r.rally_number)
            for i in range(len(sorted_rallies)):
                r = sorted_rallies[i]
                if r.start_time >= r.end_time:
                    errors.append(f"Rally {r.rally_number}: start_time ({r.start_time}) >=
end_time ({r.end_time})")

                if i > 0:
                    prev = sorted_rallies[i - 1]
                    if r.start_time < prev.end_time:
                        errors.append(
                            f"Rally overlap detected between Rally {prev.rally_number} "
                            f"(ends at {prev.end_time}s) and Rally {r.rally_number} "
                            f"(starts at {r.start_time}s)"
                        )
            return len(errors) == 0, errors

```

15. user (2026-06-10T05:34:03.627Z)

```
from pydantic import BaseModel, Field
from typing import Optional
from enum import Enum

class SportType(str, Enum):
    BADMINTON = "badminton"
    TENNIS = "tennis"
    TABLE_TENNIS = "tabletennis"
    PICKLEBALL = "pickleball"
    CRICKET = "cricket"

class LicenseType(str, Enum):
    MIT = "MIT"
    APACHE_2 = "Apache-2.0"
    BSD = "BSD"
    CC0 = "CC0"
    CC_BY = "CC-BY"
    CC_BY_NC = "CC-BY-NC"
    CC_BY_NC_SA = "CC-BY-NC-SA"
    PUBLIC_DOMAIN = "Public-Domain"
    PROPRIETARY = "Proprietary"
    UNKNOWN = "Unknown"

class CurationStatus(str, Enum):
    STAGING = "staging"
    CURATED = "curated"
    REJECTED = "rejected"

class VideoMetadata(BaseModel):
    video_id: str = Field(..., description="Unique alphanumeric identifier for the
video")
    sport: SportType
    video_url: Optional[str] = Field(None, description="Remote URL of the video (if
available)")
    local_path: Optional[str] = Field(None, description="Local path to video file (if
downloaded or custom)")
    license_type: LicenseType
    license_url: Optional[str] = None
    is_commercial: bool = Field(False, description="Flag indicating if the license
allows commercial use")
    status: CurationStatus = CurationStatus.STAGING
    match_name: str = Field(..., description="Name of the match/event")
    fps: Optional[float] = Field(None, description="Frames per second")
    resolution: Optional[str] = Field(None, description="Resolution string (e.g. 1080p,
720p)")
```

```
41
```

16. user (2026-06-10T05:34:04.585Z)

```
1       from pydantic import BaseModel, Field, model_validator
2       from typing import Optional
3
4       class BadmintonRally(BaseModel):
5           video_id: str
6           rally_number: int = Field(..., description="Chronological rally index in the match")
7           start_time: float = Field(..., description="Start of rally in seconds")
8           end_time: float = Field(..., description="End of rally in seconds")
9           score_before: Optional[str] = Field(None, description="Score before the rally (e.g.
'0-0')")
10          score_after: Optional[str] = Field(None, description="Score after the rally (e.g.
'1-0')")
11          shots_count: Optional[int] = Field(None, description="Number of strokes in the
rally")
12          ending_reason: Optional[str] = Field(None, description="Why the rally ended:
'winner' | 'forced_error' | 'unforced_error' | 'service_fault' | 'let' | 'other'")
13          last_shot_outcome: Optional[str] = Field(None, description="Where the last shot
landed: 'in' | 'net' | 'out' | 'n_a' (n_a for service_fault/let)")
14
15          @model_validator(mode='after')
16          def validate_timestamps(self) -> 'BadmintonRally':
17              if self.start_time < 0:
18                  raise ValueError("start_time must be non-negative")
19              if self.end_time <= self.start_time:
20                  raise ValueError("end_time must be strictly greater than start_time")
21              return self
22
```

17. user (2026-06-10T05:34:04.664Z)

```
1       """Central configuration loader.
2
3       A single place to read ``config/config.yaml`` so modules don't each re-open the
4       file with slightly different fallbacks. Returns ``{}`` when the file is missing or
5       unreadable (callers apply their own defaults), matching the prior per-module
6       behaviour but without the duplicated boilerplate and broad excepts.
7       """
8       import os
9       from typing import Any, Dict
10
11      import yaml
12
13      DEFAULT_CONFIG_PATH = "config/config.yaml"
14      DEFAULT_DB_PATH = "data/sports_metadata.db"
15
16
17      def load_config(config_path: str = DEFAULT_CONFIG_PATH) -> Dict[str, Any]:
18          """Load a YAML config into a dict; return {} if missing/unreadable/empty."""
19          if not os.path.exists(config_path):
20              return {}
21          try:
22              with open(config_path, "r", encoding="utf-8") as f:
23                  return yaml.safe_load(f) or {}
24          except (OSError, yaml.YAMLError):
25              return {}
26
27
28      def get_database_path(config: Dict[str, Any]) -> str:
29          """Resolve the SQLite database path from a loaded config, with a default."""
30          return config.get("database_path", DEFAULT_DB_PATH)
31
```

18. user (2026-06-10T05:34:10.916Z)

```
1       """Adapt collector data into an obsreport envelope.
2
3       Reuses the existing rally_cvat report primitives (``Issue``, records, severities)
4       rather than re-deriving anything ? this module only *maps* them onto the shared
5       report shape. Imported only after ``src.reporting`` confirms obsreport is present,
6       so it may import obsreport freely.
7       """
8       from __future__ import annotations
9
10      from pathlib import Path
11      from typing import Any, List, Optional
12
13      import obsreport
14
15      _SPEC_PATH = Path(__file__).resolve().parent / "spec.yaml"
16
17      # Delivery issue severity -> report flag severity.
18      _SEVERITY = {"blocker": "error", "high": "warn", "medium": "warn", "low": "info"}
19      # Every issue code is documented in this FAQ table.
20      _ISSUE_FAQ = "docs/INGESTION_PIPELINE.md#what-does-validation-check"
21
22
23      def load_spec() -> "obsreport.ReportSpec":
24          return obsreport.load_spec(_SPEC_PATH)
25
26
27      # Heuristic checks that flag a region to REVIEW ? not hard failures. We frame these
28      # gently for the vendor so a "possible" finding isn't read as a certainty/blocker.
29      _REVIEW_ONLY = {"possible_missed_rally", "shot_density"}
30
31
32      def issue_to_flag(issue: Any, record: Any = None, fps: Any = None) -> "obsreport.Flag":
33          """Map a rally_cvat.Issue onto a report Flag. Actionable issues (blocker/high/
34          medium) also carry a customer_message so the delivery summary shows the vendor
35          exactly what to fix, in plain language. When the rally's source record + fps are
36          known, frame-level evidence is attached so a CVAT (frame-based) reviewer can act
37          without converting seconds to frames by hand."""
38          sev = _SEVERITY.get(issue.severity, "info")
39          customer = issue.message if sev in ("error", "warn") else None
40          if customer and issue.code in _REVIEW_ONLY:
41              customer += (" (Review only ? a heuristic, not a blocker: check the video and "
42                           "act only if confirmed.)")
43          evidence: dict = {}
44          if issue.rally_number is not None:
45              evidence["rally_number"] = issue.rally_number
46          if record is not None and getattr(record, "frame_min", -1) is not None and
getattr(record, "frame_min", -1) >= 0:
47              evidence["rally_frames"] = f"{record.frame_min}-{record.frame_max}"
48          if fps:
49              evidence["fps"] = fps
50          return obsreport.Flag(
51              code=issue.code,
52              severity=sev,
53              message=issue.message,
54              stage="validation",
55              faq_ref=_ISSUE_FAQ,
56              customer_message=customer,
57              evidence=evidence,
58          )
59
60
61      def _recorder(source: str, video_path: Optional[str], spec) -> "obsreport.RunRecorder":
62          return obsreport.RunRecorder(
```

```
            "sports-data-collector",
            video_id=source,
            video_id_kind="human_id",
            video_path=video_path,
            spec=spec if spec is not None else load_spec(),
        )


    def build_delivery_recorder(
        *,
        records: List[Any],
        issues: List[Any],
        fps: Optional[float],
        source: str,
        video_path: Optional[str] = None,
        video_duration_s: Optional[float] = None,
        source_format: str = "cvat",
        step: Optional[int] = None,
        spec=None,
    ) -> "obsreport.RunRecorder":
        """Envelope for `validate-delivery` / `convert-cvat`: parse + validation stages,
        one flag per delivery issue, a rally summary."""
        rec = _recorder(source, video_path, spec)
        if fps:
            rec.set_video_meta(fps=fps, fps_source="declared")
        if video_duration_s:
            rec.set_video_meta(duration_s=round(video_duration_s, 1))

        with rec.stage("parse") as st:
            st.add_metric("rallies_parsed", len(records))
            st.details.update({"source_format": source_format, "fps": fps, "sampling_step":
step})

        blockers = [i for i in issues if i.severity == "blocker"]
        with rec.stage("validation") as st:
            for sev in ("blocker", "high", "medium", "low"):
                st.add_metric(f"issues_{sev}", sum(1 for i in issues if i.severity == sev))
            st.add_metric("n_blockers", len(blockers))
            st.details["gate"] = "BLOCKED" if blockers else "PASS"
            st.status = "error" if blockers else ("warn" if issues else "ok")

        records_by_rally = {getattr(r, "rally_number", None): r for r in records}
        for issue in issues:
            record = records_by_rally.get(issue.rally_number)
            rec.envelope.flags.append(issue_to_flag(issue, record=record, fps=fps))

        _rally_summary(rec, records, video_duration_s)
        rec.finalize()
        return rec


    def build_import_recorder(
        *,
        video_meta_obj: Any,
        annotations: List[Any],
        is_valid: bool,
        errors: List[str],
        count: int,
        sport: str,
        source_format: str,
        spec=None,
    ) -> "obsreport.RunRecorder":
        """Envelope for `import-local`: parse + validation + registration stages."""
        rec = _recorder(video_meta_obj.video_id, video_meta_obj.local_path, spec)
        fps = getattr(video_meta_obj, "fps", None)
```

```
        rec.set_video_meta(
            fps=fps,
            fps_source="declared" if fps else "unknown",
            resolution=getattr(video_meta_obj, "resolution", None),
        )

        with rec.stage("parse") as st:
            st.add_metric("records_parsed", len(annotations))
            st.details["source_format"] = source_format

        with rec.stage("validation") as st:
            st.status = "ok" if is_valid else "error"
            st.add_metric("valid", is_valid)
            for e in errors or []:
                st.messages.append(str(e))

        with rec.stage("registration") as st:
            st.add_metric("segments_registered", count)
            st.details.update({
                "sport": sport,
                "license": getattr(getattr(video_meta_obj, "license_type", None), "value",
                                   str(getattr(video_meta_obj, "license_type", ""))),
                "is_commercial": bool(getattr(video_meta_obj, "is_commercial", False)),
                "status": getattr(getattr(video_meta_obj, "status", None), "value",
                                  str(getattr(video_meta_obj, "status", ""))),
            })

        _rally_summary(rec, annotations, None)
        rec.finalize()
        return rec


    def _rally_summary(rec: "obsreport.RunRecorder", records: List[Any], video_duration_s:
Optional[float]) -> None:
        total = 0.0
        for r in records:
            try:
                total += float(r.end_time) - float(r.start_time)
            except (AttributeError, TypeError, ValueError):
                pass
        coverage = round(100.0 * total / video_duration_s, 1) if video_duration_s else None
        notes = [] if records else ["No rallies were found in this delivery."]
        rec.set_highlights(segment_count=len(records), total_highlight_s=round(total, 1),
                           coverage_pct=coverage, notes=notes)

```

19. user (2026-06-10T05:34:11.517Z)

```
      """Per-video observability reports for the collector.

      `obsreport` is a SOFT dependency: absent -> these emitters no-op and the CLI runs
      unchanged. Every emitter is safe to call from normal control flow and NEVER raises
      (emitting a report must not break ingestion). Returns the written paths, or None.
      """
      from __future__ import annotations

      import os
      from typing import Any, Dict, List, Optional

      try:  # soft dependency
          import obsreport  # noqa: F401
          from . import adapter

          OBSREPORT_AVAILABLE = True
      except Exception:  # pragma: no cover - only when obsreport is absent
```

```
18          OBSREPORT_AVAILABLE = False
19
20
21      def emit_delivery_report(
22          *,
23          out_dir: str,
24          source: str,
25          records: List[Any],
26          issues: List[Any],
27          fps: Optional[float],
28          video_path: Optional[str] = None,
29          video_duration_s: Optional[float] = None,
30          source_format: str = "cvat",
31          step: Optional[int] = None,
32      ) -> Optional[Dict[str, str]]:
33          """Write the delivery observability report into out_dir (alongside the existing
34          <id>_report.* artifacts). Used by validate-delivery / convert-cvat. Never raises."""
35          if not OBSREPORT_AVAILABLE:
36              return None
37          try:
38              rec = adapter.build_delivery_recorder(
39                  records=records, issues=issues, fps=fps, source=source,
video_path=video_path,
40                  video_duration_s=video_duration_s, source_format=source_format, step=step,
41              )
42              res = rec.write(output_dir=out_dir, stem=source)
43              return res.paths or None
44          except Exception:  # pragma: no cover - reporting must never break the command
45              return None
46
47
48      def emit_import_report(
49          *,
50          video_meta_obj: Any,
51          annotations: List[Any],
52          is_valid: bool,
53          errors: List[str],
54          count: int,
55          sport: str,
56          source_format: str,
57          fallback_dir: Optional[str] = None,
58      ) -> Optional[Dict[str, str]]:
59          """Write the import observability report next to the imported video. If the video
60          file doesn't exist on disk (registration-only imports), fall back to
61          ``fallback_dir`` (e.g. the annotations directory) rather than the cwd. Never
raises."""
62          if not OBSREPORT_AVAILABLE:
63              return None
64          try:
65              rec = adapter.build_import_recorder(
66                  video_meta_obj=video_meta_obj, annotations=annotations, is_valid=is_valid,
67                  errors=errors, count=count, sport=sport, source_format=source_format,
68              )
69              video_path = getattr(video_meta_obj, "local_path", None)
70              if video_path and os.path.exists(video_path):
71                  res = rec.write()  # next to the video
72              else:
73                  res = rec.write(output_dir=fallback_dir or ".",
74                                  stem=getattr(video_meta_obj, "video_id", None))
75              return res.paths or None
76          except Exception:  # pragma: no cover
77              return None
78
```

20. user (2026-06-10T05:34:20.663Z)

```
1       import argparse
2       import sys
3       from src.core.config import load_config, get_database_path
4       from src.core.db import SportsDatabase
5       from src.parsers.badminton.coachai import CoachAIBadmintonParser
6
7       def main():
8           parser = argparse.ArgumentParser(description="Ingest external datasets into the
curation staging area")
9           parser.add_argument("--sport", required=True, choices=["badminton"], help="Sport
type")
10          parser.add_argument("--parser", required=True, choices=["coachai"], help="Parser
type to use")
11          parser.add_argument("--file-path", required=True, help="Path to raw dataset file")
12          parser.add_argument("--license", default="MIT", help="Dataset license type")
13          parser.add_argument("--video-url", default="https://youtube.com/watch?v=example",
help="URL to the video")
14          parser.add_argument("--match-name", help="Name of the match")
15          parser.add_argument("--video-id", help="Video unique identifier")
16
17          args = parser.parse_args()
18
19          # Load db path from the central config (falls back to the default).
20          # NOTE: this importer is intentionally badminton/CoachAI-only for now; other
21          # sports are populated via `curate import-local` or vendor golden sets.
22          db = SportsDatabase(get_database_path(load_config()))
23
24          if args.sport == "badminton" and args.parser == "coachai":
25              match_parser = CoachAIBadmintonParser()
26              try:
27                  video_meta, rallies = match_parser.parse(
28                      args.file_path,
29                      video_id=args.video_id,
30                      video_url=args.video_url,
31                      license=args.license,
32                      match_name=args.match_name
33                  )
34
35                  # Insert into database (by default parsed entries are marked as staging)
36                  db.insert_video(video_meta)
37                  count = db.insert_badminton_rallies(rallies)
38
39                  print(f"Successfully ingested match '{video_meta.match_name}' into
staging:")
40                  print(f"  - Video ID: {video_meta.video_id}")
41                  print(f"  - Total Rallies: {count}")
42                  print(f"  - License: {video_meta.license_type.value}")
43                  print(f"  - Commercial safe: {video_meta.is_commercial}")
44              except Exception as e:
45                  print(f"Error parsing file: {e}")
46                  sys.exit(1)
47
48      if __name__ == "__main__":
49          main()
50
```

21. user (2026-06-10T05:34:21.072Z)

```
1       import csv
2       import os
3       from typing import Tuple, List, Dict, Any, Optional
4       from src.parsers.base_parser import BaseParser
5       from src.core.parsing import safe_int as _safe_int, parse_time_string
6       from src.core.schemas.base import VideoMetadata, SportType, LicenseType, CurationStatus
7       from src.core.schemas.badminton import BadmintonRally
```

```
8       from src.core.validator import DatasetValidator
9
10
11      class CoachAIBadmintonParser(BaseParser):
12          def __init__(
13              self,
14              start_buffer: float = 1.0,
15              end_buffer: float = 2.0,
16              validator: Optional[DatasetValidator] = None,
17          ):
18              """
19              Args:
20                  start_buffer: Seconds to subtract from first stroke (serve) time.
21                  end_buffer: Seconds to add to last stroke time.
22                  validator: Used to decide commercial-license safety from the config
23                      whitelist. Defaults to a DatasetValidator with the default config so
24                      license classification stays consistent across the codebase.
25              """
26              self.start_buffer = start_buffer
27              self.end_buffer = end_buffer
28              self.validator = validator or DatasetValidator()
29
30          def parse(self, filepath: str, **kwargs) -> Tuple[VideoMetadata,
List[BadmintonRally]]:
31              """
32              Parses a CoachAI/ShuttleSet CSV format.
33
34              Expected CSV Columns (any subset or mapped names):
35                  - set: Set number (1, 2, 3)
36                  - rally: Rally number
37                  - ball_round: Shot number within the rally (1, 2, 3...)
38                  - time: Hitting time of the shot (in seconds)
39                  - type: Stroke type (e.g., smash, lob, net...)
40                  - score_a / score_b: Current scores
41              """
42              if not os.path.exists(filepath):
43                  raise FileNotFoundError(f"File not found: {filepath}")
44
45              # Extract video details from kwargs or name
46              match_name = kwargs.get("match_name") or
os.path.basename(filepath).replace(".csv", "")
47              video_id = kwargs.get("video_id") or match_name.lower().replace(" ", "_")
48              video_url = kwargs.get("video_url") or "https://youtube.com/watch?v=example"
49              license_str = kwargs.get("license") or "MIT"
50
51              # Check if license matches commercial whitelist
52              try:
53                  license_type = LicenseType(license_str)
54              except ValueError:
55                  license_type = LicenseType.UNKNOWN
56
57              # Group rows by set & rally
58              # Key: (set_num, rally_num) -> List of rows (dict)
59              rallies_data: Dict[Tuple[int, int], List[Dict[str, Any]]] = {}
60
61              with open(filepath, mode='r', encoding='utf-8') as f:
62                  reader = csv.DictReader(f)
63                  # Normalize column names to lowercase/strip
64                  reader.fieldnames = [name.lower().strip() for name in reader.fieldnames] if
reader.fieldnames else []
65
66                  for row in reader:
67                      # Resolve key columns (tolerate empty cells and float-strings)
68                      set_num = _safe_int(row.get('set'), 1)
69                      rally_num = _safe_int(row.get('rally'), 0)
```

```
                    ball_round = _safe_int(row.get('ball_round', row.get('ball_seq')), 1)

                    # Resolve time (safely parse float or HH:MM:SS strings)
                    raw_time = row.get('time', row.get('timestamp', row.get('sec', '0.0')))
                    time_val = parse_time_string(raw_time)

                    key = (set_num, rally_num)
                    if key not in rallies_data:
                        rallies_data[key] = []

                    rallies_data[key].append({
                        'ball_round': ball_round,
                        'time': time_val,
                        'type': row.get('type', ''),
                        'score_a': row.get('score_a', row.get('round_score_a', '0')),
                        'score_b': row.get('score_b', row.get('round_score_b', '0'))
                    })

            # First pass: build raw rally records (unbuffered stroke bounds) in chrono
order.
            raw_rallies: List[Dict[str, Any]] = []
            for key in sorted(rallies_data.keys()):
                rows_sorted = sorted(rallies_data[key], key=lambda x: x['ball_round'])
                if not rows_sorted:
                    continue
                first_stroke = rows_sorted[0]
                last_stroke = rows_sorted[-1]
                raw_rallies.append({
                    'first_time': first_stroke['time'],
                    'last_time': last_stroke['time'],
                    'ending_reason': last_stroke['type'] if last_stroke['type'] else None,
                    'shots_count': len(rows_sorted),
                    'score_before': f"{first_stroke['score_a']}-{first_stroke['score_b']}",
                    'score_after': f"{last_stroke['score_a']}-{last_stroke['score_b']}",
                })

            # Second pass: apply start/end buffers, then resolve any boundary where the
            # padding of two adjacent rallies would collide by meeting at the midpoint of
            # the gap between their nearest strokes. This keeps the buffers from ever
            # producing an artificial overlap (which the validator would otherwise reject).
            starts = [max(0.0, rr['first_time'] - self.start_buffer) for rr in raw_rallies]
            ends = [rr['last_time'] + self.end_buffer for rr in raw_rallies]
            for i in range(len(raw_rallies) - 1):
                if ends[i] > starts[i + 1]:
                    midpoint = (raw_rallies[i]['last_time'] + raw_rallies[i +
1]['first_time']) / 2
                    ends[i] = min(ends[i], midpoint)
                    starts[i + 1] = max(starts[i + 1], midpoint)

            rallies: List[BadmintonRally] = []
            for i, rr in enumerate(raw_rallies):
                start_time = starts[i]
                end_time = ends[i]
                # Guarantee a strictly positive duration even after clamping.
                if end_time <= start_time:
                    end_time = start_time + 0.001

                rallies.append(BadmintonRally(
                    video_id=video_id,
                    rally_number=i + 1,
                    start_time=start_time,
                    end_time=end_time,
                    score_before=rr['score_before'],
                    score_after=rr['score_after'],
                    shots_count=rr['shots_count'],
```

```
133                     ending_reason=rr['ending_reason']
134                 ))
135 
136             # Populate VideoMetadata
137             video_metadata = VideoMetadata(
138                 video_id=video_id,
139                 sport=SportType.BADMINTON,
140                 video_url=video_url,
141                 local_path=kwargs.get("local_path", None),
142                 license_type=license_type,
143                 license_url=kwargs.get("license_url", None),
144                 is_commercial=self.validator.check_license_commercial(license_type),
145                 status=CurationStatus.STAGING,
146                 match_name=match_name,
147                 fps=kwargs.get("fps", 30.0),
148                 resolution=kwargs.get("resolution", "1080p")
149             )
150             return video_metadata, rallies
151 
```

22. assistant (2026-06-10T05:34:29.808Z)

Now let me look at the tests and run the suite.

23. user (2026-06-10T05:34:30.837Z)

```
tests\test_downloader.py
tests\test_local_import.py
tests\test_schemas.py
tests\test_validator.py
tests\test_cli.py
tests\test_config.py
tests\test_parser.py
tests\test_parsing.py
tests\test_db.py
tests\test_cvat_parser.py
tests\test_reporting.py
tests\test_reporting_faq.py
tests\test_normalize_annotations.py
```

24. user (2026-06-10T05:34:31.725Z)

Sports Data Collector Configuration

Path to the SQLite database

```
database_path: "./data/sports_metadata.db"
```

Storage configuration for downloaded videos

```
video_storage:
  # Folder where downloaded or local videos will be stored
  local_dir: "./data/videos"
  # Maximum allowed size for automatic downloads (in Megabytes)
  max_download_size_mb: 12000
  # Whether to attempt video downloads automatically during ingestion
  auto_download: true
  # Optional cap on download resolution height (e.g. 1080 to avoid pulling 4K).
  # Leave unset/null to always fetch the best resolution available.
  max_height: null
```

Whitelist of licenses considered safe for commercial use

```
commercial_licenses:
```

```
  - "MIT"
  - "Apache-2.0"
  - "BSD"
  - "CC0"
  - "CC-BY"
  - "Public-Domain"
  - "Proprietary"  # Used for custom user-added local videos
```

Ingestion limits

```
ingestion_limits:
  max_total_download_size_mb: 60000
  max_total_video_count: 10
```

Whitelist of all sports supported in the system

```
supported_sports:
  - "badminton"
  - "tennis"
  - "tabletennis"
  - "pickleball"
  - "cricket"
[build-system]
requires = ["setuptools>=61.0.0"]
build-backend = "setuptools.build_meta"

[project]
name = "sports-data-collector"
version = "0.1.0"
description = "A collector and curator for sports video datasets with sport-specific schemas and
license checks."
readme = "README.md"
requires-python = ">=3.9"
dependencies = [
    "pydantic>=2.0.0",
    "pyyaml>=6.0",
    "tabulate>=0.9.0"
]

[project.optional-dependencies]
```

Video acquisition (`fetch` / crawler). Also requires the external binaries ffprobe (FFmpeg) and, for YouTube high-res, a JS runtime such as deno ? neither is pip-installable; see README "External requirements".

```
fetch = [
    "yt-dlp>=2024.1.0"
]
dev = [
    "pytest>=7.0.0",
    "pytest-cov>=4.0.0"
]
```

Per-video observability reports (SOFT dependency: ingestion degrades to a no-op if absent). Shared, permissively-licensed report toolkit. Pin a tag once the private repo is published, e.g.:

obsreport @ git+ssh://git@github.com/avidullu/sports-obsreport.git@v0.1.3

For local development install it editable from a sibling checkout:

pip install -e ../sports-obsreport --no-build-isolation

```
report = [
    "obsreport>=0.1.3",
]
```

25. user (2026-06-10T05:34:42.152Z)

```
.......................................................................... [ 58%]
..............................................                             [100%]
124 passed in 2.56s
```

26. user (2026-06-10T05:34:54.797Z)

```
.......................................................................... [ 58%]
..............................................                             [100%]
============================== tests coverage ==============================
_____________ coverage: platform win32, python 3.13.13-final-0 _____________

Name                                Stmts   Miss  Cover   Missing
-----------------------------------------------------------------
src\cli\curate.py                     567    180    68%   35, 166, 185, 209, 263, 287-288,
329-334, 337-339, 361, 371-373, 377, 380-381, 386-387, 391-392, 398-401, 409, 425, 457, 469-471,
479-480, 484-485, 533, 535, 549-582, 595-656, 687-688, 690-691, 702-709, 727-738, 761, 773-774,
783-814, 817-818, 825-827, 835, 841, 948, 950, 960
src\cli\normalize_annotations.py      133     35    74%   56-74, 83, 88, 91, 93-94, 103, 106,
132-134, 207-214, 218
src\core\config.py                     15      0   100%
src\core\db.py                        191      7    96%   164, 212, 241, 268, 297, 325, 537
src\core\downloader.py                 82      9    89%   106-109, 127-128, 138-139, 180
src\core\parsing.py                    35      2    94%   44-45
src\core\schemas\badminton.py          19      0   100%
src\core\schemas\base.py               36      0   100%
src\core\schemas\cricket.py            20      0   100%
src\core\schemas\pickleball.py         19      0   100%
src\core\schemas\tabletennis.py        19      0   100%
src\core\schemas\tennis.py             18      0   100%
src\core\validator.py                  88      5    94%   46, 68, 89, 111, 134
src\parsers\badminton\coachai.py       62      4    94%   43, 54-55, 93
src\parsers\base_parser.py              7      1    86%   15
src\parsers\cvat\__init__.py            0      0   100%
src\parsers\cvat\rally_cvat.py        251     37    85%   148, 162-163, 177-178, 189-190,
198-202, 208, 214, 295, 339-340, 377-396, 416-417, 420-421, 432, 441, 449, 510
src\reporting\__init__.py              24      2    92%   36, 63
src\reporting\adapter.py               76      3    96%   141, 164-165
-----------------------------------------------------------------
TOTAL                                1662    285    83%
124 passed in 3.28s
```

27. assistant (2026-06-10T05:35:02.373Z)

All 124 tests pass (83% coverage). Now let me cross-check the indexer side of the contract.

28. user (2026-06-10T05:35:03.515Z)

```
/c/Users/avidu/Projects/badminton-highlight-indexer/:
CLAUDE.md
README.md
TEST_RUN_SUMMARY.md
__pycache__
backend
config.json
docs
output
pytest.ini
requirements.txt
run_all_tests.py
run_eval.py
run_phase3_eval.py
run_process_and_stitch.py
```

```
run_regression.py
scratch
scripts
setup.py
tests

/c/Users/avidu/Projects/badminton-highlight-indexer/backend/eval/:
__init__.py
__pycache__
annotations.py
audio
batch.py
calibrate_local.py
calibrate_wasb.py
calibration.py
classifier.py
distill_local.py
eval_partial_labels.py
export_wasb_segments.py
gemini_refine.py
harness.py
metrics.py
rally_seq_proto.py
regression.py
training.py
```

29. user (2026-06-10T05:35:04.583Z)

```
Found 10 files
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\eval\rally_seq_proto.py
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\eval\eval_partial_labels.py
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\eval\batch.py
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\eval\export_wasb_segments.py
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\eval\distill_local.py
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\eval\calibrate_wasb.py
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\eval\audio\e1_probe.py
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\eval\calibrate_local.py
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\eval\gemini_refine.py
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\eval\annotations.py
```

30. user (2026-06-10T05:35:09.847Z)

```
1       """Batch evaluation across a collector export manifest.
2
3       Phase 3 of the collector->indexer bridge: walk the `manifest.json` emitted by
4       sports-data-collector's `curate export`, (optionally) process each video through
5       a segmenter so detections land in the indexer DB, then score every video's
6       detected rallies against its ground-truth CSV and aggregate the results.
7
8       This module holds the *pure* pieces (path resolution, stage selection, metric
9       aggregation) so they can be unit-tested without touching video files, the GPU,
10      or any API. The orchestration/side-effects live in `run_phase3_eval.py`.
11      """
12
13      import os
14      from typing import Dict, List, Optional
15
16
17      # Segmenters that only ever populate the `final` play_segments table (no raw
18      # candidate stage), so scoring `candidates` for them would be meaninglessly empty.
19      _FINAL_ONLY_SEGMENTERS = {"motion", "gemini"}
20
21
22      def resolve_video_path(local_path: Optional[str], videos_root: str) -> Optional[str]:
23          """Resolve a manifest `local_path` to an absolute path.
```

```
24
25          The collector stores paths like ``./data/videos\\shuttleset_1.mp4`` relative
26          to its own repo root and with Windows separators. Normalise separators, strip
27          a leading ``./``, and resolve relative paths against `videos_root` (the
28          collector root). Returns None for a missing/empty path.
29          """
30          if not local_path:
31              return None
32          p = local_path.replace("\\", "/")
33          if p.startswith("./"):
34              p = p[2:]
35          # Cross-platform: treat Windows drive-letter absolute paths (C:/...) as absolute
36          # even when running on Linux (os.path.isabs("C:/...") == False on Unix).
37          # This keeps collector manifest paths working in tests and mixed environments.
38          if (len(p) > 1 and p[1] == ":" and p[0].isalpha()) or os.path.isabs(p):
39              return os.path.normpath(p)
40          return os.path.normpath(os.path.join(videos_root, p))
41
42
43      def default_stages_for(segmenter: str, requested: str = "auto") -> List[str]:
44          """Pick which prediction stages to score.
45
46          ``auto`` scores both stages for two-stage detectors (yolo_hybrid/tracknet_*)
47          but only ``final`` for segmenters that don't emit candidates. An explicit
48          request ('candidates' | 'final' | 'both') overrides the heuristic.
49          """
50          if requested == "both":
51              return ["candidates", "final"]
52          if requested in ("candidates", "final"):
53              return [requested]
54          # auto
55          if segmenter in _FINAL_ONLY_SEGMENTERS:
56              return ["final"]
57          return ["candidates", "final"]
58
59
60      def _safe_div(num: float, den: float) -> float:
61          return num / den if den else 0.0
62
63
64      def aggregate(per_video: List[dict], stages: List[str]) -> Dict[str, dict]:
65          """Aggregate per-video stage scores into corpus-level metrics.
66
67          `per_video` is a list of dicts shaped like ``report_to_dict`` output (each has
68          ``stages[stage]["score"]`` with tp/fp/fn/mean_iou/...). For each stage we
69          compute **micro** precision/recall/F1 (pooling TP/FP/FN across all videos) and
70          a TP-weighted mean IoU, plus the macro mean of per-video F1.
71          """
72          out: Dict[str, dict] = {}
73          for stage in stages:
74              tp = fp = fn = 0
75              iou_weighted = 0.0
76              f1s: List[float] = []
77              n_videos = 0
78              for rep in per_video:
79                  sr = rep.get("stages", {}).get(stage)
80                  if not sr:
81                      continue
82                  s = sr["score"]
83                  tp += s["true_positives"]
84                  fp += s["false_positives"]
85                  fn += s["false_negatives"]
86                  iou_weighted += s["mean_iou"] * s["true_positives"]
87                  f1s.append(s["f1"])
88                  n_videos += 1
```

```
            precision = _safe_div(tp, tp + fp)
            recall = _safe_div(tp, tp + fn)
            out[stage] = {
                "videos": n_videos,
                "tp": tp, "fp": fp, "fn": fn,
                "micro_precision": precision,
                "micro_recall": recall,
                "micro_f1": _safe_div(2 * precision * recall, precision + recall),
                "macro_f1": _safe_div(sum(f1s), len(f1s)),
                "mean_iou": _safe_div(iou_weighted, tp),
            }
        return out


    def format_aggregate(agg: Dict[str, dict]) -> str:
        """Human-readable corpus-level summary table."""
        lines = [
            "=" * 78,
            " CORPUS AGGREGATE",
            "=" * 78,
            f" {'Stage':<12} {'Vids':>4} {'TP':>5} {'FP':>5} {'FN':>5} "
            f"{'mPrec':>6} {'mRec':>6} {'microF1':>8} {'macroF1':>8} {'mIoU':>6}",
            "-" * 78,
        ]
        for stage, a in agg.items():
            lines.append(
                f" {stage:<12} {a['videos']:>4} {a['tp']:>5} {a['fp']:>5} {a['fn']:>5} "
                f"{a['micro_precision']:>6.2f} {a['micro_recall']:>6.2f} "
                f"{a['micro_f1']:>8.2f} {a['macro_f1']:>8.2f} {a['mean_iou']:>6.2f}"
            )
        lines.append("=" * 78)
        return "\n".join(lines)

```

31. user (2026-06-10T05:35:14.969Z)

```
      """Phase 3 driver: batch process + evaluate a collector export against ground truth.

      Walks the `manifest.json` produced by sports-data-collector's `curate export`,
      processes each video through a segmenter (so detections land in the indexer DB),
      scores each against its ground-truth rally CSV, and prints per-video plus
      corpus-aggregate precision/recall/F1, mean IoU, and boundary error.

      Examples
      --------
      Free CPU baseline on a single video (validate the loop):
          python run_phase3_eval.py --manifest ../sports-data-
collector/data/eval_export/manifest.json \
              --segmenter motion --only shuttleset_1

      Full corpus with the two-stage detector (needs GEMINI_API_KEY; costs API calls):
          python run_phase3_eval.py --manifest .../manifest.json --segmenter yolo_hybrid \
              --json output/phase3_report.json

      Re-score only (videos already processed), no re-segmentation:
          python run_phase3_eval.py --manifest .../manifest.json --score-only
      """

      import os
      import sys
      import json
      import time
      import argparse
      import logging

```

```
    from dotenv import load_dotenv
    load_dotenv()  # make GEMINI_API_KEY (etc.) available to the AI-backed segmenters

    from backend.infrastructure.database import Database
    from backend.config import load_config
    from backend.utils.hashing import compute_video_id as get_file_md5  # canonical video_id
    # Importing the package registers all segmenters (motion/gemini/yolo_hybrid/...).
    from backend.pipeline.segmenters import segmenter_registry
    from backend.eval.annotations import load_annotations
    from backend.eval.harness import evaluate, format_report_text, report_to_dict
    from backend.eval import batch

    logger = logging.getLogger(__name__)


    def build_parser() -> argparse.ArgumentParser:
        p = argparse.ArgumentParser(
            description="Batch process + evaluate a collector export manifest (Phase 3).",
            formatter_class=argparse.RawDescriptionHelpFormatter,
        )
        p.add_argument("--manifest", required=True, help="Path to the collector's
manifest.json.")
        p.add_argument("--videos-root", default=None,
                       help="Root for resolving manifest relative local_paths "
                            "(default: the collector repo root, two levels above the
manifest).")
        p.add_argument("--segmenter", default="motion",
                       help="Segmenter to run (default: motion ? free/CPU). "
                            "yolo_hybrid/gemini need GEMINI_API_KEY and cost API calls.")
        p.add_argument("--stage", choices=("candidates", "final", "both", "auto"),
default="auto",
                       help="Which prediction stage(s) to score (default: auto by
segmenter).")
        p.add_argument("--detector", default=None, help="Restrict candidate scoring to this
detector.")
        p.add_argument("--iou", type=float, default=0.5, help="IoU match threshold (default:
0.5).")
        p.add_argument("--only", action="append", default=None,
                       help="Only this video_id (repeatable).")
        p.add_argument("--limit", type=int, default=None, help="Process at most N videos.")
        p.add_argument("--max-frames", type=int, default=None,
                       help="Cap segmenter frames (fast validation).")
        p.add_argument("--score-only", action="store_true",
                       help="Skip segmentation; only score detections already in the DB.")
        p.add_argument("--reprocess", action="store_true",
                       help="Re-run segmentation even if the video already has segments.")
        p.add_argument("--db", default=None, help="Indexer DB path (default:
output/<db_name>).")
        p.add_argument("--json", dest="json_out", default=None, help="Write a JSON report
here.")
        p.add_argument("--show-failures", action="store_true", help="List missed/spurious
rallies per video.")
        return p


    def main(argv=None) -> int:
        logging.basicConfig(level=logging.INFO, format="%(levelname)s %(message)s")
        args = build_parser().parse_args(argv)

        manifest_path = os.path.abspath(args.manifest)
        if not os.path.exists(manifest_path):
            print(f"ERROR: manifest not found: {manifest_path}", file=sys.stderr)
            return 2
        manifest_dir = os.path.dirname(manifest_path)
        videos_root = args.videos_root or os.path.dirname(os.path.dirname(manifest_dir))
```

```
85
86          with open(manifest_path) as f:
87              manifest = json.load(f)
88          entries = manifest.get("eval_sets", [])
89
90          cfg = load_config()  # typed AppConfig (same object the live pipeline feeds
segmenters)
91          db_path = args.db or os.path.join(cfg.output.output_dir, cfg.output.db_name)
92          os.makedirs(os.path.dirname(db_path) or ".", exist_ok=True)
93          db = Database(db_path)
94
95          stages = batch.default_stages_for(args.segmenter, args.stage)
96
97          # Resolve the segmenter class up front (unless purely scoring).
98          segmenter = None
99          if not args.score_only:
100             seg_cls = segmenter_registry.get(args.segmenter)
101             if not seg_cls:
102                 print(f"ERROR: segmenter '{args.segmenter}' not found. "
103                       f"Available: {list(segmenter_registry.list_available().keys())}",
file=sys.stderr)
104                 return 2
105             segmenter = seg_cls(db, cfg)
106
107         # Filter entries.
108         targets = []
109         for e in entries:
110             if args.only and e["video_id"] not in args.only:
111                 continue
112             targets.append(e)
113         if args.limit:
114             targets = targets[: args.limit]
115
116         per_video = []
117         skipped = []
118         print(f"Phase 3: {len(targets)} target(s) | segmenter={args.segmenter} |
stages={stages} | db={db_path}\n")
119
120         for e in targets:
121             vid = e["video_id"]
122             video_path = batch.resolve_video_path(e.get("local_path"), videos_root)
123             csv_path = os.path.join(manifest_dir, e["csv"])
124
125             if not video_path or not os.path.exists(video_path):
126                 skipped.append((vid, "no video file"))
127                 print(f"[skip] {vid}: no video file ({e.get('local_path')})")
128                 continue
129             if not os.path.exists(csv_path):
130                 skipped.append((vid, "no GT csv"))
131                 print(f"[skip] {vid}: GT csv missing ({csv_path})")
132                 continue
133
134             video_id = get_file_md5(video_path)
135
136             # Segment unless score-only or already present (and not --reprocess).
137             if not args.score_only:
138                 already = db.query_play_segments(video_id, {}) or
db.query_candidate_segments(video_id)
139                 if already and not args.reprocess:
140                     print(f"[have] {vid}: detections already in DB (use --reprocess to
redo)")
141                 else:
142                     print(f"[proc] {vid}: segmenting with {args.segmenter}...")
143                     db.add_video(video_id, video_path, "processed", [])
144                     t0 = time.time()
```

```
                    segs = segmenter.process_video(video_id, video_path,
max_frames=args.max_frames)
                    print(f"        -> {len(segs)} segments in {time.time() - t0:.0f}s")

            gt = load_annotations(csv_path)
            report = evaluate(db, video_id, gt, stages, iou_threshold=args.iou,
detector=args.detector)
            print(format_report_text(report, show_failures=args.show_failures))
            print()
            d = report_to_dict(report)
            d["collector_video_id"] = vid
            per_video.append(d)

        agg = batch.aggregate(per_video, stages)
        print(batch.format_aggregate(agg))
        if skipped:
            print(f"\nSkipped {len(skipped)}: " + ", ".join(f"{v} ({why})" for v, why in
skipped))

        if args.json_out:
            with open(args.json_out, "w") as f:
                json.dump({"segmenter": args.segmenter, "iou": args.iou, "stages": stages,
                           "aggregate": agg, "per_video": per_video,
                           "skipped": skipped}, f, indent=2)
            print(f"\nJSON report written to {args.json_out}")

        return 0


    if __name__ == "__main__":
        raise SystemExit(main())

```

32. user (2026-06-10T05:35:20.456Z)

```
"""Load hand-annotated ground-truth rally timestamps from CSV.

Annotation file format (one file per video):

    start,end
    00:01:12,00:01:39
    102.5,131.0

- A header row `start,end` is optional (auto-detected and skipped).
- Each timestamp is either decimal seconds (`102.5`) or colon-separated
  `MM:SS` / `HH:MM:SS` (`00:01:12`). Both forms may be mixed in one file.
- Blank lines and `#` comment lines are ignored.
"""

import csv
import logging
from typing import List, Tuple

logger = logging.getLogger(__name__)

Interval = Tuple[float, float]


def parse_timestamp(value: str) -> float:
    """Parse a timestamp string to float seconds.

    Accepts decimal seconds or colon-separated MM:SS / HH:MM:SS.
    Raises ValueError on anything unparseable so bad annotations fail loudly.
    """
    value = value.strip()
```

```
31          if not value:
32              raise ValueError("empty timestamp")
33          if ":" in value:
34              parts = value.split(":")
35              if len(parts) > 3:
36                  raise ValueError(f"too many ':' segments in timestamp '{value}'")
37              total = 0.0
38              for part in parts:                # most-significant first (H, M, S)
39                  total = total * 60 + float(part)
40              return total
41          return float(value)
42
43
44      def load_annotations(csv_path: str) -> List[Interval]:
45          """Load and validate ground-truth rally intervals from a CSV file.
46
47          Returns intervals sorted by start time. Raises ValueError on malformed rows
48          (bad timestamp, start >= end) so labeling mistakes surface immediately rather
49          than silently skewing scores.
50          """
51          intervals: List[Interval] = []
52          with open(csv_path, newline="") as f:
53              reader = csv.reader(f)
54              for lineno, row in enumerate(reader, start=1):
55                  if not row:
56                      continue
57                  first = row[0].strip()
58                  if not first or first.startswith("#"):
59                      continue
60                  # Skip an optional header row.
61                  if first.lower() in ("start", "start_time"):
62                      continue
63                  if len(row) < 2:
64                      raise ValueError(f"{csv_path}:{lineno}: expected 'start,end', got
{row!r}")
65                  try:
66                      start = parse_timestamp(row[0])
67                      end = parse_timestamp(row[1])
68                  except ValueError as e:
69                      raise ValueError(f"{csv_path}:{lineno}: {e}") from e
70                  if start >= end:
71                      raise ValueError(f"{csv_path}:{lineno}: start ({start}) >= end ({end})")
72                  intervals.append((start, end))
73
74          intervals.sort(key=lambda iv: iv[0])
75          logger.info("Loaded %d ground-truth rallies from %s", len(intervals), csv_path)
76          return intervals
77
78
79      def write_scaffold(csv_path: str) -> None:
80          """Write an empty annotation file with just the header, for hand-filling."""
81          with open(csv_path, "w", newline="") as f:
82              writer = csv.writer(f)
83              writer.writerow(["start", "end"])
84
```

33. user (2026-06-10T05:35:22.101Z)

```
29-Config = Tuple[float, float, float]   # (velocity_thresh, merge_gap, min_window_duration)
30-
31-BASELINE: Config = (0.02, 2.0, 2.0)   # the WasbConfig / trajectory_to_action_windows
defaults
32-
33-
34:def load_golden_gts(csv_path: str) -> List[Interval]:
```

```
35:    """Rally [start,end] seconds from the golden CSV (start_time/end_time cols)."""
36-    gts: List[Interval] = []
37-    with open(csv_path, newline="", encoding="utf-8") as f:
38-        reader = csv.DictReader(f)
39-        reader.fieldnames = [(n or "").strip().lower() for n in (reader.fieldnames or [])]
40-        for row in reader:
41-            row = {(k or "").strip().lower(): v for k, v in row.items()}
42-            try:
43:                s = float((row.get("start_time") or "").strip())
44:                e = float((row.get("end_time") or "").strip())
45-            except (TypeError, ValueError):
46-                continue
47-            if e > s:
48-                gts.append((s, e))
49-    return sorted(gts)
--
101-
102-
103-def main():
104-    ap = argparse.ArgumentParser()
105-    ap.add_argument("--trajectory", required=True, help="WASB trajectory CSV
(Frame,Visibility,X,Y)")
106:    ap.add_argument("--labels", required=True, help="golden labels CSV (start_time/end_time
cols)")
107-    ap.add_argument("--fps", type=float, default=59.94)
108-    ap.add_argument("--frame-width", type=float, default=1920.0)
109-    ap.add_argument("--iou", type=float, default=0.5)
110-    args = ap.parse_args()
111-    run(args.trajectory, args.labels, args.fps, args.frame_width, args.iou)
```

34. user (2026-06-10T05:36:10.687Z)

```
1       # Roora Annotation Brief ? Rally Golden Set
2
3       **Project:** racket-sport rally segmentation golden / regression dataset
4       **Sports in scope:** Badminton, Pickleball, Table Tennis (each in **singles _and_
doubles**)
5       **Companion doc:** [`ANNOTATION_GUIDE.md`](./ANNOTATION_GUIDE.md)
6
7       > This brief is the vendor-facing spec. The annotation deliverable is a per-rally CSV
that
8       > imports directly via `python -m src.cli.curate import-local`.
9
10      ## 0. What we need in one sentence
11      For each provided match video, produce **one CSV file** listing **every rally** with its
12      start time, end time, number of shots, and how it ended ? **excluding all dead time**.
13
14      ## 1. Deliverable format (canonical)
15      One CSV per video (UTF-8), named exactly `<video_id>.csv` (we supply the `video_id`).
16      One row per rally, chronological, header row required.
17
18      | column | meaning |
19      |---|---|
20      | `rally_number` | integer, 1-based, chronological. No gaps, no repeats. |
21      | `start_mmss` | rater-entered time as `m:ss.mmm` (e.g. `1:12.40`) ? serve contact. |
22      | `end_mmss` | rater-entered time as `m:ss.mmm` ? ball/shuttle dead. |
23      | `start_time` | decimal **seconds** (e.g. `72.40`). **Auto-filled** from `start_mmss`.
|
24      | `end_time` | decimal **seconds**. **Auto-filled** from `end_mmss`. |
25      | `shots_count` | integer. Total racket/paddle/bat contacts (serve = shot 1). |
26      | `ending_reason` | one of: `winner` \| `forced_error` \| `unforced_error` \|
`service_fault` \| `let` \| `other` (**why** it ended) |
27      | `last_shot_outcome` | one of: `in` \| `net` \| `out` \| `n_a` (**where** the last shot
landed; `n_a` for service_fault/let) |
28      | `score_before` | optional, `"A-B"`. Blank if not confidently readable. |
```

```
| `score_after` | optional, `"A-B"`. |
| `notes` | optional free text. **Required** when `ending_reason = other`. |

**Times (mm:ss ? decimal seconds).** Raters type only `start_mmss`/`end_mmss`. The template
Google Sheet fills `start_time`/`end_time` with this formula (shown for `start_time`, row 2):

```
=IF(B2="","", IFERROR(VALUE(LEFT(B2,FIND(":",B2)-1))*60 + VALUE(MID(B2,FIND(":",B2)+1,20)), VALUE(B2)))
```

On **Download as CSV**, the computed decimal seconds are exported. (If your tool already shows decimal seconds, type those into `start_time`/`end_time` and leave the mm:ss columns blank.)

Example (badminton):
```csv
rally_number,start_mmss,end_mmss,start_time,end_time,shots_count,ending_reason,last_shot_outcome,score_before,score_after,notes
1,0:12.50,0:25.00,12.50,25.00,9,winner,in,0-0,1-0,
2,0:45.00,0:47.30,45.00,47.30,1,service_fault,n_a,1-0,1-1,serve into net
3,1:01.20,1:18.85,61.20,78.85,14,unforced_error,out,1-1,2-1,final clear sailed long (no pressure)
```

Critical format rules
- `start_time`/`end_time` that reach us **must be decimal seconds** (the formula guarantees this).
- Times are relative to the **start of the file we provide**. Do **not** trim, clip, or re-encode the video ? our pipeline identifies a video by its file checksum.
- `end_time` > `start_time` for every row.
- No overlaps: `end_time[N] <= start_time[N+1]`.
- Everything between one rally's `end_time` and the next rally's `start_time` is **dead time** and gets **no row**.

2. The three non-negotiable rules (full detail + pictures in the Guide)
1. **Ignore all dead time.** Only label active play.
2. **Include every rally ? exclude none.** Every served point, including **service faults**,
   1?2 shot rallies, and **lets/replays**. A missed rally is the worst error in this project.
   Exception: warm-up / knock-up is NOT a rally ? pre-match and between-games warm-up looks
   like rallying but isn't scored play, so **don't label it**. Tell it apart by: no real serve,
   cooperative (not competitive) hitting, the score not progressing, often several shuttles/balls
   in use (full how-to in the Guide, Part 2). When unsure if the match has started, flag it.
3. **A shot = one contact** with the racket/paddle/bat. Serve = shot 1. Bounces and net-clips are not shots.

3. Scope & phasing
Phase A ? Pilot (this engagement): 3 videos (1 badminton + 1 pickleball + 1 table tennis),
each `< 3 min` with ~10?12 rallies. This is the **paid calibration round**; we review jointly and
lock the Guide before scale-up.

Phase B ? Scale (after a successful pilot): ~12?15 videos **per sport** (~36?45 total), each
~30 min. Same format, rules, and template.
```

```
76      > **Doubles are included** across the program. Rally rules are identical to singles; the
77      > badminton- and pickleball-specific doubles notes are in the Guide.
78
79      ## 4. Quality bar (acceptance criteria)
80      Automated validation (no overlaps, positive durations, schema-valid) runs on every file,
then a
81      ~20% human spot-check. A file is returned for rework (in scope) if it misses:
82      - **Completeness:** every rally present; zero missed rallies; zero dead-time rows.
83      - **Boundaries:** `start_time`/`end_time` within **±0.3 s** of true serve-contact /
landing.
84      - **shots_count:** exact for rallies ? 15 shots; ±1 allowed for longer/faster rallies.
85      - **ending_reason:** one of the six allowed values, per the Guide's decision tree.
86      - **Format:** decimal seconds present, valid columns, non-overlapping, chronological.
87
88      ## 5. Per-sport quick reference (full version + examples in the Guide)
89      **Badminton (singles & doubles)** ? start: racket?shuttle serve contact · end: shuttle
floors / fault · shot: each racket?shuttle contact.
90      **Pickleball (singles & doubles)** ? start: paddle?ball serve contact · end: double
bounce / out / net / fault · shot: each paddle?ball contact (floor bounces are **not** shots;
two-bounce rule is normal play).
91      **Table tennis** ? start: racket?ball serve strike (not the toss) · end: failed legal
return · shot: each racket?ball contact (table bounces are **not** shots). Net-cord serve =
`let`.
92
93      ## 6. ending_reason values (decision tree in the Guide)
94      This vocabulary attributes **why** the point ended; it is shared across all
95      net-separated sports (badminton, pickleball, table tennis, tennis). Record **where**
96      the last shot physically landed in the separate **`last_shot_outcome`** column
97      (`in` \| `net` \| `out` \| `n_a`) ? e.g. a forced error that flew long is
98      `ending_reason=forced_error, last_shot_outcome=out`.
99
100     | value | meaning |
101     |---|---|
102     | `winner` | last shot landed **in** and the opponent made no legal contact (a clean
winner). |
103     | `forced_error` | the losing player erred (netted / out / missed) **under pressure** ?
stretched, wrong-footed, or rushed by the opponent's strong shot. |
104     | `unforced_error` | the losing player erred on a **routine, unpressured** ball they had
time and position to make. |
105     | `service_fault` | the serve itself failed / was illegal. |
106     | `let` | the point was replayed. |
107     | `other` | anything else / cannot tell. **Must** add a note. |
108
109     ## 7. Recommended capture method
110     Use any tool that shows current time (mm:ss.mmm or seconds, or frame + known fps) and
can
111     **step one frame** forward/back: a frame-steppable player (VLC frame-step) or a timeline
tool
112     (Label Studio / CVAT). The deliverable is the CSV in the schema above.
113     `seconds = frame_number / fps` (we provide fps per video).
114
115     ## 8. Logistics & contacts
116     - **TODO(avidullu): paste the Google Drive folder link** where source videos are shared
with Roora.
117     - Per-video `fps` + `video_id`: provided in the intake manifest ? template
[`INTAKE_MANIFEST_TEMPLATE.csv`](./INTAKE_MANIFEST_TEMPLATE.csv). **TODO(avidullu): fill the
manifest (one row per pilot video) and share its link.**
118     - **TODO(avidullu): paste the Drive folder link** where Roora uploads completed
`<video_id>.csv` files.
119     - Blank annotation template Sheet (mm:ss ? seconds formula from §1 pre-filled):
**TODO(avidullu): create it and paste the link.**
120     - **TODO(avidullu): add the escalation channel** (email / WhatsApp group) for edge-case
questions.
121     - NDA before any footage is shared ? template [`NDA_TEMPLATE.md`](./NDA_TEMPLATE.md).
**TODO(avidullu): have counsel review, fill the blanks, and have Roora sign.**
```

```
122     - Roora contact on file: `info@rooragrp.com` (HSR Layout, Bangalore).
123
124     ## 9. Pricing request (please quote)
125     1. Price **per rally** and **per minute** of footage.
126     2. **Pilot** price (3 short videos, ~30?36 rallies) as a paid calibration round.
127     3. Indicative **scale** price for ~36?45 longer videos.
128     4. Turnaround for the pilot and throughput (rallies/day) at scale.
129     5. QA process and rework policy.
130     6. Whether annotators have racket-sport familiarity (helpful, not required).
131     7. Confirmation you can deliver the CSV schema above.
132
```

35. user (2026-06-10T05:36:20.602Z)

```
1       """Tests for the CVAT rally adapter (src/parsers/cvat/rally_cvat.py).
2
3       The adapter's whole reason for existing is to (a) recompute rally time from the
4       box FRAME RANGE at the true fps ? ignoring the vendor's wrong start/end_time
5       attributes ? and (b) report content defects rather than silently fixing them.
6       These tests pin both behaviours, plus the end-to-end hand-off to import-local.
7       """
8       import csv as _csv
9
10      from src.parsers.cvat.rally_cvat import (
11          RallyRecord, parse_cvat_boxes, build_records, detect_issues, convert,
12          write_canonical_csv, load_canonical_csv, build_summary, render_validation_md,
13          write_review_csv,
14      )
15      from src.cli.curate import SportsDataCLI
16
17
18      # --- helpers ---------------------------------------------------------
19
20      def _img_box(frame, attrs):
21          rows = "".join(f'<attribute name="{k}">{v}</attribute>' for k, v in attrs.items())
22          return (f'<image id="{frame}" name="frame_{frame:06d}" width="1920" height="1080">'
23                  f'<box label="rally" xtl="1" ytl="1" xbr="2" ybr="2">{rows}</box></image>')
24
25
26      def _write_cvat(path, images, step=30, stop_frame=1000):
27          body = "".join(images)
28          xml = (
29              '<?xml version="1.0" encoding="utf-8"?><annotations><version>1.1</version>'
30              f'<meta><job><stop_frame>{stop_frame}</stop_frame>'
31              f'<frame_filter>step={step}</frame_filter></job></meta>'
32              f'{body}</annotations>'
33          )
34          path.write_text(xml, encoding="utf-8")
35          return str(path)
36
37
38      def _rec(rn, start, end, shots=5, reason="winner", fmin=None, fmax=None, n=3,
outcome=None):
39          return RallyRecord(
40              rally_number=rn, start_time=start, end_time=end, shots_count=shots,
41              ending_reason=reason, last_shot_outcome=outcome,
42              score_before=None, score_after=None, notes=None,
43              frame_min=fmin if fmin is not None else int(start * 60),
44              frame_max=fmax if fmax is not None else int(end * 60), n_boxes=n)
45
46
47      # --- the core fix: time from frames, vendor times ignored ------------
48
49      def test_time_recomputed_from_frames_not_attributes(tmp_path):
50          # rally 1 spans source frames 60..120; the vendor's start/end attributes are
```

```
        # deliberately wrong (the frame/900 bug). At 60 fps the truth is 1.0..2.0s.
        xml = _write_cvat(tmp_path / "j.xml", [
            _img_box(60,  {"rally_number": "1.0", "start_time": "0.2", "end_time": "1.4",
                           "shots_count": "6.0", "ending_reason": "winner",
                           "score_before (A-B)": "0-0", "notes": "clean"}),
            _img_box(90,  {"rally_number": "1.0", "start_time": "0.2", "end_time": "1.4"}),
            _img_box(120, {"rally_number": "1.0", "start_time": "0.2", "end_time": "1.4"}),
        ])
        res = convert(xml, fps=60.0)
        assert len(res.records) == 1
        r = res.records[0]
        assert r.start_time == 1.0 and r.end_time == 2.0     # frames/fps, not 0.2/1.4
        assert r.vendor_start_time == "0.2"                  # preserved for transparency
        assert r.shots_count == 6
        assert r.ending_reason == "winner"
        assert r.score_before == "0-0"                       # "(A-B)" alias resolved
        assert r.notes == "clean"


    def test_step_and_fps_read_from_meta_and_arg(tmp_path):
        xml = _write_cvat(tmp_path / "j.xml", [
            _img_box(0, {"rally_number": "1", "shots_count": "3"}),
            _img_box(30, {"rally_number": "1"}),
        ], step=30)
        res = convert(xml, fps=60.0)
        assert res.step == 30
        assert res.fps == 60.0


    def test_fps_required_when_unavailable(tmp_path):
        xml = _write_cvat(tmp_path / "j.xml", [_img_box(0, {"rally_number": "1"})])
        try:
            convert(xml)  # no fps, no video_path
        except ValueError as e:
            assert "fps" in str(e).lower()
        else:
            raise AssertionError("expected ValueError when fps cannot be determined")


    # --- track-mode parsing + outside filtering --------------------------------

    def test_track_mode_and_outside_filtering(tmp_path):
        xml = (
            '<?xml version="1.0"?><annotations><meta><job>'
            '<stop_frame>1000</stop_frame><frame_filter>step=30</frame_filter></job></meta>'
            '<track id="0" label="rally">'
            '<box frame="60" outside="0" xtl="1" ytl="1" xbr="2" ybr="2">'
            '<attribute name="rally_number">1</attribute>'
            '<attribute name="shots_count">4</attribute></box>'
            '<box frame="120" outside="0" xtl="1" ytl="1" xbr="2" ybr="2">'
            '<attribute name="rally_number">1</attribute></box>'
            '<box frame="150" outside="1" xtl="1" ytl="1" xbr="2" ybr="2">'  # terminator
            '<attribute name="rally_number">1</attribute></box>'
            '</track></annotations>'
        )
        p = tmp_path / "track.xml"
        p.write_text(xml, encoding="utf-8")
        meta, boxes = parse_cvat_boxes(str(p))
        assert len(boxes) == 2                    # the outside=1 terminator dropped
        recs = build_records(boxes, fps=60.0)
        assert recs[0].frame_max == 120           # not 150 (the terminator)
        assert recs[0].end_time == 2.0


    # --- issue detection: surfaced, not silently fixed -------------------------
```

```
116
117     def test_overlap_is_a_blocker():
118         recs = [_rec(1, 1.0, 5.0), _rec(2, 4.0, 8.0)]   # r2 starts before r1 ends
119         issues = detect_issues(recs)
120         overlaps = [i for i in issues if i.code == "overlap"]
121         assert len(overlaps) == 1 and overlaps[0].severity == "blocker"
122
123
124     def test_zero_duration_is_a_blocker():
125         recs = [_rec(1, 2.0, 2.0, fmin=120, fmax=120, n=1)]
126         issues = detect_issues(recs)
127         assert any(i.code == "zero_duration" and i.severity == "blocker" for i in issues)
128
129
130     def test_offvocab_ending_reason_flagged():
131         recs = [_rec(1, 1.0, 3.0, reason="into_net")]
132         issues = detect_issues(recs)
133         assert any(i.code == "ending_reason_vocab" for i in issues)
134         # the adopted forced/unforced terms are accepted, not flagged
135         ok = detect_issues([_rec(1, 1.0, 3.0, reason="forced_error")])
136         assert not any(i.code == "ending_reason_vocab" for i in ok)
137
138
139     def test_low_shot_density_flags_inflated_span():
140         recs = [_rec(1, 0.0, 16.0, shots=2)]        # 2 shots over 16s -> 0.12/s
141         issues = detect_issues(recs)
142         assert any(i.code == "shot_density" for i in issues)
143
144
145     def test_outcome_vocab_flagged():
146         bad = detect_issues([_rec(1, 1.0, 3.0, outcome="long")])
147         assert any(i.code == "outcome_vocab" for i in bad)
148         # the agreed outcomes are accepted, not flagged
149         ok = detect_issues([_rec(1, 1.0, 3.0, outcome="out")])
150         assert not any(i.code == "outcome_vocab" for i in ok)
151
152
153     # --- end-to-end: emitted CSV imports via the existing import-local parser ---
154
155     def test_canonical_csv_round_trips_through_import_local(tmp_path):
156         xml = _write_cvat(tmp_path / "j.xml", [
157             _img_box(60,  {"rally_number": "1", "start_time": "0.2", "end_time": "1.4",
158                            "shots_count": "6", "ending_reason": "winner",
159                            "last_shot_outcome": "in"}),
160             _img_box(120, {"rally_number": "1"}),
161             _img_box(180, {"rally_number": "2", "shots_count": "4",
162                            "ending_reason": "forced_error", "last_shot_outcome": "out"}),
163             _img_box(240, {"rally_number": "2"}),
164         ])
165         res = convert(xml, fps=60.0)
166         assert res.n_blockers == 0
167         assert res.records[1].last_shot_outcome == "out"   # parsed from the box attribute
168         out = tmp_path / "canonical.csv"
169         write_canonical_csv(res.records, str(out))
170
171         # Re-read with the actual import-local parser (self is unused -> None is safe).
172         items = SportsDataCLI._parse_local_annotations(None, "badminton", "vid1", str(out))
173         assert len(items) == 2
174         assert items[0].start_time == 1.0 and items[0].end_time == 2.0   # not 0.2/1.4
175         assert items[1].ending_reason == "forced_error"
176         assert items[1].last_shot_outcome == "out"          # survives the round-trip into
the schema
177         assert items[1].start_time == 3.0 and items[1].end_time == 4.0
178
179
```

```
180     # --- validate-delivery: gap heuristic, CSV loading, report artifacts ----------
181
182     def test_possible_missed_rally_gap_flagged():
183         # 20s of unlabeled time between two rallies -> flag (catches the 1:46-style miss)
184         recs = [_rec(1, 95.6, 98.0), _rec(2, 118.0, 119.6)]
185         issues = detect_issues(recs)
186         assert any(i.code == "possible_missed_rally" for i in issues)
187
188
189     def test_normal_gap_not_flagged():
190         recs = [_rec(1, 10.0, 14.0), _rec(2, 22.0, 26.0)]   # 8s gap -> fine
191         issues = detect_issues(recs)
192         assert not any(i.code == "possible_missed_rally" for i in issues)
193
194
195     def test_load_canonical_csv_decimal_and_mmss(tmp_path):
196         p = tmp_path / "c.csv"
197         p.write_text(
198             "rally_number,start_time,end_time,start_mmss,end_mmss,shots_count,ending_reason,
last_shot_outcome\n"
199             "1,3.0,21.0,,,15,unforced_error,out\n"
200             "2,,,0:29.000,0:32.000,2,winner,in\n", encoding="utf-8")
201         recs = load_canonical_csv(str(p))
202         assert len(recs) == 2
203         assert recs[0].start_time == 3.0 and recs[0].last_shot_outcome == "out"
204         assert recs[1].start_time == 29.0 and recs[1].end_time == 32.0   # mm:ss fallback
205
206
207     def test_load_canonical_csv_keeps_rally_starting_at_zero(tmp_path):
208         # Regression: a rally that legitimately starts at 0.0s must not be dropped.
209         # The old `start_time or start_mmss` fallback treated the falsy 0.0 as "missing"
210         # and skipped the row when there was no start_mmss column to fall back to.
211         p = tmp_path / "c.csv"
212         p.write_text(
213             "rally_number,start_time,end_time,shots_count,ending_reason,last_shot_outcome\n"
214             "1,0.0,4.0,8,winner,in\n"
215             "2,10.0,14.0,6,forced_error,out\n"
216             "3,20.0,24.0,5,unforced_error,out\n", encoding="utf-8")
217         recs = load_canonical_csv(str(p))
218         assert len(recs) == 3                       # the zero-start row 1 is not lost
219         assert recs[0].start_time == 0.0 and recs[0].end_time == 4.0
220
221
222     def test_validation_artifacts(tmp_path):
223         recs = [_rec(1, 1.0, 5.0), _rec(2, 4.0, 8.0, reason="into_net")]  # overlap + bad
vocab
224         issues = detect_issues(recs)
225         summary = build_summary(recs, issues, source="vid", fps=59.94)
226         assert summary["n_rallies"] == 2
227         assert summary["passed"] is False and summary["n_blockers"] >= 1
228         assert 2 in summary["flagged_rallies"]
229         md = render_validation_md(recs, issues, "vid")
230         assert "BLOCKED" in md and "## BLOCKER" in md
231         out = tmp_path / "review.csv"
232         write_review_csv(recs, issues, str(out))
233         rows = list(_csv.DictReader(open(out, encoding="utf-8")))
234         assert "VERIFIED(Y/N)" in rows[0] and "auto_flags" in rows[0]
235         assert any("overlap" in r["auto_flags"] for r in rows)
236
```

36. user (2026-06-10T05:36:28.894Z)

```
No matches found
```

37. user (2026-06-10T05:36:30.187Z)

```
3-The VLC rally-annotator produces a human-rated CSV (rally_number,start_time,end_time,
4-ending_reason,sport). While a video is only partially tagged, the labels cover a
5-PREFIX of the video ? every rally up to some point is marked, and everything after is
6-simply un-watched (not "no rally there"). Scoring naively against such labels is wrong:
7-the detector's *correct* rallies in the un-labeled tail would be counted as false
8-positives and crush precision.
9-
10-This harness fixes that by restricting the evaluation to the LABELED WINDOW
11:[window_start, window_end] (window_end defaults to the last labeled rally's end) and
12-dropping predicted segments that fall outside it. Within that window the human labels
13-ARE complete, so precision/recall/F1 are honest.
14-
15-It reuses everything from the existing eval stack (no new windowing/metrics/gate logic):
16-  - calibrate_wasb.make_predict_fn / load_golden_gts / BASELINE
17-  - pipeline.detectors.rally_gate.apply_gate   (net-crossing "gate A")
18-  - eval.metrics.match_intervals / score
19-  - export_wasb_segments.build_comparison / write_comparison_csv  (per-rally detail CSV)
--
39-Interval = Tuple[float, float]
40-Config = Tuple[float, float, float]
41-
42-# Operating points to compare. The tuned config wins on its fit-video but over-segments
43-# and does not transfer; baseline (+ net-crossing gate) is the stronger general point.
44-TUNED: Config = (0.05, 1.0, 1.0)
45-
46-
47:def restrict_to_window(intervals: List[Interval], w0: float, w1: float) -> List[Interval]:
48-    """Keep intervals that overlap the labeled window [w0, w1].
49-
50-    A predicted segment is evaluated iff it overlaps the labeled region; segments wholly
51-    in the un-labeled tail (start >= w1) or before the window (end <= w0) are dropped so
52-    they are neither rewarded nor penalized. GTs are assumed already inside the window.
53-    """
54-    return [(s, e) for (s, e) in intervals if e > w0 and s < w1]
55-
--
61-    if gate_crossings > 0 and points:
62-        preds = rally_gate.apply_gate(points, preds, fps, min_crossings=gate_crossings)
63-    preds = restrict_to_window(preds, *window)
64-    sc = score(preds, gts, iou)
65-    return {"cfg": cfg, "gate": gate_crossings, "n_pred": len(preds), "score": sc, "preds":
preds}
66-
67-
68-def run(trajectory_csv: str, labels_csv: str, fps: float, frame_width: float,
69:        window_start: float = 0.0, window_end: Optional[float] = None,
70-        gate_crossings: int = 2, iou: float = 0.5,
71-        comparison_out: Optional[str] = None) -> dict:
72-    gts_all = load_golden_gts(labels_csv)
73-    if not gts_all:
74-        raise SystemExit(f"no usable human rallies in {labels_csv}")
75:    if window_end is None:
76:        window_end = max(e for _, e in gts_all)
77:    window = (window_start, window_end)
78-    gts = restrict_to_window(gts_all, *window)
79-
80-    predict_fn = make_predict_fn(trajectory_csv, fps, frame_width)
81-    points = TrackNetRunner.parse_trajectory_csv(trajectory_csv)
82-    traj_end = (max(p.frame for p in points) / fps) if points else 0.0
83-
84-    print("=== Pipeline vs PARTIAL human labels ===")
```

[Showing results with pagination = limit: 60]

38. user (2026-06-10T05:37:54.084Z)

```
    """Best-quality video acquisition.

    A single place that knows how to fetch the *highest resolution available* for a
    source URL and report what was actually obtained. Both the ShuttleSet crawler
    (initial ingestion) and the `fetch` CLI command (in-place upgrade of already-
    ingested videos) go through here, so the resolution policy lives in exactly one
    spot.

    Design notes:
    - `bv*+ba/b` selects best video + best audio, falling back to the best
      pre-muxed stream. Combined with `-S res,fps,...`, yt-dlp sorts candidate
      formats by resolution (then fps) descending and picks the top ? i.e. *always*
      the best resolution available, regardless of codec/container quirks.
    - `--merge-output-format mp4` guarantees an MP4 container, which the downstream
      indexer requires.
    - We never trust the requested resolution; we ffprobe the resulting file and
      record the *actual* width/height/fps as the source of truth.
    """

    import os
    import shutil
    import subprocess
    from dataclasses import dataclass
    from typing import List, Optional, Tuple

    # Best video+audio, prefer mp4/m4a, fall back to best single stream.
    _FORMAT = "bv*+ba/b"
    # Sort candidates so the highest resolution (then fps) wins; prefer mp4/m4a on ties.
    _SORT = "res,fps,ext:mp4:m4a"


    def height_to_label(height: Optional[int]) -> Optional[str]:
        """Map a pixel height to a conventional resolution label (e.g. 1080 -> '1080p').

        Snaps to the nearest standard tier at or below the actual height so a 1072px
        crop still reads as '1080p' rather than an odd number.
        """
        if not height or height <= 0:
            return None
        # Allow a 5% tolerance below each tier so a lightly letterboxed/cropped frame
        # (e.g. 1072px) still reads as the intended tier ('1080p') rather than dropping.
        for tier in (2160, 1440, 1080, 720, 480, 360, 240, 144):
            if height >= tier * 0.95:
                return f"{tier}p"
        return f"{height}p"


    def build_ytdlp_command(url: str, dest_path: str, max_height: Optional[int] = None,
                            cookies_from_browser: Optional[str] = None,
                            cookies_file: Optional[str] = None,
                            remote_components: Optional[str] = "ejs:github",
                            player_client: Optional[str] = "default") -> List[str]:
        """Build the yt-dlp argv for a best-quality download.

        `max_height`, when set, caps resolution (e.g. avoid pulling 4K); left unset
        it fetches the absolute best available. Pure/argv-only so it is unit-testable
        without invoking the network.

        `player_client` selects YouTube's player client (default `default` = yt-dlp's
        maintained client set, which serves real downloadable 1080p DASH with auth
        cookies). Avoid `web`/`web_safari`: YouTube forces those into SABR streaming,
        which strips download URLs and silently drops to the lowest stream (the 144p
        bug). Harmless for non-YouTube URLs (the extractor-arg is ignored). Set to
        None to omit the arg entirely.

```

```
        `remote_components` enables yt-dlp's EJS challenge solver (default
        `ejs:github`), the JS-runtime (deno) path for solving YouTube's n-challenge.
        Set to None to disable (e.g. for non-YouTube sources or offline tests).
        """
        fmt = _FORMAT
        if max_height:
            # Cap each stream selector; keep the pre-muxed fallback capped too.
            fmt = f"bv*[height<={max_height}]+ba/b[height<={max_height}]/b"
        cmd = [
            "yt-dlp",
            "-f", fmt,
            "-S", _SORT,
            "--merge-output-format", "mp4",
            "--no-playlist",
            "--no-progress",
            "-o", dest_path,
        ]
        if player_client:
            cmd += ["--extractor-args", f"youtube:player_client={player_client}"]
        if remote_components:
            cmd += ["--remote-components", remote_components]
        if cookies_from_browser:
            cmd += ["--cookies-from-browser", cookies_from_browser]
        if cookies_file:
            cmd += ["--cookies", cookies_file]
        cmd.append(url)
        return cmd


    def _env_with_deno_on_path() -> dict:
        """Return an environment with the default deno install dir on PATH.

        The EJS challenge solver needs a JS runtime (deno). The official installer
        drops it at ~/.deno/bin, but a shell started before install won't have it on
        PATH yet. We add it here so `fetch` works without requiring a terminal
        restart ? without clobbering a deno the user already has elsewhere.
        """
        env = os.environ.copy()
        if shutil.which("deno"):
            return env  # already discoverable
        deno_bin = os.path.join(os.path.expanduser("~"), ".deno", "bin")
        if os.path.isdir(deno_bin):
            env["PATH"] = deno_bin + os.pathsep + env.get("PATH", "")
        return env


    def probe_resolution_fps(path: str) -> Tuple[Optional[int], Optional[int],
Optional[float]]:
        """Return (width, height, fps) of a video file via ffprobe, best-effort.

        Returns (None, None, None) if ffprobe is unavailable or the file can't be
        parsed, so a missing ffprobe degrades gracefully rather than crashing a fetch.
        """
        if not shutil.which("ffprobe") or not os.path.exists(path):
            return (None, None, None)
        try:
            out = subprocess.run(
                ["ffprobe", "-v", "error", "-select_streams", "v:0",
                 "-show_entries", "stream=width,height,avg_frame_rate",
                 "-of", "default=noprint_wrappers=1:nokey=1", path],
                stdout=subprocess.PIPE, stderr=subprocess.PIPE, text=True, check=True,
            ).stdout.split()
        except (subprocess.SubprocessError, OSError):
            return (None, None, None)

```

```
        width = int(out[0]) if len(out) > 0 and out[0].isdigit() else None
        height = int(out[1]) if len(out) > 1 and out[1].isdigit() else None
        fps = None
        if len(out) > 2 and "/" in out[2]:
            num, den = out[2].split("/")
            try:
                den_f = float(den)
                fps = round(float(num) / den_f, 3) if den_f else None
            except ValueError:
                fps = None
        return (width, height, fps)


    @dataclass
    class DownloadResult:
        success: bool
        path: Optional[str] = None
        size_mb: float = 0.0
        width: Optional[int] = None
        height: Optional[int] = None
        fps: Optional[float] = None
        resolution_label: Optional[str] = None
        error: Optional[str] = None


    def download_best_video(url: str, dest_path: str, max_height: Optional[int] = None,
                            max_size_mb: Optional[float] = None,
                            cookies_from_browser: Optional[str] = None,
                            cookies_file: Optional[str] = None,
                            player_client: Optional[str] = "default") -> DownloadResult:
        """Download the best-resolution video for `url` to `dest_path` (MP4).

        On success, ffprobes the file and reports actual width/height/fps. If
        `max_size_mb` is exceeded the file is removed and a failed result returned,
        so callers never register an over-budget download.
        """
        os.makedirs(os.path.dirname(os.path.abspath(dest_path)), exist_ok=True)
        cmd = build_ytdlp_command(url, dest_path, max_height=max_height,
                                  cookies_from_browser=cookies_from_browser,
                                  cookies_file=cookies_file,
                                  player_client=player_client)
        try:
            subprocess.run(cmd, stdout=subprocess.PIPE, stderr=subprocess.PIPE,
                           text=True, check=True, env=_env_with_deno_on_path())
        except FileNotFoundError:
            return DownloadResult(success=False, error="yt-dlp not found on PATH")
        except subprocess.CalledProcessError as e:
            return DownloadResult(success=False, error=(e.stderr or str(e)).strip()[:500])

        if not os.path.exists(dest_path):
            return DownloadResult(success=False, error="yt-dlp reported success but no file
was produced")

        size_mb = os.path.getsize(dest_path) / (1024 * 1024)
        if max_size_mb is not None and size_mb > max_size_mb:
            os.remove(dest_path)
            return DownloadResult(
                success=False, size_mb=size_mb,
                error=f"downloaded size {size_mb:.1f} MB exceeds limit {max_size_mb:.1f}
MB",
            )

        width, height, fps = probe_resolution_fps(dest_path)
        return DownloadResult(
            success=True, path=dest_path, size_mb=size_mb,
```

```
193             width=width, height=height, fps=fps,
194             resolution_label=height_to_label(height),
195         )
196
```

39. user (2026-06-10T05:37:56.129Z)

```
eval_export
mock_local_badminton_anno.json
roora_badminton_canonical.csv
roora_badminton_issues.txt
sports_metadata.db
staging_coachai_match.csv
temp
validation
videos
#!/usr/bin/env python3
import os
import sys
import shutil
import argparse
import math

from src.core.config import load_config

def get_file_or_dir_size(path):
    """Calculates the size of a file or directory in bytes."""
    if not os.path.exists(path):
        return 0
    if os.path.isfile(path):
        return os.path.getsize(path)

    total_size = 0
    for dirpath, _, filenames in os.walk(path):
        for f in filenames:
            fp = os.path.join(dirpath, f)
            if os.path.exists(fp) and not os.path.islink(fp):
                total_size += os.path.getsize(fp)
    return total_size

def format_size(size_bytes):
    """Formats bytes to a human-readable string."""
    if size_bytes == 0:
        return "0 B"
    size_name = ("B", "KB", "MB", "GB")
    i = int(math.floor(math.log(size_bytes, 1024)))
    p = math.pow(1024, i)
    s = round(size_bytes / p, 2)
    return f"{s} {size_name[i]}"

def get_all_items_in_dir(path):
    """Returns a list of absolute paths of all files/directories inside a directory."""
    items = []
    if not os.path.exists(path) or not os.path.isdir(path):
        return items
    for entry in os.listdir(path):
        items.append(os.path.join(path, entry))
    return items

def main():
    parser = argparse.ArgumentParser(
        description="Purge stored state including SQLite database, downloaded videos, and
temporary files."
    )
    parser.add_argument(
```

```
        "--config",
        default="config/config.yaml",
  320 /c/Users/avidu/Projects/sports-data-collector/src/crawlers/badminton_crawler.py
  301 /c/Users/avidu/Projects/sports-data-collector/tests/test_cli.py
   25 /c/Users/avidu/Projects/sports-data-collector/tests/test_config.py
  235 /c/Users/avidu/Projects/sports-data-collector/tests/test_cvat_parser.py
  247 /c/Users/avidu/Projects/sports-data-collector/tests/test_db.py
  145 /c/Users/avidu/Projects/sports-data-collector/tests/test_downloader.py
   46 /c/Users/avidu/Projects/sports-data-collector/tests/test_local_import.py
  111 /c/Users/avidu/Projects/sports-data-collector/tests/test_normalize_annotations.py
  111 /c/Users/avidu/Projects/sports-data-collector/tests/test_parser.py
   31 /c/Users/avidu/Projects/sports-data-collector/tests/test_parsing.py
  135 /c/Users/avidu/Projects/sports-data-collector/tests/test_reporting.py
   49 /c/Users/avidu/Projects/sports-data-collector/tests/test_reporting_faq.py
   37 /c/Users/avidu/Projects/sports-data-collector/tests/test_schemas.py
  117 /c/Users/avidu/Projects/sports-data-collector/tests/test_validator.py
 1910 total
```

40. assistant (2026-06-10T05:38:07.052Z)

Now let me empirically verify the key landmines (duplicate rally_number PK collision, corrupt XML handling) in a temp sandbox.

41. user (2026-06-10T05:38:16.187Z)

```
INSERT raised: IntegrityError UNIQUE constraint failed: badminton_rallies.video_id,
badminton_rallies.rally_number
rows in table after failure: 0
validator ok? True errors: []
corrupt xml raises: ParseError | is ValueError? False | is ParseError? True
```

42. user (2026-06-10T05:38:42.842Z)

```
records: 0 issues: 0 blockers: 0
summary passed? True n_rallies: 0
Rally_Number (case only) records: 1
```

43. user (2026-06-10T05:38:54.615Z)

```
/c/Users/avidu/Projects/sports-data-collector/tests/test_cli.py:58:def
test_import_local_badminton_json(cli, tmp_path):
/c/Users/avidu/Projects/sports-data-collector/tests/test_cli.py:68:def
test_import_local_pickleball_csv(cli, tmp_path):
/c/Users/avidu/Projects/sports-data-collector/tests/test_cli.py:76:def
test_import_local_tabletennis_json(cli, tmp_path):
/c/Users/avidu/Projects/sports-data-collector/tests/test_cli.py:85:def
test_import_local_tennis_csv(cli, tmp_path):
/c/Users/avidu/Projects/sports-data-collector/tests/test_cli.py:93:def
test_import_local_cricket_json(cli, tmp_path):
/c/Users/avidu/Projects/sports-data-collector/tests/test_cli.py:106:def
test_import_local_unsupported_sport_exits(cli, tmp_path):
/c/Users/avidu/Projects/sports-data-collector/tests/test_cli.py:113:def
test_import_local_missing_annotations_exits(cli):
/c/Users/avidu/Projects/sports-data-collector/tests/test_cli.py:118:def
test_import_local_overlap_fails_validation(cli, tmp_path):
/c/Users/avidu/Projects/sports-data-collector/tests/test_cli.py:126:def
test_import_local_unknown_license_defaults(cli, tmp_path):
/c/Users/avidu/Projects/sports-data-collector/tests/test_cli.py:135:def
test_import_local_json_missing_required_field_exits(cli, tmp_path):
/c/Users/avidu/Projects/sports-data-collector/tests/test_cli.py:143:def
test_import_local_bad_extension_exits(cli, tmp_path):
/c/Users/avidu/Projects/sports-data-collector/tests/test_cli.py:154:def
test_status_empty_and_populated(cli, tmp_path):
/c/Users/avidu/Projects/sports-data-collector/tests/test_cli.py:162:def
```

```
test_export_eval_set(cli, tmp_path):
/c/Users/avidu/Projects/sports-data-collector/tests/test_cli.py:197:def test_curate_quit(cli):
/c/Users/avidu/Projects/sports-data-collector/tests/test_cli.py:203:def test_curate_empty(cli):
/c/Users/avidu/Projects/sports-data-collector/tests/test_cli.py:207:def
test_curate_review_keep(cli):
/c/Users/avidu/Projects/sports-data-collector/tests/test_cli.py:214:def
test_curate_approve(cli):
/c/Users/avidu/Projects/sports-data-collector/tests/test_cli.py:221:def test_curate_reject(cli):
/c/Users/avidu/Projects/sports-data-collector/tests/test_cli.py:228:def
test_curate_play_segment(cli):
/c/Users/avidu/Projects/sports-data-collector/tests/test_cli.py:250:def
test_fetch_downloads_and_updates_db(cli, tmp_path):
/c/Users/avidu/Projects/sports-data-collector/tests/test_cli.py:261:def
test_fetch_handles_failure(cli):
/c/Users/avidu/Projects/sports-data-collector/tests/test_cli.py:283:def test_main_dispatch(argv,
method):
/c/Users/avidu/Projects/sports-data-collector/tests/test_cli.py:290:def
test_main_import_local_dispatch():
/c/Users/avidu/Projects/sports-data-collector/tests/test_cli.py:298:def
test_main_no_command_prints_help():
/c/Users/avidu/Projects/sports-data-collector/tests/test_db.py:16:def
test_insert_and_get_video(temp_db):
/c/Users/avidu/Projects/sports-data-collector/tests/test_db.py:39:def
test_badminton_rally_insertion(temp_db):
/c/Users/avidu/Projects/sports-data-collector/tests/test_db.py:62:def
test_reinsert_video_preserves_child_rallies(temp_db):
/c/Users/avidu/Projects/sports-data-collector/tests/test_db.py:86:def
test_get_segment_intervals_badminton(temp_db):
/c/Users/avidu/Projects/sports-data-collector/tests/test_db.py:104:def
test_pickleball_rally_insertion(temp_db):
/c/Users/avidu/Projects/sports-data-collector/tests/test_db.py:128:def
test_tabletennis_rally_insertion(temp_db):
/c/Users/avidu/Projects/sports-data-collector/tests/test_db.py:149:def
test_get_segment_intervals_pickleball(temp_db):
/c/Users/avidu/Projects/sports-data-collector/tests/test_db.py:163:def
test_tennis_point_insertion(temp_db):
/c/Users/avidu/Projects/sports-data-collector/tests/test_db.py:180:def
test_cricket_delivery_insertion(temp_db):
/c/Users/avidu/Projects/sports-data-collector/tests/test_db.py:198:def
test_delete_cascades_child_rows(temp_db):
/c/Users/avidu/Projects/sports-data-collector/tests/test_db.py:211:def
test_get_segment_intervals_empty_returns_empty(temp_db):
/c/Users/avidu/Projects/sports-data-collector/tests/test_db.py:216:def
test_get_status_counts(temp_db):
/c/Users/avidu/Projects/sports-data-collector/tests/test_db.py:230:def
test_update_status_and_delete(temp_db):
/c/Users/avidu/Projects/sports-data-collector/tests/test_local_import.py:17:def
test_parse_pickleball_csv(tmp_path):
/c/Users/avidu/Projects/sports-data-collector/tests/test_local_import.py:34:def
test_parse_tabletennis_json(tmp_path):
/c/Users/avidu/Projects/sports-data-collector/tests/test_normalize_annotations.py:18:def
test_drops_blank_and_reversed_end_rows():
/c/Users/avidu/Projects/sports-data-collector/tests/test_normalize_annotations.py:32:def
test_drops_negative_start_rows():
/c/Users/avidu/Projects/sports-data-collector/tests/test_normalize_annotations.py:43:def
test_renumbers_fractional_and_duplicate_ids_by_start():
/c/Users/avidu/Projects/sports-data-collector/tests/test_normalize_annotations.py:56:def
test_duplicate_truncating_ids_are_separated():
/c/Users/avidu/Projects/sports-data-collector/tests/test_normalize_annotations.py:66:def
test_mmss_times_converted_and_counted():
/c/Users/avidu/Projects/sports-data-collector/tests/test_normalize_annotations.py:74:def
test_pipe_delimited_reason_folds_to_primary_plus_note():
/c/Users/avidu/Projects/sports-data-collector/tests/test_normalize_annotations.py:82:def
test_clean_lean_input_is_passthrough():
/c/Users/avidu/Projects/sports-data-collector/tests/test_normalize_annotations.py:91:def
```

```
test_normalized_output_ingests_through_import_local_parser(tmp_path):
/c/Users/avidu/Projects/sports-data-collector/tests/test_validator.py:15:def
test_license_checks(validator):
/c/Users/avidu/Projects/sports-data-collector/tests/test_validator.py:23:def
test_validate_badminton_rallies(validator):
/c/Users/avidu/Projects/sports-data-collector/tests/test_validator.py:43:def
test_validate_tennis_points(validator):
/c/Users/avidu/Projects/sports-data-collector/tests/test_validator.py:61:def
test_validate_empty_lists_are_ok(validator):
/c/Users/avidu/Projects/sports-data-collector/tests/test_validator.py:69:def
test_validate_cricket_deliveries(validator):
/c/Users/avidu/Projects/sports-data-collector/tests/test_validator.py:85:def
test_validate_pickleball_rallies(validator):
/c/Users/avidu/Projects/sports-data-collector/tests/test_validator.py:102:def
test_validate_tabletennis_rallies(validator):
/c/Users/avidu/Projects/sports-data-collector/tests/test_schemas.py:15:def
test_rally_end_not_after_start_raises(cls):
/c/Users/avidu/Projects/sports-data-collector/tests/test_schemas.py:21:def
test_rally_negative_start_raises(cls):
/c/Users/avidu/Projects/sports-data-collector/tests/test_schemas.py:26:def
test_tennis_point_guards():
/c/Users/avidu/Projects/sports-data-collector/tests/test_schemas.py:33:def
test_cricket_delivery_guards():
/c/Users/avidu/Projects/sports-data-collector/tests/test_downloader.py:15:def
test_command_requests_best_quality_mp4():
/c/Users/avidu/Projects/sports-data-collector/tests/test_downloader.py:29:def
test_command_can_disable_remote_components():
/c/Users/avidu/Projects/sports-data-collector/tests/test_downloader.py:34:def
test_command_pins_player_client_by_default():
/c/Users/avidu/Projects/sports-data-collector/tests/test_downloader.py:40:def
test_command_player_client_can_be_disabled():
/c/Users/avidu/Projects/sports-data-collector/tests/test_downloader.py:45:def
test_command_caps_resolution_when_max_height_set():
/c/Users/avidu/Projects/sports-data-collector/tests/test_downloader.py:51:def
test_command_passes_cookies_when_requested():
/c/Users/avidu/Projects/sports-data-collector/tests/test_downloader.py:56:def
test_command_passes_cookies_file():
/c/Users/avidu/Projects/sports-data-collector/tests/test_downloader.py:67:def
test_height_to_label(height, label):
/c/Users/avidu/Projects/sports-data-collector/tests/test_downloader.py:73:def
test_probe_resolution_fps_parses_ffprobe(monkeypatch):
/c/Users/avidu/Projects/sports-data-collector/tests/test_downloader.py:86:def
test_probe_resolution_fps_missing_ffprobe(monkeypatch):
/c/Users/avidu/Projects/sports-data-collector/tests/test_downloader.py:100:def
test_download_best_video_success(monkeypatch):
/c/Users/avidu/Projects/sports-data-collector/tests/test_downloader.py:114:def
test_download_best_video_over_size_limit_removes_file(monkeypatch):
/c/Users/avidu/Projects/sports-data-collector/tests/test_downloader.py:128:def
test_download_best_video_ytdlp_missing(monkeypatch):
/c/Users/avidu/Projects/sports-data-collector/tests/test_downloader.py:138:def
test_download_best_video_ytdlp_error(monkeypatch):
/c/Users/avidu/Projects/sports-data-collector/tests/test_parser.py:20:def
test_coachai_parser(mock_coachai_csv):
/c/Users/avidu/Projects/sports-data-collector/tests/test_parser.py:56:def
test_coachai_parser_time_string():
/c/Users/avidu/Projects/sports-data-collector/tests/test_parser.py:65:def
test_coachai_parser_tolerates_empty_cells(tmp_path):
/c/Users/avidu/Projects/sports-data-collector/tests/test_parser.py:81:def
test_coachai_buffers_never_create_overlap(tmp_path):
/c/Users/avidu/Projects/sports-data-collector/tests/test_parser.py:99:def
test_coachai_commercial_flag_uses_config_whitelist(tmp_path):
/c/Users/avidu/Projects/sports-data-collector/tests/test_reporting.py:26:def
test_issue_to_flag_severity_and_customer_message():
/c/Users/avidu/Projects/sports-data-collector/tests/test_reporting.py:39:def
test_delivery_with_blocker_fails_and_surfaces_to_customer():
```

```
/c/Users/avidu/Projects/sports-data-collector/tests/test_reporting.py:60:def
test_delivery_flag_carries_frame_evidence_and_review_framing():
/c/Users/avidu/Projects/sports-data-collector/tests/test_reporting.py:75:def
test_delivery_clean_passes():
/c/Users/avidu/Projects/sports-data-collector/tests/test_reporting.py:91:def
test_import_clean_run():
/c/Users/avidu/Projects/sports-data-collector/tests/test_reporting.py:102:def
test_import_noncommercial_warns():
/c/Users/avidu/Projects/sports-data-collector/tests/test_reporting.py:108:def
test_import_missing_fps_warns():
/c/Users/avidu/Projects/sports-data-collector/tests/test_reporting.py:116:def
test_emit_delivery_writes_into_out_dir(tmp_path):
/c/Users/avidu/Projects/sports-data-collector/tests/test_reporting.py:125:def
test_emit_import_writes_next_to_video(tmp_path):
/c/Users/avidu/Projects/sports-data-collector/tests/test_reporting_faq.py:37:def
test_all_collector_faq_refs_resolve():
/c/Users/avidu/Projects/sports-data-collector/tests/test_reporting_faq.py:46:def
test_actionable_rules_have_faq_refs():
/c/Users/avidu/Projects/sports-data-collector/tests/test_parsing.py:6:def test_safe_int():
/c/Users/avidu/Projects/sports-data-collector/tests/test_parsing.py:16:def
test_safe_float_required():
/c/Users/avidu/Projects/sports-data-collector/tests/test_parsing.py:25:def
test_parse_time_string():
/c/Users/avidu/Projects/sports-data-collector/tests/test_config.py:6:def
test_load_config_missing_returns_empty(tmp_path):
/c/Users/avidu/Projects/sports-data-collector/tests/test_config.py:10:def
test_load_config_reads_values(tmp_path):
/c/Users/avidu/Projects/sports-data-collector/tests/test_config.py:18:def
test_load_config_malformed_returns_empty(tmp_path):
/c/Users/avidu/Projects/sports-data-collector/tests/test_config.py:24:def
test_get_database_path_default():
```

44. user (2026-06-10T05:39:08.463Z)

```
1       """End-to-end coverage for the SportsDataCLI commands.
2
3       Each test runs against a throwaway config + SQLite DB in tmp_path, so the real
4       data/ directory is never touched. Network (yt-dlp) and the browser are mocked.
5       """
6       import csv as _csv
7       import json
8       from types import SimpleNamespace
9       from unittest import mock
10
11      import pytest
12      import yaml
13
14      from src.cli.curate import SportsDataCLI
15      from src.core.schemas.base import VideoMetadata, SportType, LicenseType, CurationStatus
16      from src.core.schemas.badminton import BadmintonRally
17
18
19      @pytest.fixture
20      def cli(tmp_path):
21          cfg = {
22              "database_path": str(tmp_path / "test.db"),
23              "video_storage": {
24                  "local_dir": str(tmp_path / "videos"),
25                  "max_download_size_mb": 100,
26                  "auto_download": False,
27                  "max_height": None,
28              },
29              "commercial_licenses": ["MIT", "Apache-2.0", "BSD", "CC0", "CC-BY", "Public-
Domain", "Proprietary"],
30              "supported_sports": ["badminton", "tennis", "tabletennis", "pickleball",
```

```
"cricket"],
        }
        cfg_path = tmp_path / "config.yaml"
        cfg_path.write_text(yaml.safe_dump(cfg), encoding="utf-8")
        return SportsDataCLI(config_path=str(cfg_path))


    def _import_args(**kw):
        base = dict(
            sport=None, video_path="nonexistent.mp4", annotations_path=None,
            match_name=None, video_id=None, license="Proprietary", fps=30.0,
resolution="1080p",
        )
        base.update(kw)
        return SimpleNamespace(**base)


    def _write_csv(path, header, rows):
        with open(path, "w", newline="", encoding="utf-8") as f:
            w = _csv.writer(f)
            w.writerow(header)
            for r in rows:
                w.writerow(r)


    # ---------------------------------------------------------------------- #
    # import-local: every sport, both JSON and CSV (covers dispatch + parse)       #
    # ---------------------------------------------------------------------- #

    def test_import_local_badminton_json(cli, tmp_path):
        p = tmp_path / "bad.json"
        p.write_text(json.dumps([
```

45. user (2026-06-10T05:39:08.891Z)

```
    def test_status_empty_and_populated(cli, tmp_path):
        cli.status()  # empty branch
        p = tmp_path / "b.json"
        p.write_text(json.dumps([{"rally_number": 1, "start_time": 1.0, "end_time": 2.0}]),
encoding="utf-8")
        cli.import_local(_import_args(sport="badminton", annotations_path=str(p),
video_id="s1"))
        cli.status()  # populated branch


    def test_export_eval_set(cli, tmp_path):
        p = tmp_path / "b.json"
        p.write_text(json.dumps([
            {"rally_number": 1, "start_time": 10.0, "end_time": 20.0},
            {"rally_number": 2, "start_time": 25.0, "end_time": 35.0},
        ]), encoding="utf-8")
        cli.import_local(_import_args(sport="badminton", annotations_path=str(p),
video_id="exp1", license="MIT"))

        out_dir = tmp_path / "out"
        args = SimpleNamespace(out_dir=str(out_dir), sport=None, include_staging=False,
include_noncommercial=False)
        cli.export_eval_set(args)

        manifest = out_dir / "manifest.json"
        assert manifest.exists()
        doc = json.loads(manifest.read_text(encoding="utf-8"))
        assert doc["total_videos"] == 1
        assert doc["total_segments"] == 2
        assert (out_dir / "badminton" / "exp1.csv").exists()
```

```
180
181
182     # ---------------------------------------------------------------------- #
183     # interactive curate loop (mocked input + browser)                       #
184     # ---------------------------------------------------------------------- #
185
186     def _stage_badminton(cli, vid="stg1"):
187         cli.db.insert_video(VideoMetadata(
188             video_id=vid, sport=SportType.BADMINTON, license_type=LicenseType.MIT,
189             is_commercial=True, match_name="Staged Match", status=CurationStatus.STAGING,
190             video_url="https://www.youtube.com/watch?v=abc123",
191         ))
192         cli.db.insert_badminton_rallies([
193             BadmintonRally(video_id=vid, rally_number=1, start_time=10.0, end_time=20.0,
shots_count=5),
194         ])
195
196
197     def test_curate_quit(cli):
198         _stage_badminton(cli)
199         with mock.patch("builtins.input", side_effect=["q"]):
200             cli.curate()
201
202
203     def test_curate_empty(cli):
204         cli.curate()  # no staging entries branch
205
206
207     def test_curate_review_keep(cli):
208         _stage_badminton(cli)
209         with mock.patch("builtins.input", side_effect=["1", "k"]):
210             cli.curate()
211         assert cli.db.get_video("stg1").status == CurationStatus.STAGING
212
213
214     def test_curate_approve(cli):
215         _stage_badminton(cli)
216         with mock.patch("builtins.input", side_effect=["1", "a"]):
217             cli.curate()
218         assert cli.db.get_video("stg1").status == CurationStatus.CURATED
219
220
221     def test_curate_reject(cli):
222         _stage_badminton(cli)
223         with mock.patch("builtins.input", side_effect=["1", "r"]):
224             cli.curate()
225         assert cli.db.get_video("stg1") is None
226
227
228     def test_curate_play_segment(cli):
229         _stage_badminton(cli)
230         with mock.patch("builtins.input", side_effect=["1", "p", "1", "k"]), \
231              mock.patch("webbrowser.open") as wb:
232             cli.curate()
233         wb.assert_called_once()
234
235
236     # ---------------------------------------------------------------------- #
237     # fetch (mocked downloader)                                               #
238     # ---------------------------------------------------------------------- #
239
240     def _fetch_args(**kw):
241         base = dict(
242             sport=None, video_id=None, status="all", min_height=720, max_height=None,
243             force=False, include_noncommercial=False, cookies_from_browser=None,
```

```
244             cookies=None, player_client="default", sleep=0.0, retries=0, retry_backoff=0.0,
245         )
246         base.update(kw)
247         return SimpleNamespace(**base)
248
249
250     def test_fetch_downloads_and_updates_db(cli, tmp_path):
251         _stage_badminton(cli, "fvid")
252         fake = SimpleNamespace(
253             success=True, height=1080, path=str(tmp_path / "videos" / "fvid.mp4"),
```

46. user (2026-06-10T05:39:20.894Z)

```
099cf67 Manual-ingestion: normalize-annotations guard + positioning doc (#13)
0e4ccc7 Merge pull request #12 from avidullu/feat/collector-report-refinements
b8d40c8 feat(reporting): delivery-report debuggability refinements + faq-resolution golden test
94fec51 Merge pull request #11 from avidullu/feat/observability-reports
038424d feat(reporting): per-video observability report (collector)
99dfa22 Merge pull request #10 from avidullu/fix/canonical-csv-zero-start
f7cab11 fix(parser): keep a rally that starts at 0.0s in load_canonical_csv
d68c5da Merge pull request #9 from avidullu/feat/validate-delivery
2882a03 feat(ingest): validate-delivery ? per-video QA gate + structured report
23f2ed0 Merge pull request #8 from avidullu/feat/last-shot-outcome
 M data/sports_metadata.db
---
{
  "commercial_only": true,
  "statuses": [
    "curated",
    "staging"
  ],
  "total_videos": 24,
  "total_segments": 1915,
  "eval_sets": [
    {
      "video_id": "my_custom_local_match",
      "sport": "badminton",
      "csv": "badminton/my_custom_local_match.csv",
      "local_path": "data/videos/my_custom_match.mp4",
      "video_url": null,
      "match_name": "My Custom Local Match",
      "fps": 30.0,
      "resolution": "1080p",
      "license_type": "Proprietary",
      "is_commercial": true,
      "status": "curated",
      "num_segments": 2
    },
    {
      "video_id": "shuttleset_21",
      "sport": "badminton",
      "csv": "badminton/shuttleset_21.csv",
      "local_path": "./data/videos\\shuttleset_21.mp4",
      "video_url": "https://www.youtube.com/watch?v=gloiZ_gTJaE",
      "match_name": "An Se Young Ratchanok Intanon YONEX Thailand Open 2021 QuarterFinals",
      "fps": 30.0,
      "resolution": "1080p",
      "license_type": "MIT",
      "is_commercial": true,
      "status": "curated",
      "num_segments": 75
    },
    {
      "video_id": "shuttleset_1",
      "sport": "badminton",
```

```
---
data/eval_export:
badminton
manifest.json

data/videos:
_diag.mp4.part
_diag.mp4.part-Frag320.part
_diag.mp4.ytdl
_t.mp4.part
_t.mp4.part-Frag447.part
_t.mp4.ytdl
_v.mp4.part
_v.mp4.part-Frag726.part
_v.mp4.ytdl
shuttleset_1.mp4
shuttleset_10.mp4
shuttleset_11.mp4
shuttleset_13.mp4
shuttleset_14.mp4
shuttleset_15.mp4
shuttleset_16.mp4
shuttleset_17.mp4
shuttleset_18.mp4
shuttleset_19.mp4
shuttleset_2.mp4
shuttleset_20.mp4
shuttleset_21.mp4
shuttleset_22.mp4
shuttleset_23.mp4
shuttleset_24.mp4
shuttleset_25.mp4
shuttleset_3.mp4
shuttleset_4.mp4
shuttleset_6.mp4
shuttleset_8.mp4
shuttleset_9.mp4
```

47. user (2026-06-10T05:39:50.793Z)

```
master
 data/sports_metadata.db | Bin 184320 -> 200704 bytes
 1 file changed, 0 insertions(+), 0 deletions(-)
```

---------- #

```
@pytest.mark.parametrize("argv,method", [
    (["prog", "status"], "status"),
    (["prog", "curate"], "curate"),
    (["prog", "export"], "export_eval_set"),
    (["prog", "fetch"], "fetch"),
])
def test_main_dispatch(argv, method):
    from src.cli import curate as curate_mod
    with mock.patch.object(curate_mod, "SportsDataCLI") as M, mock.patch("sys.argv", argv):
        curate_mod.main()
        getattr(M.return_value, method).assert_called_once()


def test_main_import_local_dispatch():
    from src.cli import curate as curate_mod
    argv = ["prog", "import-local", "--sport", "badminton", "--video-path", "v.mp4", "--
annotations-path", "a.json"]
    with mock.patch.object(curate_mod, "SportsDataCLI") as M, mock.patch("sys.argv", argv):
        curate_mod.main()
```

```
        M.return_value.import_local.assert_called_once()


def test_main_no_command_prints_help():
    from src.cli import curate as curate_mod
    with mock.patch.object(curate_mod, "SportsDataCLI"), mock.patch("sys.argv", ["prog"]):
        curate_mod.main()  # falls through to print_help()
```

48. user (2026-06-10T05:40:09.352Z)

```
2026-06-09 18:05:00.989597200 +0530 200704
Wed Jun 10 11:10:05 IST 2026
0d27836b263ea337971d78782ce7b1e1572f78b6 *data/sports_metadata.db
................................................                     [100%]
124 passed in 2.38s
0d27836b263ea337971d78782ce7b1e1572f78b6 *data/sports_metadata.db
```

49. user (2026-06-10T05:40:30.263Z)

```
1       import os
2       import urllib.request
3       import urllib.parse
4       import csv
5       from typing import Dict, Any, List, Tuple
6       from tabulate import tabulate
7
8       from src.core.config import load_config
9       from src.core.db import SportsDatabase
10      from src.core.schemas.base import VideoMetadata, SportType, LicenseType, CurationStatus
11      from src.core.schemas.badminton import BadmintonRally
12      from src.parsers.badminton.coachai import CoachAIBadmintonParser
13      from src.core.validator import DatasetValidator
14      from src.core.downloader import download_best_video
15
16      class ShuttleSetCrawler:
17          MATCH_CSV_URL = "https://raw.githubusercontent.com/wywyWang/CoachAI-
Projects/main/ShuttleSet/set/match.csv"
18          RAW_BASE_URL = "https://raw.githubusercontent.com/wywyWang/CoachAI-
Projects/main/ShuttleSet/set"
19
20          def __init__(self, config_path: str = "config/config.yaml"):
21              self.config_path = config_path
22              self.config = self._load_config()
23              self.db_path = self.config.get("database_path", "data/sports_metadata.db")
24              self.db = SportsDatabase(self.db_path)
25              self.validator = DatasetValidator(config_path)
26
27              # Load storage config
28              storage = self.config.get("video_storage", {})
29              self.local_dir = storage.get("local_dir", "./data/videos")
30              self.max_download_size_mb = storage.get("max_download_size_mb", 1000)
31              self.auto_download = storage.get("auto_download", True)
32              # Optional resolution cap (e.g. 1080 to avoid pulling 4K). Unset = best
available.
33              self.max_height = storage.get("max_height")
34
35              # Load ingestion limits
36              limits = self.config.get("ingestion_limits", {})
37              self.limit_max_total_size_mb = limits.get("max_total_download_size_mb", 5000)
38              self.limit_max_video_count = limits.get("max_total_video_count", 25)
39
40              os.makedirs(self.local_dir, exist_ok=True)
41              os.makedirs("data/temp", exist_ok=True)
42
43          def _load_config(self) -> Dict[str, Any]:
```

```
44              return load_config(self.config_path)
45
46          def fetch_and_curate(self):
47              """Downloads ShuttleSet metadata, crawls match videos, parses rallies, and
enforces limits."""
48              print("Starting ShuttleSet dataset ingestion...")
49              print(f"Ingestion limits: Max Videos={self.limit_max_video_count}, Max Total
Size={self.limit_max_size_mb_str()}")
50
51              # 1. Download match.csv metadata
52              temp_match_csv = "data/temp/shuttleset_match.csv"
53              try:
54                  print(f"Downloading match catalog from {self.MATCH_CSV_URL}...")
55                  urllib.request.urlretrieve(self.MATCH_CSV_URL, temp_match_csv)
56              except Exception as e:
57                  print(f"Error fetching match list: {e}")
58                  return
59
60              # Read matches
61              matches: List[Dict[str, Any]] = []
62              with open(temp_match_csv, mode='r', encoding='utf-8') as f:
63                  reader = csv.DictReader(f)
64                  for row in reader:
65                      matches.append(row)
66
67              # 2. Iterate through matches
68              videos_downloaded = 0
69              total_size_downloaded_mb = 0.0
70              total_rallies_indexed = 0
71              db_videos_added = 0
72              db_videos_updated = 0
73
74              # Detailed summary logs
75              summary_log: List[Dict[str, Any]] = []
76
77              print(f"Found {len(matches)} matches in ShuttleSet catalog.")
78
79              for idx, match in enumerate(matches):
80                  # Check limits
81                  if videos_downloaded >= self.limit_max_video_count:
82                      print(f"\nLimit reached: Maximum video count
({self.limit_max_video_count}) reached. Stopping ingestion.")
83                      break
84                  if total_size_downloaded_mb >= self.limit_max_total_size_mb:
85                      print(f"\nLimit reached: Maximum download size
({self.limit_max_total_size_mb} MB) reached. Stopping.")
86                      break
87
88                  video_dir_name = match.get("video")
89                  youtube_url = match.get("url")
90                  match_id = match.get("id")
91
92                  # Skip catalog rows missing the fields we depend on rather than
93                  # crashing the entire ingestion run.
94                  if not video_dir_name or not match_id:
95                      print(f"\n[{idx+1}/{len(matches)}] Skipping catalog row with missing
'video'/'id' column.")
96                      continue
97
98                  try:
99                      set_count = int(float(str(match.get("set", "2")).strip() or "2"))
100                 except ValueError:
101                     set_count = 2
102                 match_name = video_dir_name.replace("_", " ")
103                 video_id = f"shuttleset_{match_id}"
```

```
                # ShuttleSet is MIT licensed -> commercially friendly
                license_type = LicenseType.MIT
                is_commercial = True

                print(f"\n[{idx+1}/{len(matches)}] Processing: {match_name}")
                print(f"  YouTube URL: {youtube_url}")

                # Download video files (if configured)
                local_path = None
                download_size_mb = 0.0

                probed_resolution = None
                probed_fps = None
                if self.auto_download and youtube_url:
                    # 2.1 Download the best-resolution stream available (MP4). The shared
                    # downloader sorts formats by resolution and records the actual
                    # width/height/fps it obtained ? we never assume a resolution.
                    print("  Downloading best-resolution video...")
                    video_filename = f"{video_id}.mp4"
                    dest_path = os.path.join(self.local_dir, video_filename)

                    result = download_best_video(
                        youtube_url, dest_path,
                        max_height=self.max_height,
                        max_size_mb=self.max_download_size_mb,
                    )
                    if result.success:
                        local_path = result.path
                        download_size_mb = result.size_mb
                        probed_resolution = result.resolution_label
                        probed_fps = result.fps
                        videos_downloaded += 1
                        total_size_downloaded_mb += download_size_mb
                        print(f"  Successfully downloaded video "
                              f"({download_size_mb:.2f} MB, {probed_resolution or 'unknown
res'}"
                              f"{f', {probed_fps:g}fps' if probed_fps else ''})")
                    else:
                        print(f"  Failed to download video: {result.error}")
                        local_path = None
                        download_size_mb = 0.0

                # 2.2 Download & Merge stroke CSV files for this match
                print("  Fetching annotation files...")
                rallies_merged: List[BadmintonRally] = []

                # Temporary storage to merge set csvs
                merged_csv_path = f"data/temp/{video_id}_merged.csv"
                headers = ["set", "rally", "ball_round", "time", "type", "score_a",
"score_b"]

                with open(merged_csv_path, 'w', newline='', encoding='utf-8') as mf:
                    writer = csv.writer(mf)
                    writer.writerow(headers)

                    for s in range(1, set_count + 1):
                        encoded_dir = urllib.parse.quote(video_dir_name)
                        set_url = f"{self.RAW_BASE_URL}/{encoded_dir}/set{s}.csv"
                        set_temp_path = f"data/temp/{video_id}_set{s}.csv"
                        try:
                            urllib.request.urlretrieve(set_url, set_temp_path)

                            # Read and write rows, adding set number
                            with open(set_temp_path, 'r', encoding='utf-8') as sf:
```

```
                                sreader = csv.DictReader(sf)
                                # Normalize fieldnames
                                sreader.fieldnames = [n.lower().strip() for n in
sreader.fieldnames] if sreader.fieldnames else []

                                for row in sreader:
                                    # Write to merged file
                                    writer.writerow([
                                        s,
                                        row.get("rally", "0"),
                                        row.get("ball_round", row.get("ball_seq", "1")),
                                        row.get("time", row.get("timestamp", "0.0")),
                                        row.get("type", ""),
                                        row.get("score_a", row.get("round_score_a", "0")),
                                        row.get("score_b", row.get("round_score_b", "0"))
                                    ])
                            # Clean up set CSV
                            os.remove(set_temp_path)
                        except Exception as e:
                            print(f"  Warning: Failed to fetch Set {s} CSV: {e}")

                # 2.3 Parse merged annotations
                rallies = []
                video_meta = None
                if os.path.exists(merged_csv_path):
                    parser = CoachAIBadmintonParser(start_buffer=1.0, end_buffer=2.0)
                    try:
                        video_meta, rallies = parser.parse(
                            merged_csv_path,
                            video_id=video_id,
                            video_url=youtube_url,
                            license=license_type.value,
                            match_name=match_name,
                            local_path=local_path
                        )
                        # Clean up merged CSV
                        os.remove(merged_csv_path)
                    except Exception as e:
                        print(f"  Error parsing merged CSV: {e}")
                        rallies = []

                # Override the parser's placeholder resolution/fps with what we actually
                # downloaded, so the DB reflects the real file (the indexer validates
                # resolution and keys by it).
                if video_meta is not None:
                    if probed_resolution:
                        video_meta.resolution = probed_resolution
                    if probed_fps:
                        video_meta.fps = probed_fps

                if not video_meta or not rallies:
                    print("  Skipping match due to missing metadata or rallies.")
                    if local_path and os.path.exists(local_path):
                        os.remove(local_path)
                        # Roll back download accounting for the now-discarded file so
                        # the limit checks and summary stay accurate.
                        videos_downloaded -= 1
                        total_size_downloaded_mb -= download_size_mb
                    continue

                # 2.4 Validate rallies
                is_valid, errors = self.validator.validate_badminton_rallies(rallies)
                if not is_valid:
                    print(f"  Warning: Rallies failed validation. Skipping DB insertion.")
                    for err in errors:
```

```
                        print(f"    - {err}")
                    if local_path and os.path.exists(local_path):
                        os.remove(local_path)
                        # Roll back download accounting for the now-discarded file.
                        videos_downloaded -= 1
                        total_size_downloaded_mb -= download_size_mb
                    continue

                # 2.5 Insert to Database
                # Update metrics
                existing = self.db.get_video(video_id)
                if existing:
                    db_videos_updated += 1
                else:
                    db_videos_added += 1

                self.db.insert_video(video_meta)
                count = self.db.insert_badminton_rallies(rallies)
                total_rallies_indexed += count

                print(f"  Successfully indexed {count} rallies in SQLite database.")

                summary_log.append({
                    "match_name": match_name,
                    "video_id": video_id,
                    "downloaded": local_path is not None,
                    "size_mb": download_size_mb,
                    "rallies": count
                })

            # 3. Print execution summary
            self._print_execution_summary(
                videos_downloaded,
                total_size_downloaded_mb,
                total_rallies_indexed,
                db_videos_added,
                db_videos_updated,
                summary_log
            )

            # Cleanup temp directory (best-effort).
            try:
                for f in os.listdir("data/temp"):
                    os.remove(os.path.join("data/temp", f))
                os.rmdir("data/temp")
            except OSError:
                pass

        def limit_max_size_mb_str(self) -> str:
            return f"{self.limit_max_total_size_mb} MB"

        def _print_execution_summary(self,
                                     videos_downloaded: int,
                                     total_size_mb: float,
                                     total_rallies: int,
                                     added: int,
                                     updated: int,
                                     log: List[Dict[str, Any]]):
            print("\n" + "="*50)
            print("             INGESTION EXECUTION SUMMARY            ")
            print("="*50)
            print(f"Total Video Files Downloaded: {videos_downloaded}")
            print(f"Total Download Size         : {total_size_mb:.2f} MB")
            print(f"Total Rallies Indexed       : {total_rallies}")
            print(f"Database Matches Added      : {added}")
```

```
296             print(f"Database Matches Updated    : {updated}")
297             print("-"*50)
298
299             if log:
300                 print("Match Log Breakdown:")
301                 table_data = []
302                 for item in log:
303                     table_data.append([
304                         item["match_name"],
305                         item["video_id"],
306                         "Yes" if item["downloaded"] else "No",
307                         f"{item['size_mb']:.2f} MB",
308                         item["rallies"]
309                     ])
310                 print(tabulate(table_data, headers=["Match Name", "Video ID", "Downloaded",
"File Size", "Rallies"], tablefmt="simple"))
311             else:
312                 print("No matches were successfully processed.")
313             print("="*50 + "\n")
314
315     def main():
316         crawler = ShuttleSetCrawler()
317         crawler.fetch_and_curate()
318
319     if __name__ == "__main__":
320         main()
321
```

50. user (2026-06-10T05:41:35.198Z)

```
.
..
.coverage
.git
.gitattributes
.gitignore
.pytest_cache
.sixth
README.md
__pycache__
cleanup.py
config
data
docs
pyproject.toml
src
tests
NO CI
12:    parser.add_argument("--license", default="MIT", help="Dataset license type")
31:                license=args.license,
42:            print(f"  - License: {video_meta.license_type.value}")
```

51. assistant (2026-06-10T05:42:27.899Z)

```
All evidence gathered. Compiling the audit now.
```

52. user (2026-06-10T05:44:07.674Z)

```
Structured output provided successfully
```

53. assistant (2026-06-10T05:44:08.382Z)

```
You've hit your session limit · resets 3:50pm (Asia/Calcutta)
```